



机器狗避障

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备（1学时）
- 四、工程文件目录结构
- 五、模型本地化训练及部署（2学时）
- 六、逻辑实现（5学时）
- 七、运行示例

目录

一、实验准备

1.1实验描述

1.2实验目的

1.3实验建议

1.4实验环境

1.5背景知识 MindSpore介绍

1.6.实验道具讲解

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

1.1 实验描述

本实验的目标是使用MindSpore对各种障碍物进行识别，如楼梯，狗洞、路障标识物，机器狗会根据不同障碍物做出不同的避障反应；当机器狗识别到楼梯后，机器狗步态会转变成高踏步模式，离开楼梯后会按照规划路线继续以正常步态行走；当机器狗识别到狗洞后，机器狗会从正常姿态转变为匍匐姿态，直到通过狗洞；当机器狗识别到路障标识物后，会在距离路障标识物一定距离后，偏转一定方向继续前进，直到越过路障标识物后，按照规定路线继续前进。

MindSpore是华为开源的新一代全场景AI计算框架，用于支持包括机器学习、深度学习在内的AI应用开发，提供灵活的编程范式和丰富的算法库。在实验过程中，我们收集大量障碍物数据集，对预训练权重进行模型训练和调优，并最终转换模型格式，将其在昇腾开发板上进行推理。通过海康工业相机和华为昇腾开发板结合，实现机器狗避障功能。

目录

一、实验准备

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识 MindSpore 介绍

1.6 实验道具讲解

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

1.2 实验目的

- 了解实现机器狗避障的原理和方法
- 掌握数据集的标注方法
- 学会如何将模型应用于机器狗上
- 如何使用华为云MindSpore进行模型训练

目录

一、实验准备

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识 MindSpore介绍

1.6. 实验道具讲解

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

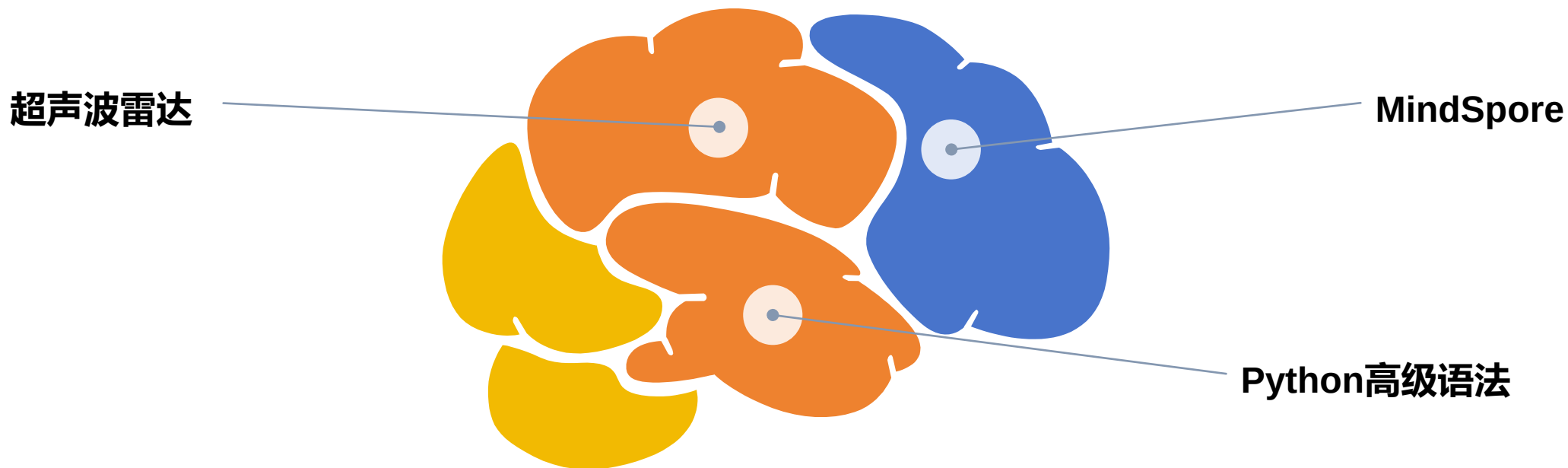
五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

1.3 实验建议

本实验主要是在MindSpore上用Python编程，模型转换过程可以在自己的PC上进行，程序执行则需要华为昇腾Atlas 200I DK开发板（Linux）上执行。实验中未注明的执行地方的，一律在开发板上执行。



目录

一、实验准备

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识 MindSpore介绍

1.6 实验道具讲解

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

1.4实验环境——实验硬件设备

本实验需要的准备的硬件如下：（详见章节3.1、3.4）

1



机器狗

2



PC

3



昇腾开发板

4

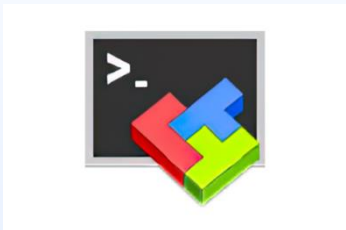


读卡器

1.4实验环境——实验软件环境

本实验需要的准备的软件环境如下：（详见章节3.2、3.3）

1



MobaXterm

2



Pycharm

3



MindSpore

目录

一、实验准备

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识 MindSpore介绍

1.6 实验道具讲解

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

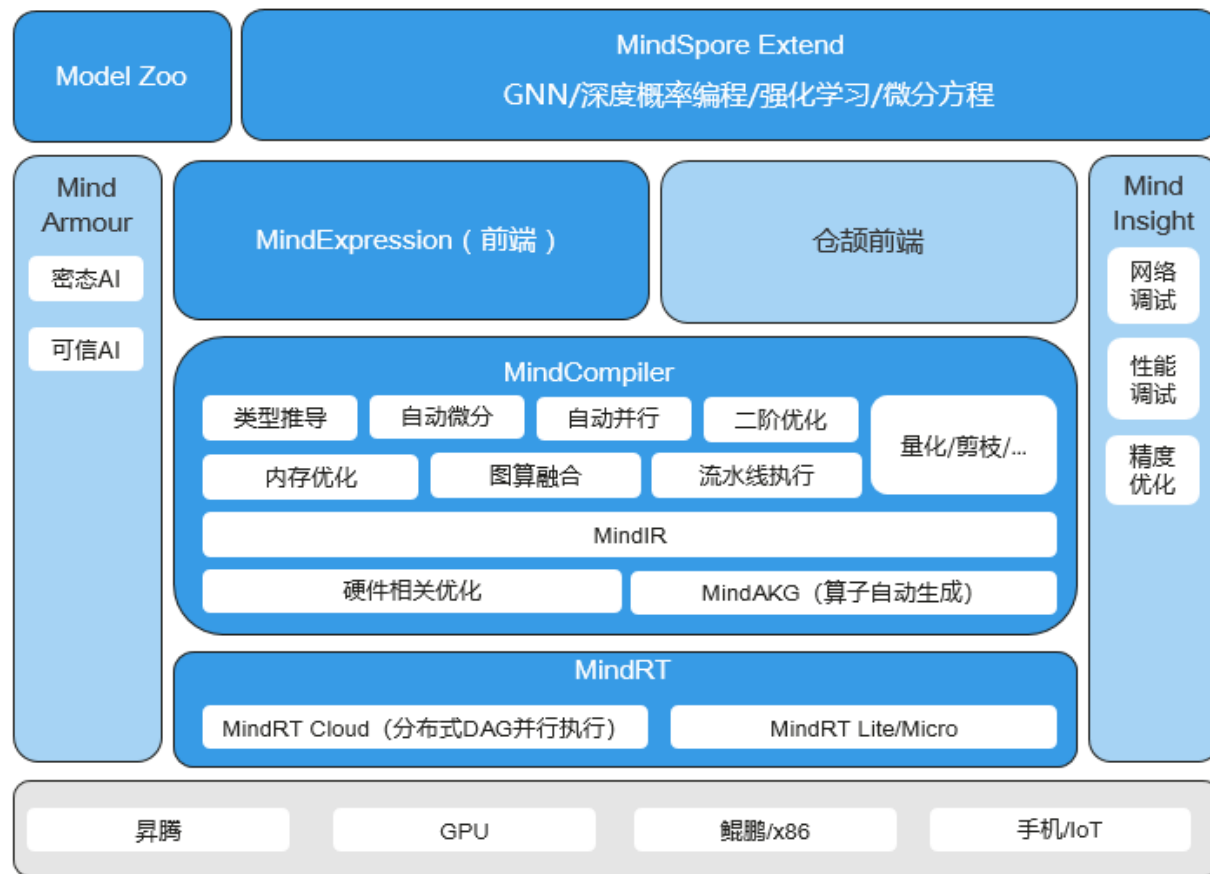
五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

1.5 背景知识 MindSpore介绍

MindSpore是一个全场景深度学习框架，旨在实现易开发、高效执行、全场景覆盖三大目标，其中易开发表现为API友好、调试难度低，高效执行包括计算效率、数据预处理效率和分布式训练效率，全场景则指框架同时支持云、边缘以及端侧场景。



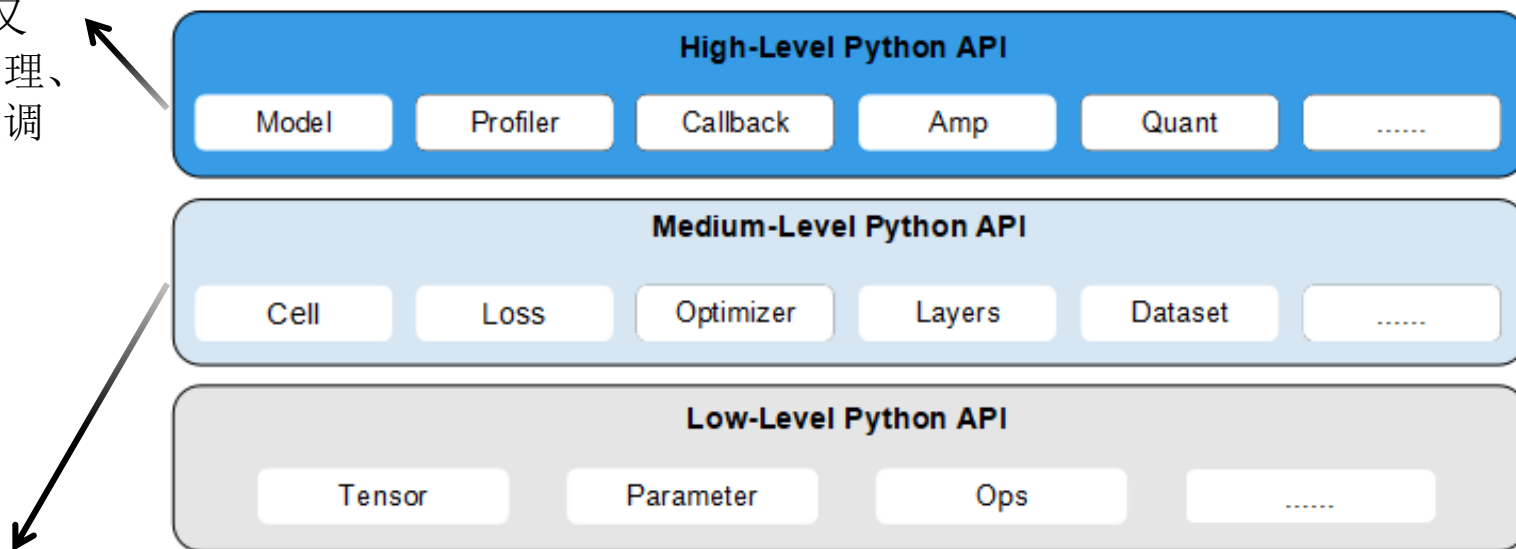
1.5 背景知识 MindSpore介绍

MindSpore的层次结构：

MindSpore向用户提供了3个不同层次的API，支撑用户进行网络构建、整图执行、子图执行以及单算子执行，从低到高分别为Low-Level Python API、Medium-Level Python API以及High-Level Python API。

第三层为高阶API，其在中阶API的基础上又提供了训练推理的管理、混合精度训练、调试调优等

第二层为中阶API，其封装了低阶API，提供网络层、优化器、损失函数等模块



第一层为低阶API，主要包括张量定义、基础算子、自动微分等模块

目录

一、实验准备

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识 MindSpore 介绍

1.6 实验道具讲解

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

1.6实验道具讲解

障碍物（雷达避障需要用到的道具）： 狗洞：

橡塑锥形路锥

产品规格

60CM*30

产品重量

1KG

温馨提示：

以上尺寸纯人工测量，纯在细微误差属于正常现象，敬请见谅！

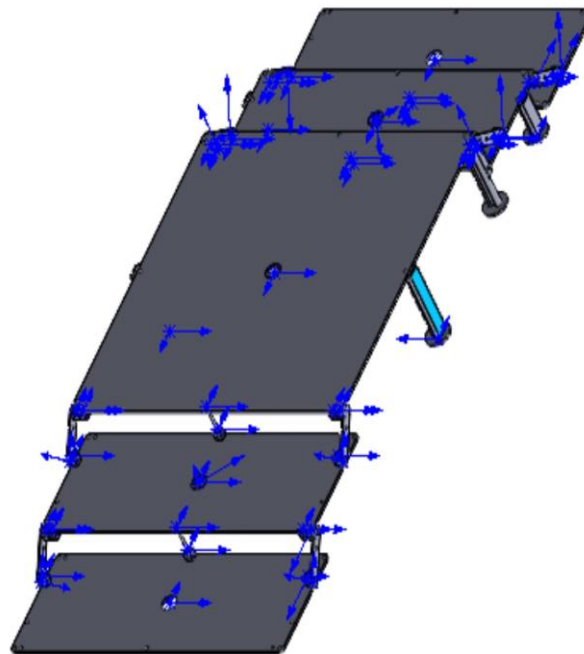


在场景中，通过超声波雷达来检测。



用来模拟需要俯身钻入的障碍物。

楼梯：



模拟在工业巡检场景中会出现的爬楼梯情景。

目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

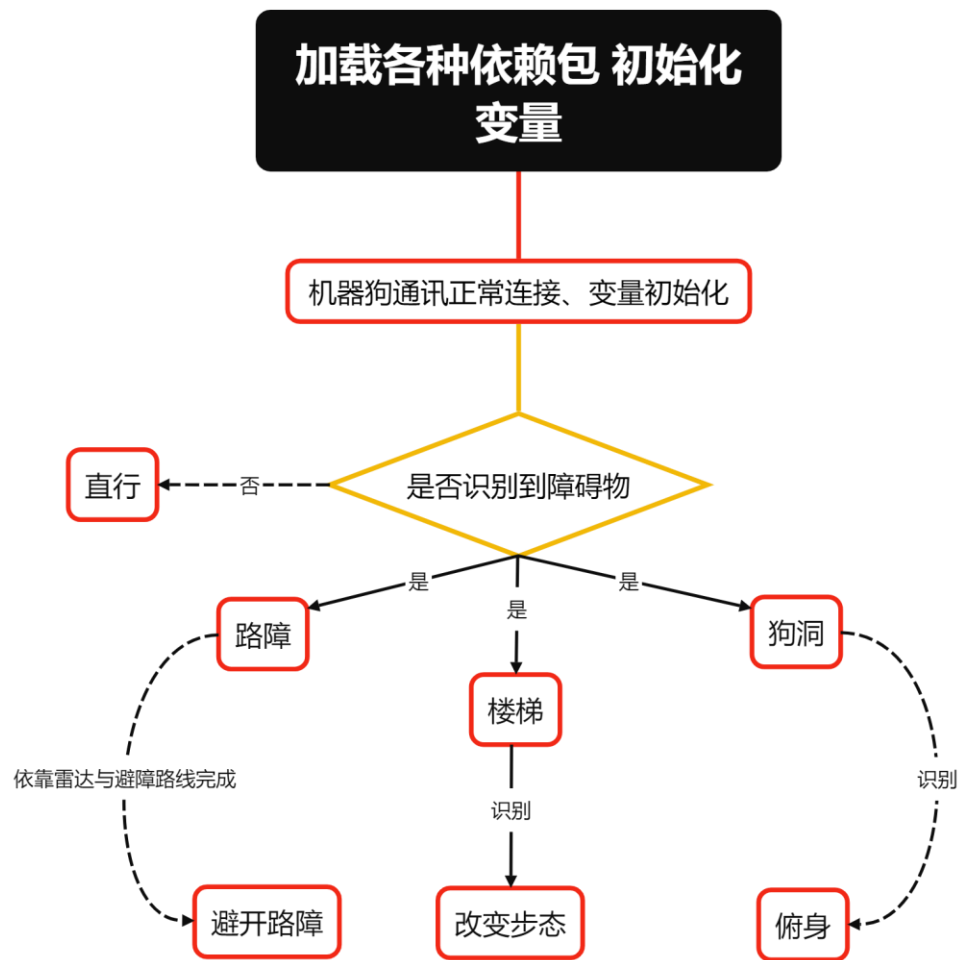
四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

2.实验过程描述



目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

3.1 硬件环境准备

3.2 下载mindyolo项目代码

3.3 创建实验环境

3.4 安装环境依赖

3.5 开发板环境（机器狗网址配置）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

3.1 硬件环境准备

(1) SD卡中系统烧录

向SD卡中烧录镜像的步骤如下：

- ① 将SD卡插入读卡器，再将读卡器插到PC上。



3.1 硬件环境准备

(1) SD卡中系统烧录

②在浏览器中进入Ascend官网，选择“产品”->“开发者套件”



3.1 硬件环境准备

(1) SD卡中系统烧录

③点击“资源”，进入资源下载界面



3.1 硬件环境准备

(1) SD卡中系统烧录

④选择“制卡工具下载”，下载完成后进行安装。

寻你所需，玩转AI开发_

快速入门



准备

请提前做好下列物品，或查看 [硬件准备清单](#)，快速上手不卡顿

- MicroSD卡（推荐64GB）及读卡器（可选）
- 获取制卡工具，一键烧录镜像完成开发环境准备

[↓ 制卡工具下载](#) [↓ 校验文件下载](#)



入门

提供入门 Atlas 200I DK A2 的全流程指导，帮助你30分钟内轻松完成从启动开发者套件到运行首样例的过程

[▶ 快速开始（课程）](#) [📖 快速开始（文档）](#)



参考工具

提供Atlas 200I DK A2 进一步开发所需的参考工具及配套资源，可按需下载

模型适配工具

使能开发者0代码完成数据标注、模型训练、模型部署到昇腾开发者套件

[↓ 软件包下载](#) [↓ 校验文件下载](#)

配套Conda环境包

模型适配工具运行所必须的配套conda依赖环境，需要提前安装conda并导入conda环境包

[↓ 软件包下载](#) [↓ 校验文件下载](#)

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Page 23

 HUAWEI

3.1 硬件环境准备

(1) SD卡中系统烧录

⑤打开镜像烧录工具Ascend AI Devkit Imager，选择Atlas 200I DK A2



3.1 硬件环境准备

(1) SD卡中系统烧录

⑥打开“配置网络信息”，选择“Type-C”选项卡。可以看到Type-C接口默认IP为192.168.0.2。选择默认IP。

镜像网络信息配置

×

以下为镜像默认IP信息，您可以根据PC/路由器IP信息重新填写网络信息

网口信息

ETH0

ETH1

Type-C

☐ 自动获取IP地址

☒ 使用下面的IP地址

IP地址

192 · 168 · 0 · 2

子网掩码

255 · 255 · 255 · 0

默认网关

· · ·

首选DNS服务器

8 · 8 · 8 · 8

备用DNS服务器

114 · 114 · 114 · 114

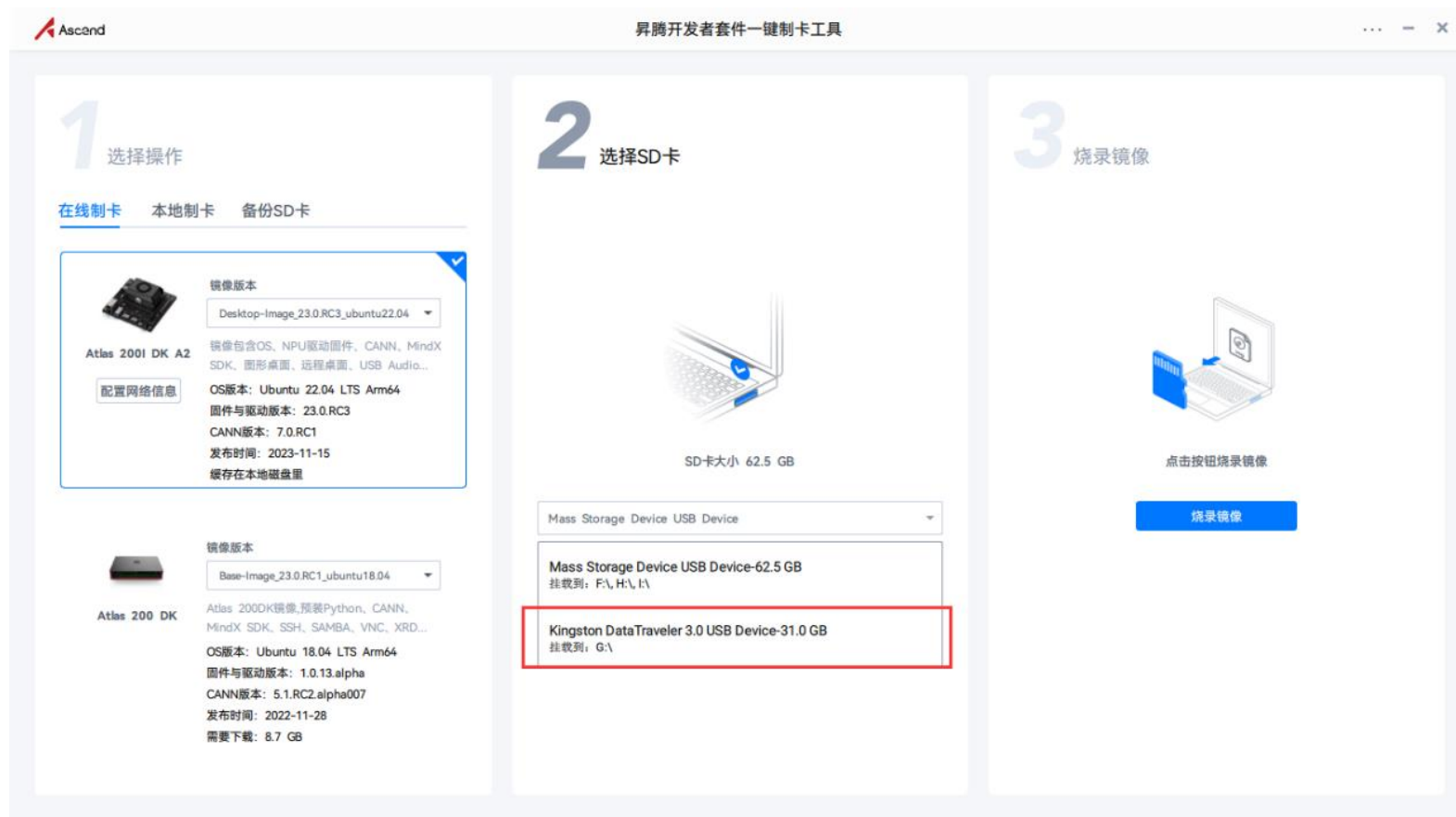
确定

取消

3.1 硬件环境准备

(1) SD卡中系统烧录

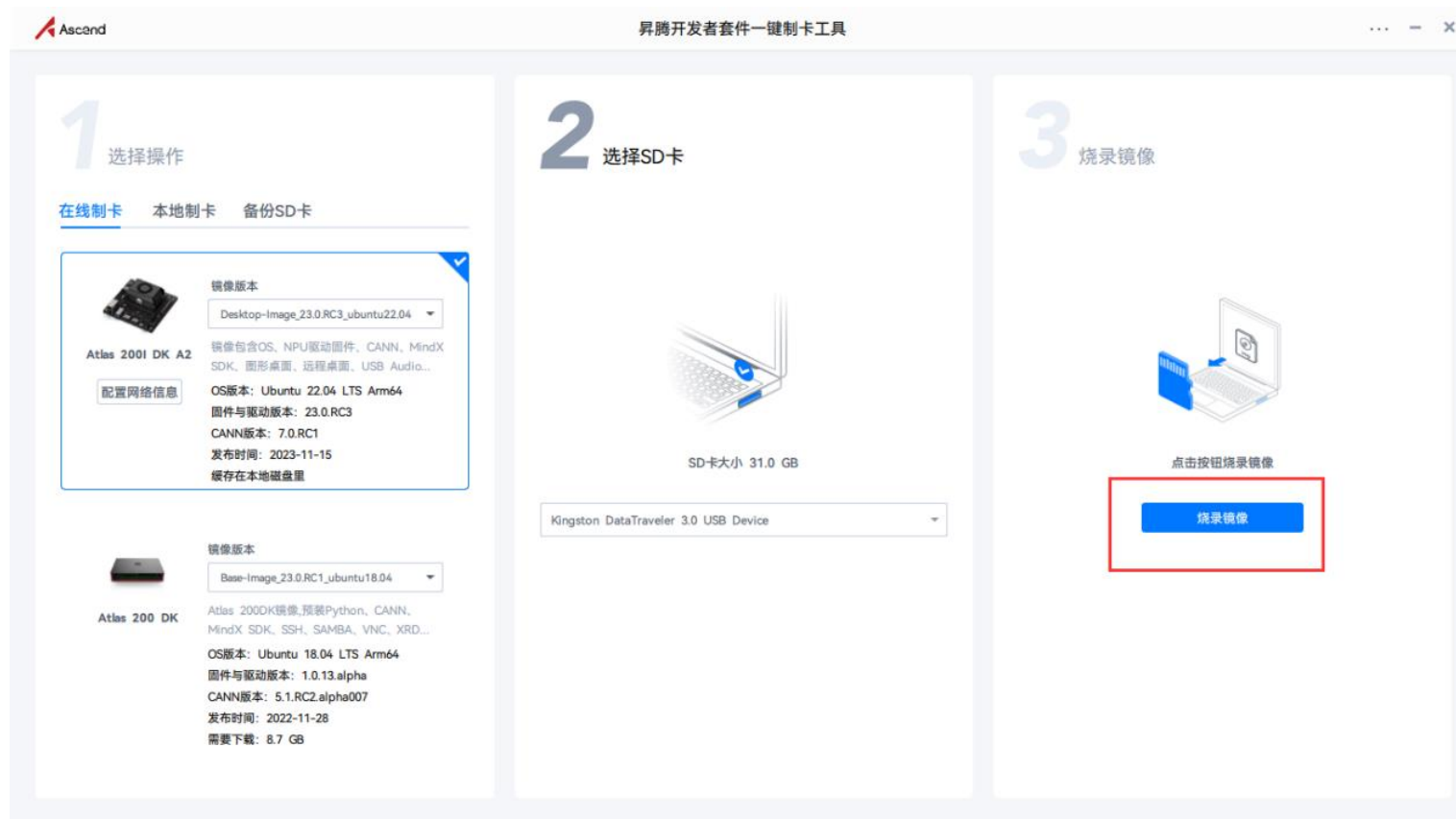
⑦选择要烧录的SD卡。



3.1 硬件环境准备

(1) SD卡中系统烧录

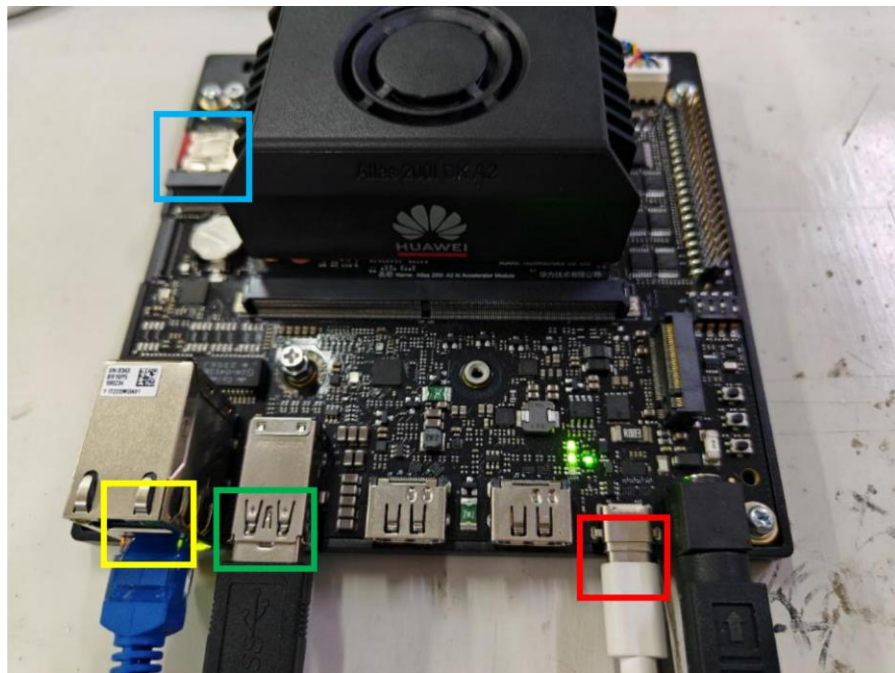
⑧点击“烧录镜像”，等待镜像烧录完成。



3.1 硬件环境准备

(2) 开发板连接

将烧录好系统的SD卡插入到开发板中（下方蓝色框中），昇腾开发板和PC机通过Type-C线连接，将Type-C线和电源线连接到开发板（下方红色框中），并将海康工业相机和机器狗的USB线连接到开发板（下方绿色框中）。同时可以连接网线（下方黄色框中），让开发版连接互联网，方便环境依赖包的下载。连接方式如下图所示。



3.1 硬件环境准备

(3) PC端远程连接开发板

当开发者套件通过Type-C接口和PC连接时，如何在PC将接口 IP地址修改为和开发者套件接口 IP地址同一个网段（如开发者套件Type-C 接口为192.168.0.2，PC对应USB或Type-C接口为192.168.0.101），并使用SSH工具远程登录开发者套件。

1) 安装Windows的USB网卡驱动（以Windows 10系统为例）

①在PC上右键单击“此电脑”，选择“更多”→“管理”，进入“计算机管理”界面。

②在“计算机管理”操作界面中选择“设备管理器→“其他设备”，如图所示，RNDIS为未识别状态。

注意：如果未出现RNDIS，请按以下方法排查：

确保开发者套件Type-C接口和PC已通过连接线正常连接。

更换具备数据传输能力的Type-C数据线（一般手机充电线拥有数据传输能力），继续查看是否出现RNDIS。

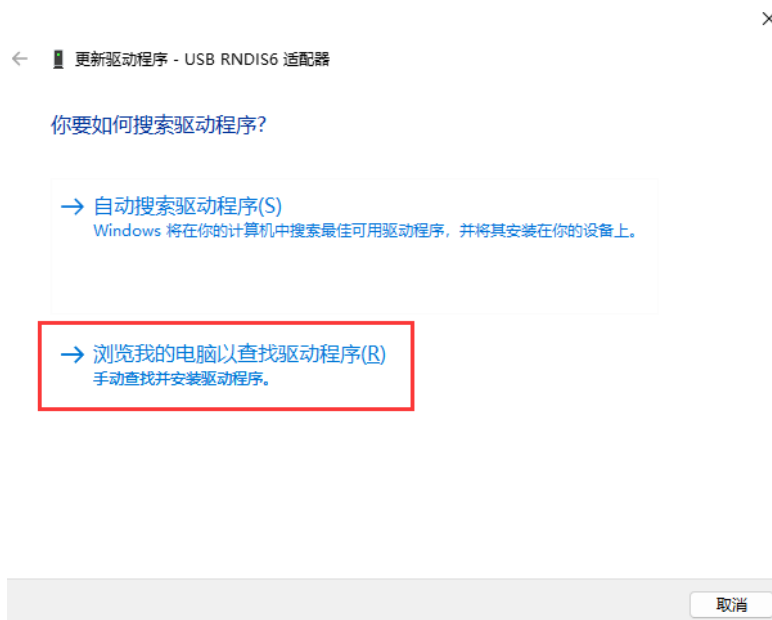
3.1 硬件环境准备

(3) PC端远程连接开发板

如果尝试现场所有Type-C数据线，仍无法出现RNDIS，则考虑使用 RJ45网线连接PC和开发者套件的以太网口的方式登录开发者套件。

③右键单击“RNDIS”，选择“更新驱动程序(P)”。

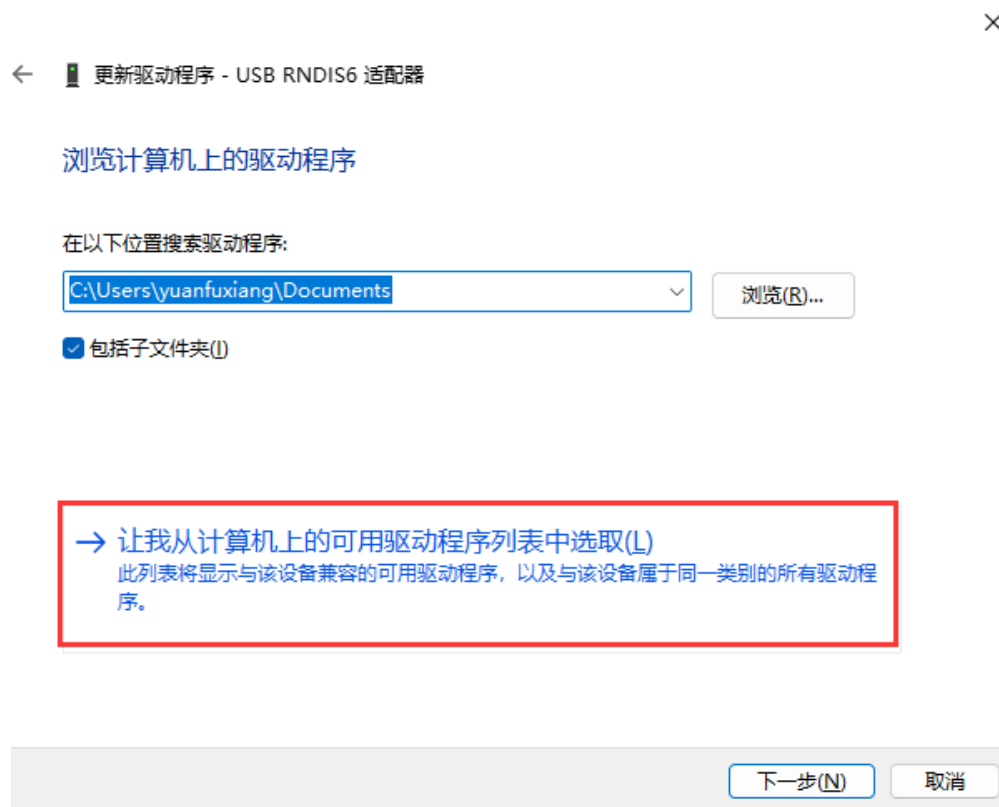
④在弹出的“更新驱动程序”→“RNDIS窗口”中选择“浏览我的计算机以查找驱动程序软件(R)”。



3.1 硬件环境准备

(3) PC端远程连接开发板

⑤然后选择“让我从计算机上的可用驱动程序列表中选择(L)”。



3.1 硬件环境准备

(3) PC端远程连接开发板

⑥在“常见硬件类型”列表中选择“网络适配器”，单击“下一页(N)”。



3.1 硬件环境准备

(3) PC端远程连接开发板

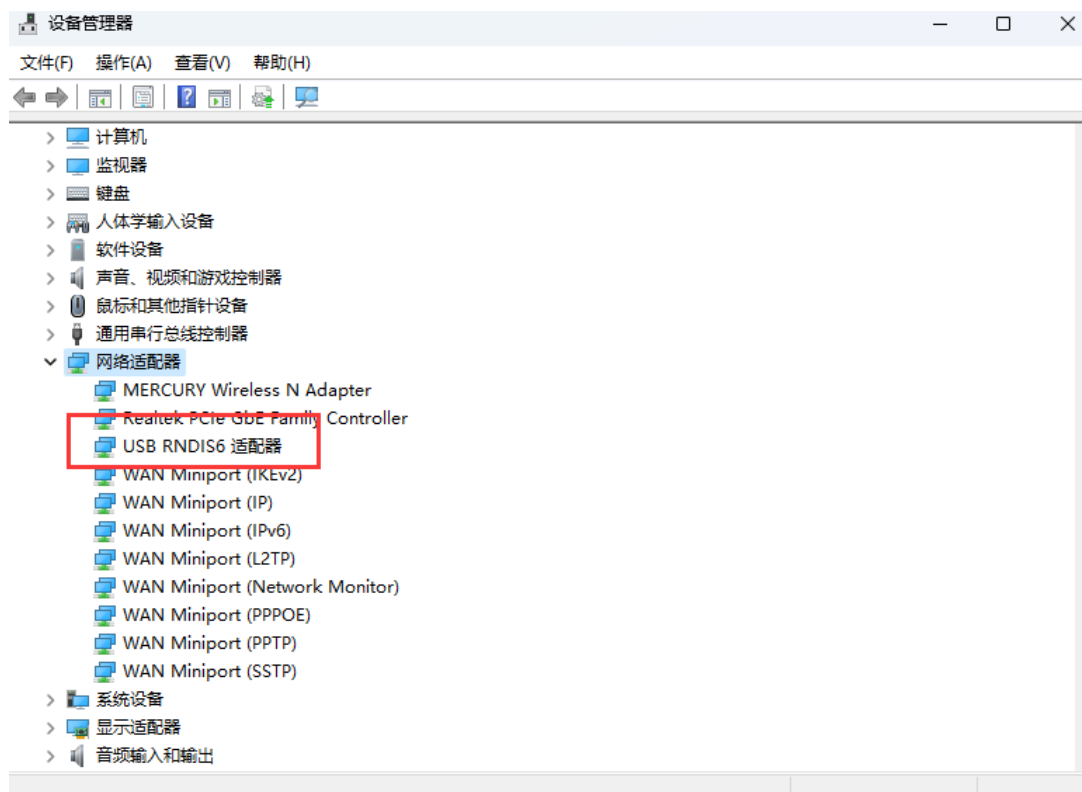
⑦在“选择要为此硬件安装的设备驱动程序”界面中选择“Microsoft”厂商的“USB RNDIS6适配器”。



3.1 硬件环境准备

(3) PC端远程连接开发板

- ⑧单击“下一页”，在弹出的“更新驱动程序警告”窗口选择“是”。
- ⑨返回“设备管理器”→“网络适配器”，可看到已经正常显示了USB RNDIS6适配器的驱动。



3.1 硬件环境准备

(4) 配置PC接口IP地址

本步骤以 Windows 10 系统为例。

①在 PC 上打开“开始”，单击“设置”按钮，进入“Windows 设置”界面。

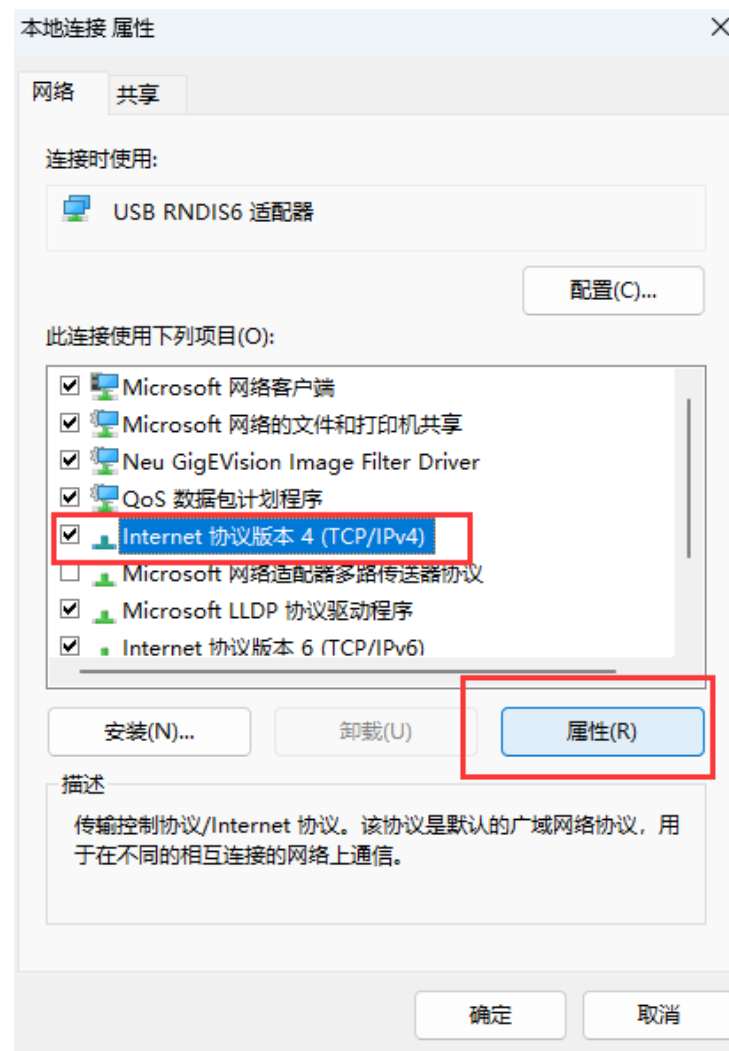
②选择“网络和 Internet”，单击“更改适配器选项”。

③鼠标右键单击“本地连接”后鼠标左键单击“属性”进入“本地连接 属性”界面（使用 Type-C 接口连接时一般为“本地连接 x”，x 为数字，以实际显示的数字为准。

3.1 硬件环境准备

(4) 配置PC接口IP地址

④选择 “Internet协议版本4 (TCP/IPv4) ” ， 单击 “属性” 。



3.1 硬件环境准备

(4) 配置PC接口IP地址

⑤勾选“使用下面的IP地址”选项，填写IP地址（图示以192.168.0.101为例）和子网掩码，默认网关与DNS服务器地址为空，单击“确定”保存。

Internet 协议版本 4 (TCP/IPv4) 属性

常规

如果网络支持此功能，则可以获取自动指派的 IP 设置。否则，你需要从网络系统管理员处获得适当的 IP 设置。

☐ 自动获得 IP 地址(O)

☒ 使用下面的 IP 地址(S):

IP 地址(I): 192 . 168 . 0 . 101

子网掩码(U): 255 . 255 . 255 . 0

默认网关(D): . . .

☐ 自动获得 DNS 服务器地址(B)

☒ 使用下面的 DNS 服务器地址(E):

首选 DNS 服务器(P): . . .

备用 DNS 服务器(A): . . .

☐ 退出时验证设置(L)

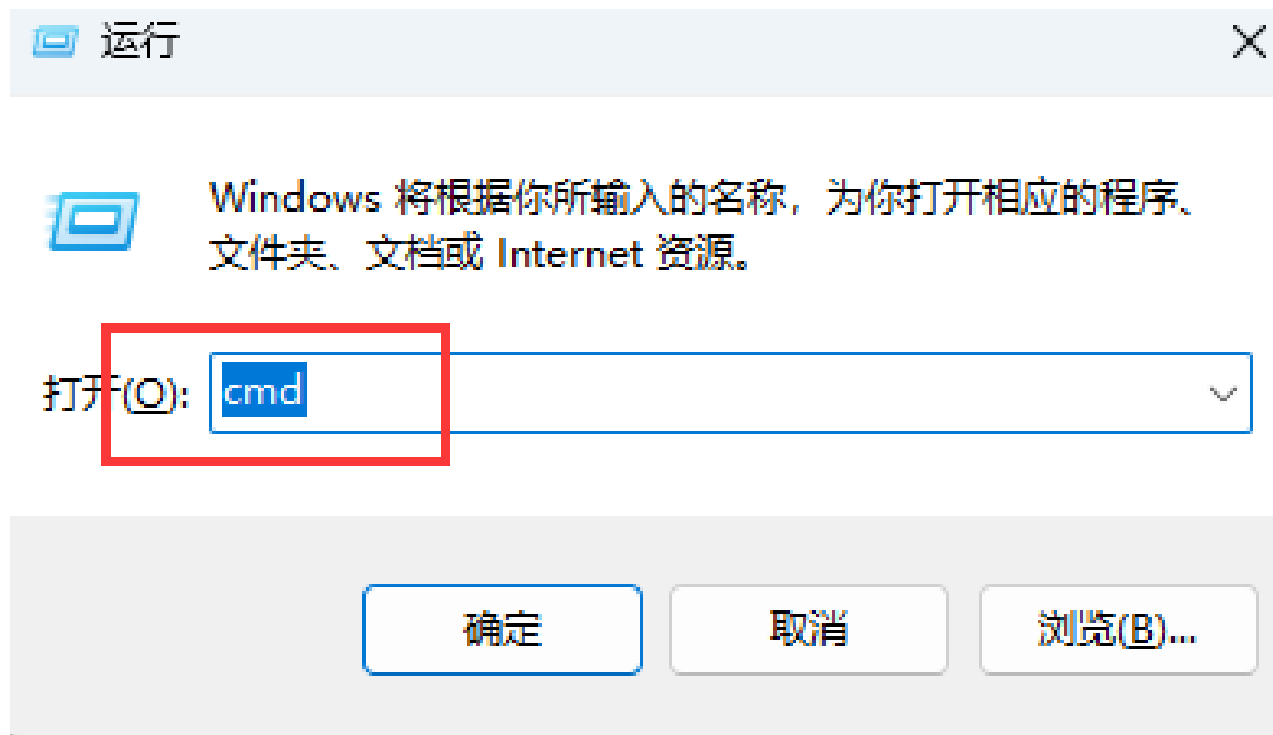
高级(V)...

确定 取消

3.1 硬件环境准备

(4) 配置PC接口IP地址

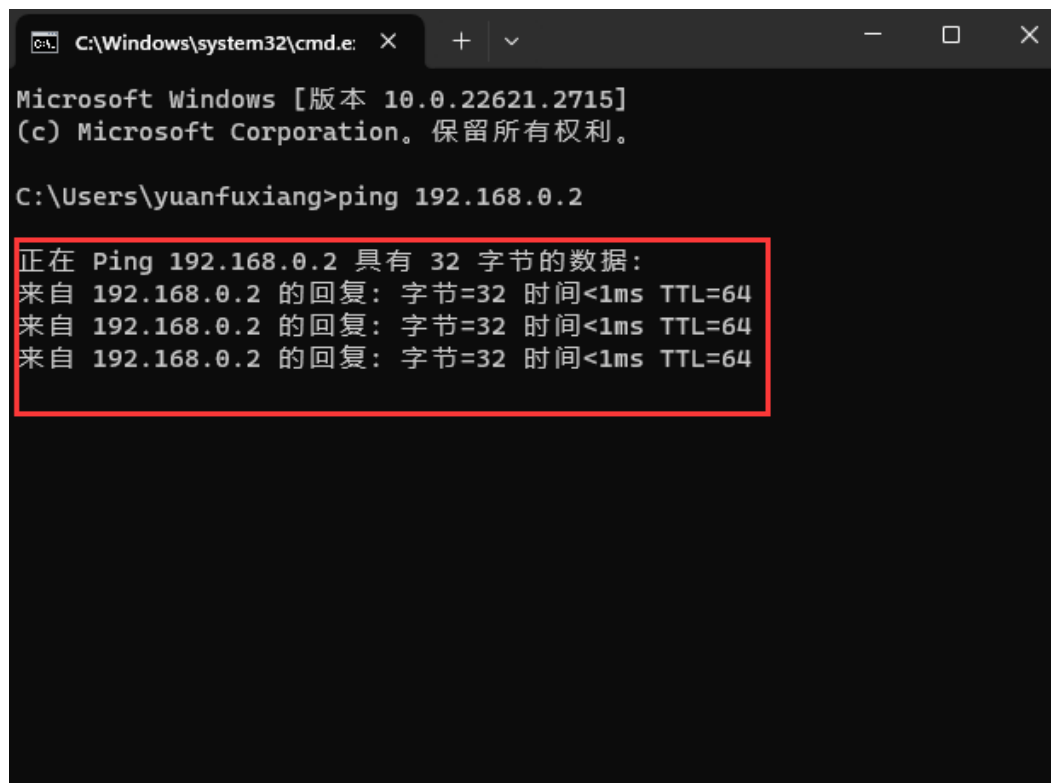
⑥使用快捷键 “Win+R” ， 在运行窗口输入cmd进入命令行窗口。



3.1 硬件环境准备

(4) 配置PC接口IP地址

⑦输入ping 192.168.0.2命令，查看是否能够访问开发板IP。



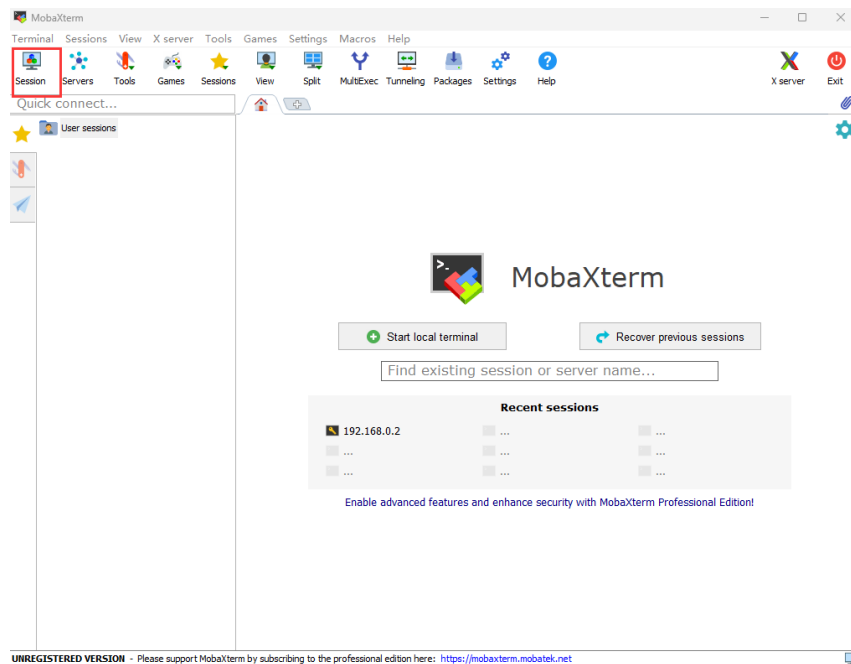
```
C:\Windows\system32\cmd.exe X + - □ X  
Microsoft Windows [版本 10.0.22621.2715]  
(c) Microsoft Corporation。保留所有权利。  
C:\Users\yuanfuxiang>ping 192.168.0.2  
正在 Ping 192.168.0.2 具有 32 字节的数据:  
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64  
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64  
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64
```

3.1 硬件环境准备

(5) 使用ssh远程登录开发板

本文使用MobaXterm工具在PC上远程登录开发板，单击下载链接获取MobaXterm软件压缩包，解压获得“MobaXterm_Personal_22.2.exe”。

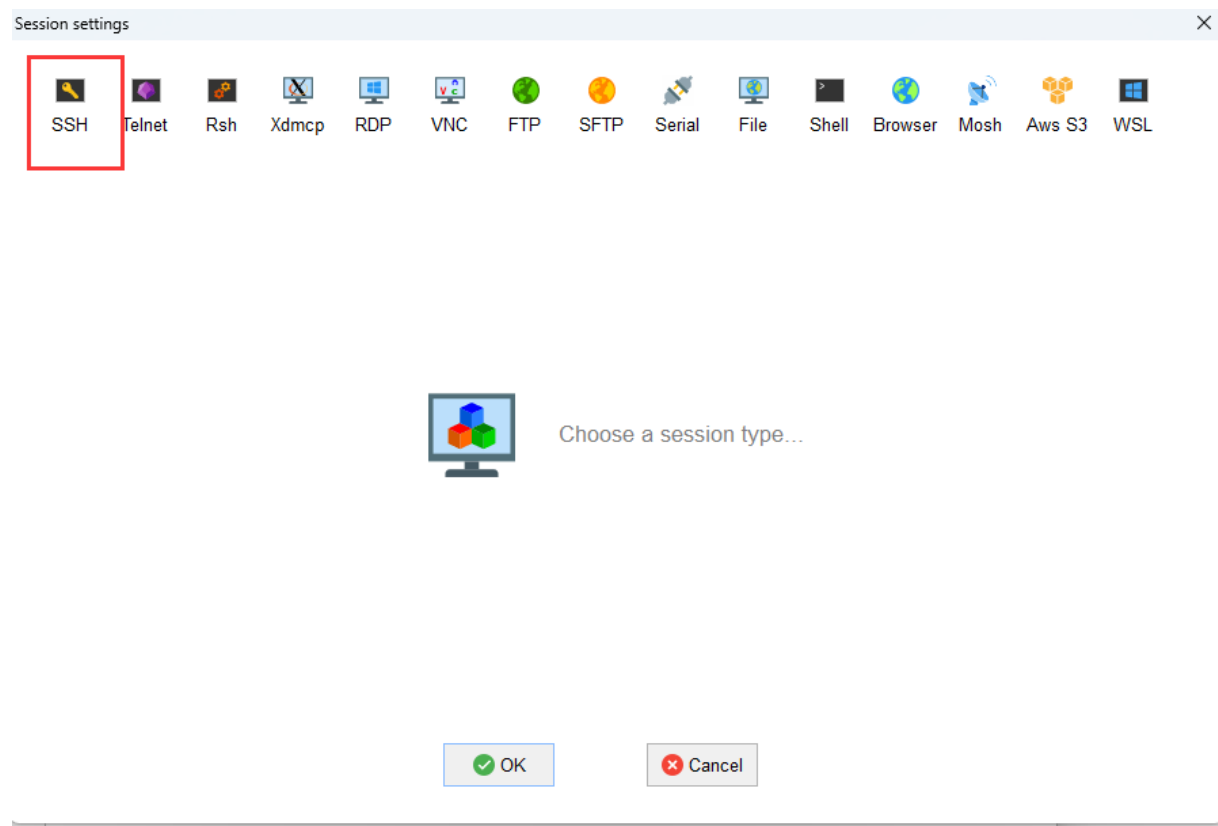
- ①双击“MobaXterm_Personal_22.2.exe”启动SSH登录工具。
- ②单击左上方的“Session”进入界面。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

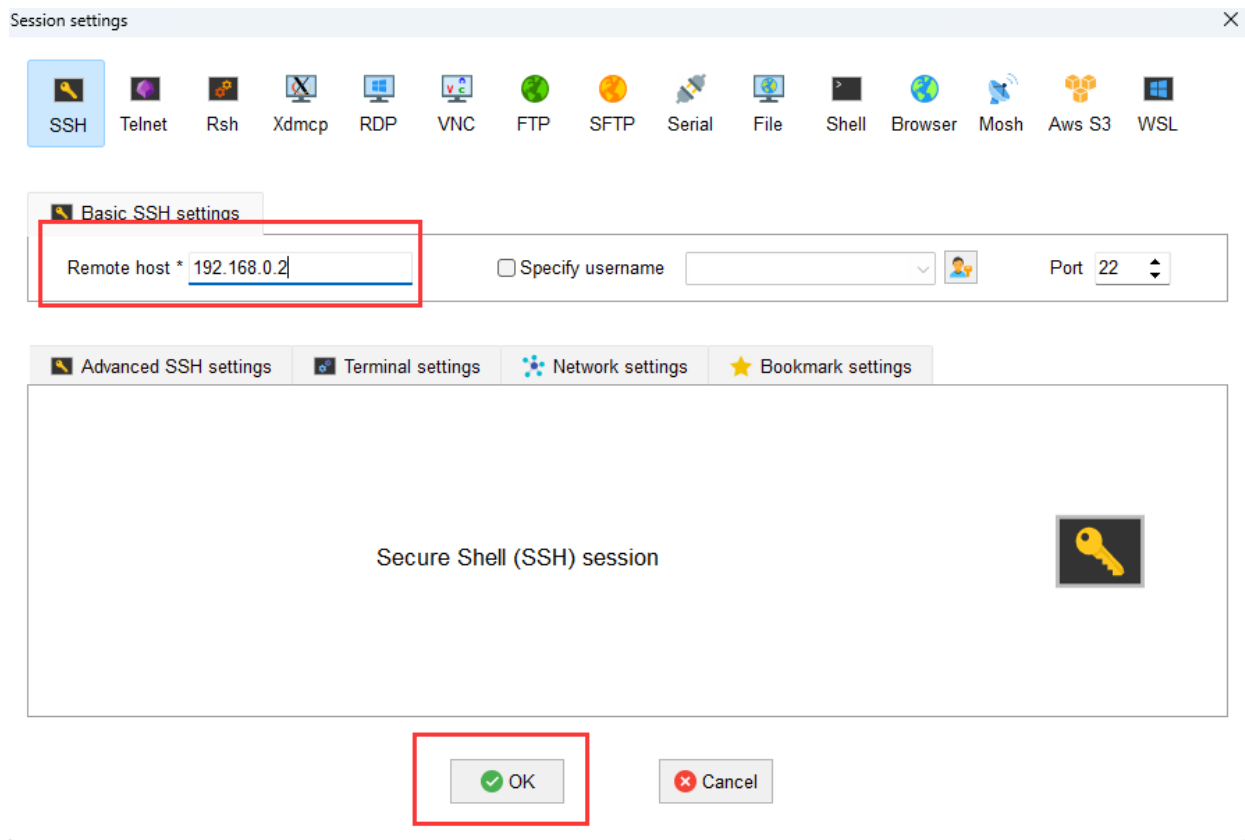
③单击左上方的“SSH”进入 SSH 连接配置界面



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

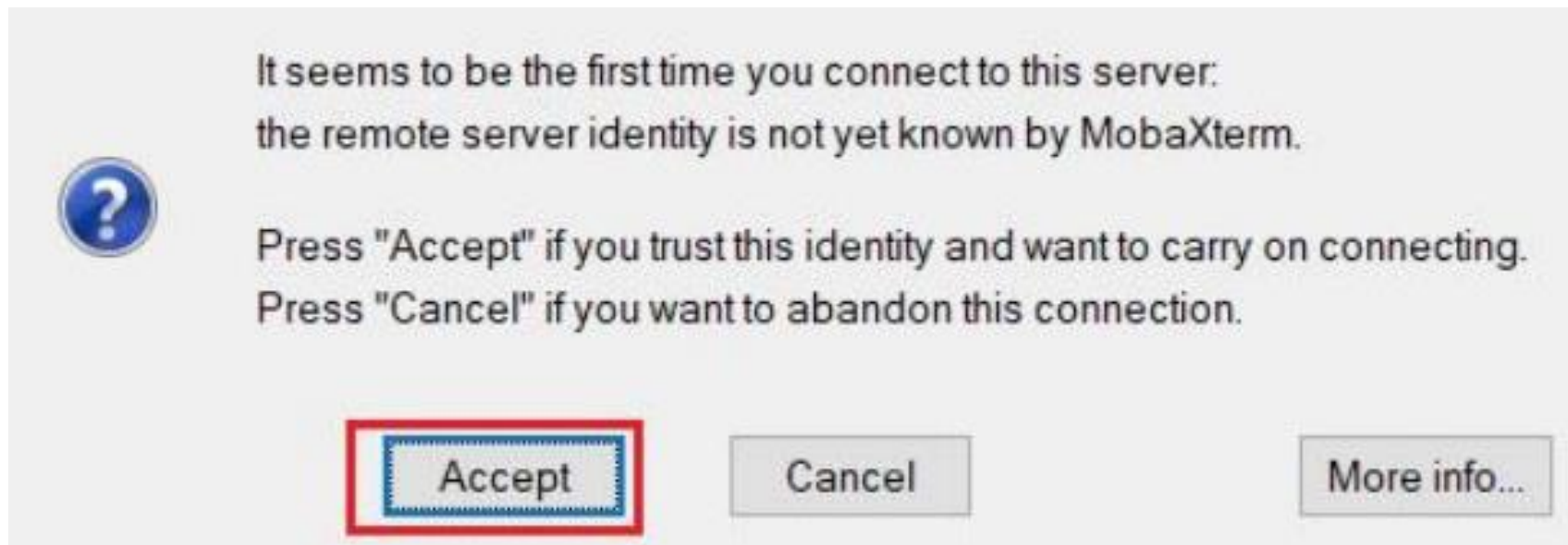
④根据硬件连接方式填写开发者套件连接PC的接口实际IP地址（一键制卡中配置的IP地址）。图示以192.168.0.2为例，请根据实际修改。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

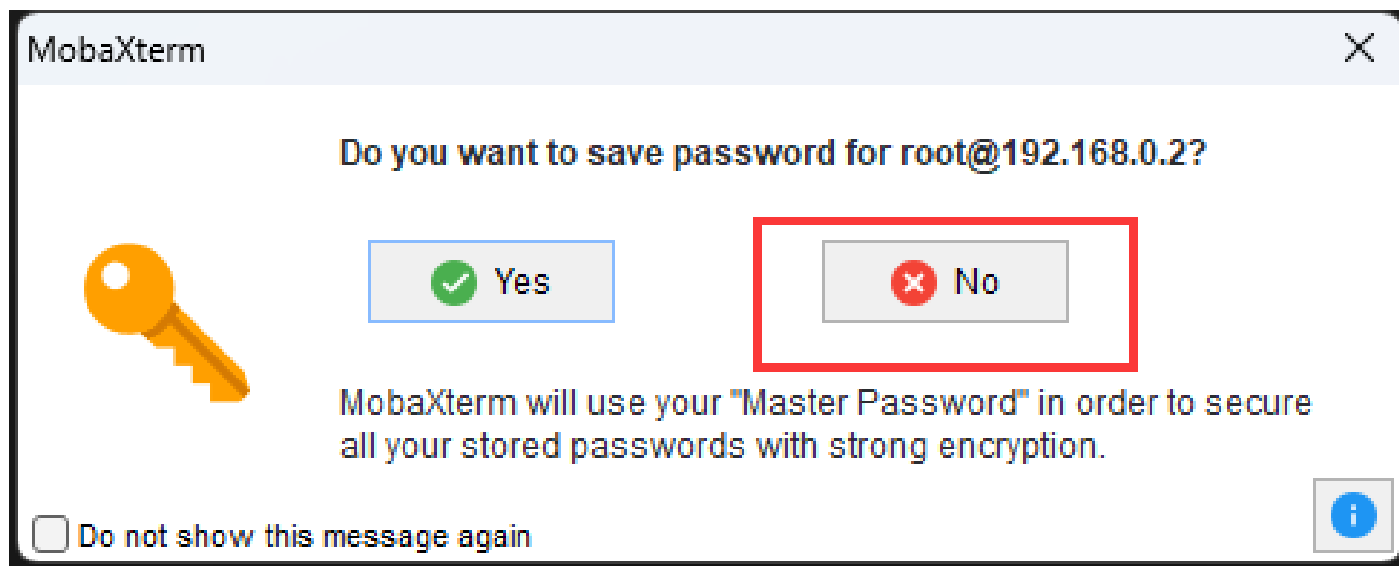
⑤单击“OK”按钮，首次连接开发者套件时，SSH工具提示是否信任连接的设备，单击“Accept”。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

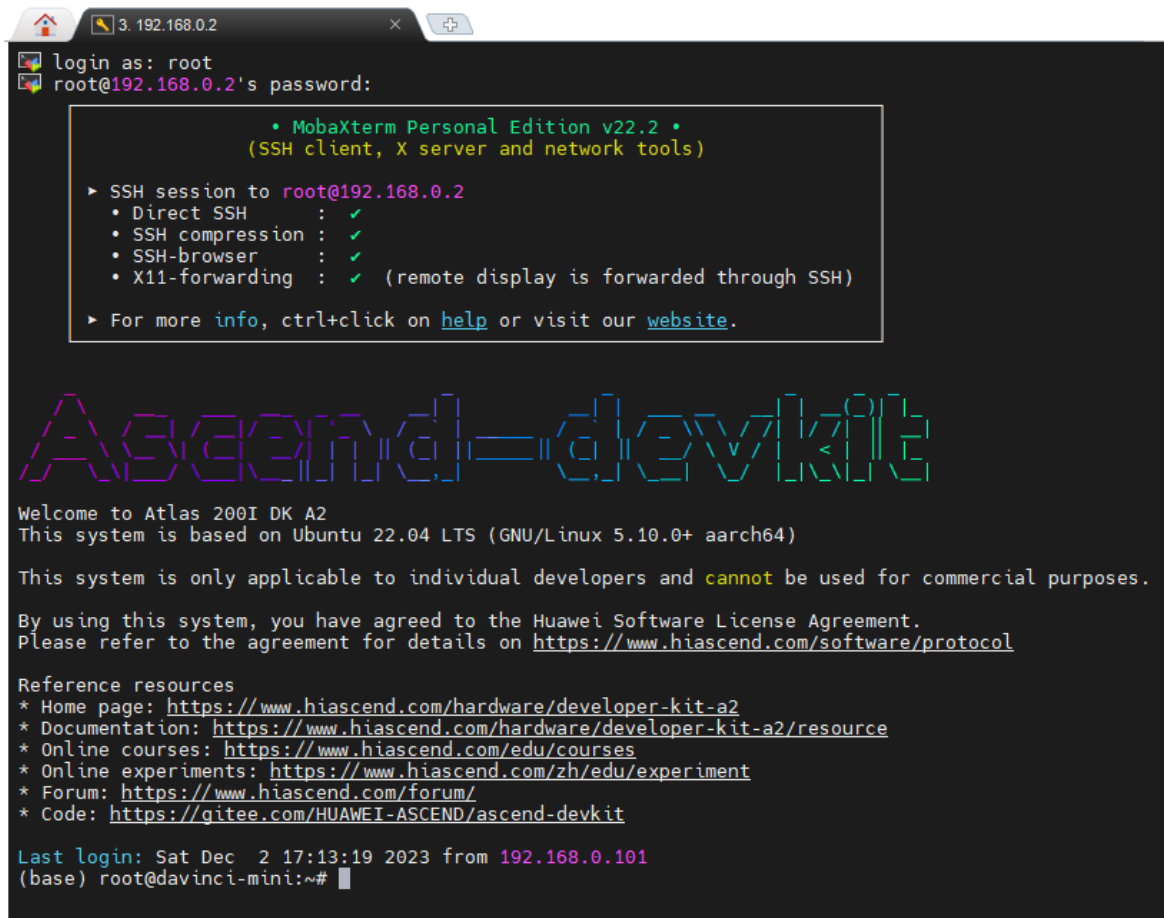
⑥进入远程登录界面后，输入 root 用户名登录密码（默认为 Mind@123）登录开发者套件，请修改默认密码，并妥善保管新密码。SSH 工具界面会出现保存密码提示，可以单击 “No”，不保存密码直接登录开发者套件。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

⑦远程登录开发者套件成功界面如图所示。



```
login as: root
root@192.168.0.2's password:

• MobaXterm Personal Edition v22.2 •
  (SSH client, X server and network tools)

▶ SSH session to root@192.168.0.2
  • Direct SSH      : ✓
  • SSH compression : ✓
  • SSH-browser     : ✓
  • X11-forwarding  : ✓ (remote display is forwarded through SSH)

▶ For more info, ctrl+click on help or visit our website.

Ascend=devkit

Welcome to Atlas 200I DK A2
This system is based on Ubuntu 22.04 LTS (GNU/Linux 5.10.0+ aarch64)

This system is only applicable to individual developers and cannot be used for commercial purposes.

By using this system, you have agreed to the Huawei Software License Agreement.
Please refer to the agreement for details on https://www.hiascend.com/software/protocol

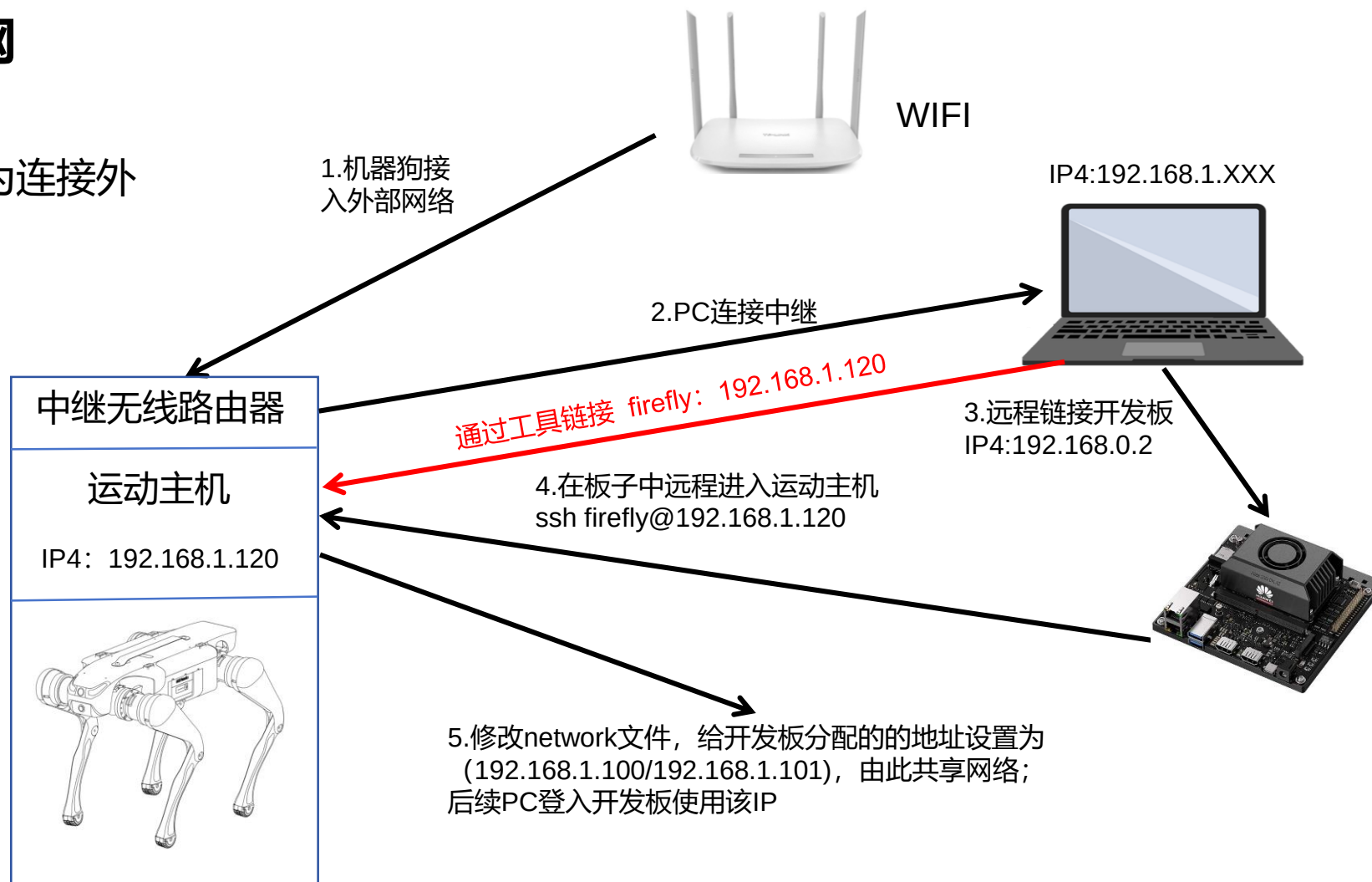
Reference resources
* Home page: https://www.hiascend.com/hardware/developer-kit-a2
* Documentation: https://www.hiascend.com/hardware/developer-kit-a2/resource
* Online courses: https://www.hiascend.com/edu/courses
* Online experiments: https://www.hiascend.com/zh/edu/experiment
* Forum: https://www.hiascend.com/forum/
* Code: https://gitee.com/HUAWEI-ASCEND/ascend-devkit

Last login: Sat Dec  2 17:13:19 2023 from 192.168.0.101
(base) root@davinci-mini:~#
```

3.1 硬件环境准备

(6) 机器狗中继连接外网

此步骤是为了让单机环境变为连接外部网络的环境



3.1 硬件环境准备

(6) 机器狗中继连接外网

注：本教程主要为暂时没有外置无线网卡的用户提供连接外网方案，教程以中继手机热点为例进行说明，中继无线路由器同理。

1、电脑连接 机器狗的wifi，点击电脑右下角的wifi图标，点击属性查看IPv4 DNS服务器的ip地址。如下图所示。

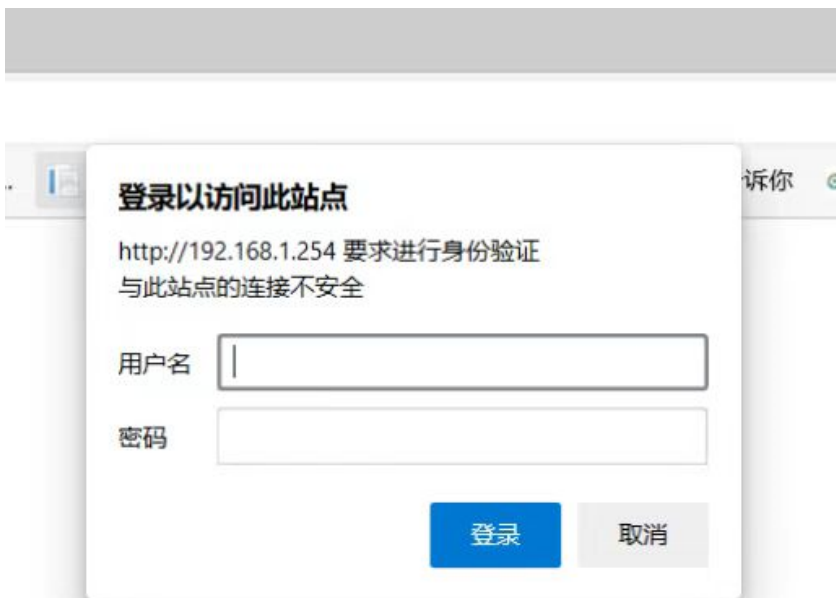


属性	
SSID:	YSC-JYML-emzqzj
协议:	Wi-Fi 4 (802.11n)
安全类型:	WPA2-个人
网络频带:	2.4 GHz
网络通道:	11
链接速度(接收/传输):	144/144 (Mbps)
IPv4 地址:	192.168.1.101
IPv4 DNS 服务器:	192.168.1.254 8.8.8.8
制造商:	Intel Corporation
描述:	Intel(R) Wireless-AC 9560 160MHz
驱动程序版本:	22.200.0.6
物理地址(MAC):	
复制	

3.1 硬件环境准备

(6) 机器狗中继连接外网

2、打开电脑浏览器，输入IPv4 DNS服务器的ip，默认登录用户和密码均为admin。
如下图所示。

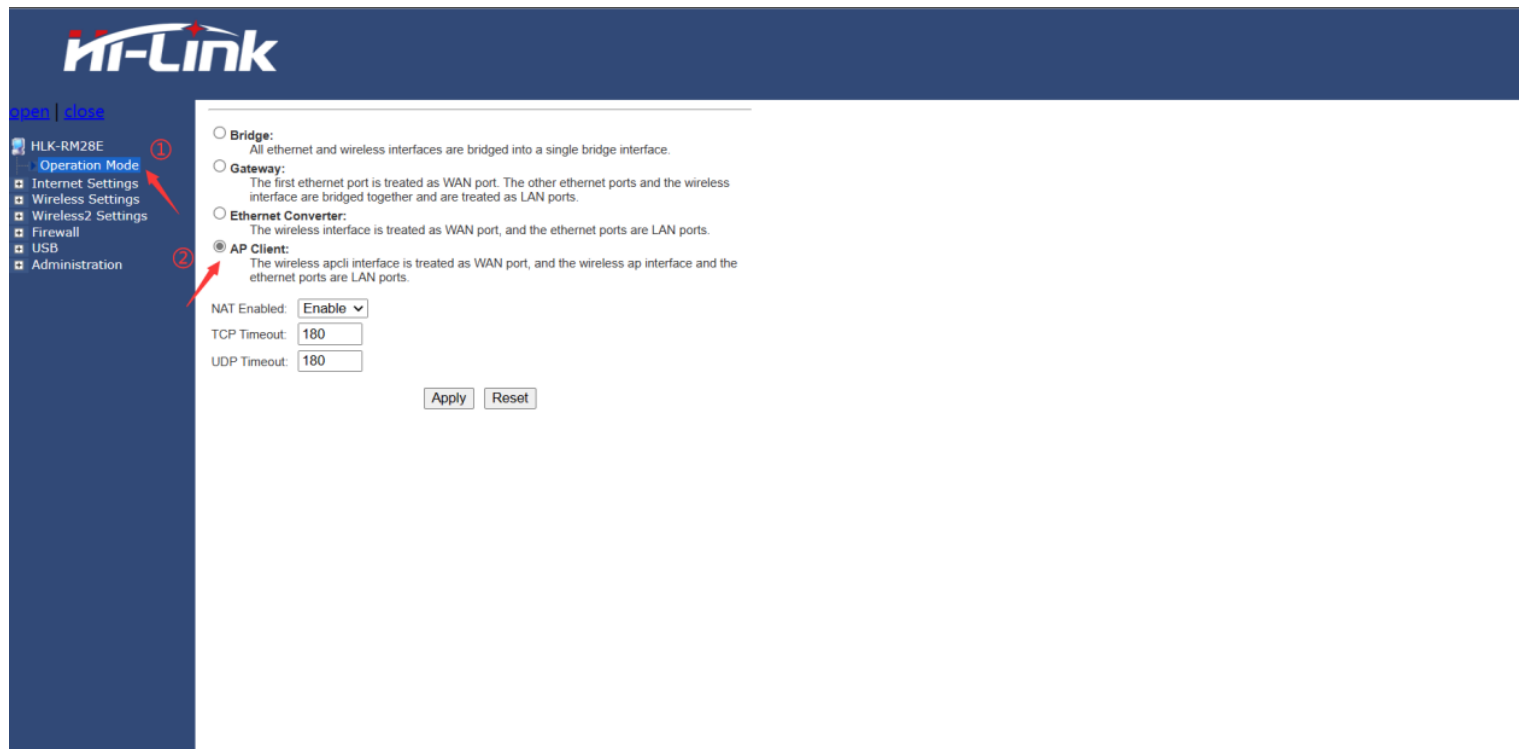


3.1 硬件环境准备

(6) 机器狗中继连接外网

3、打开手机热点。

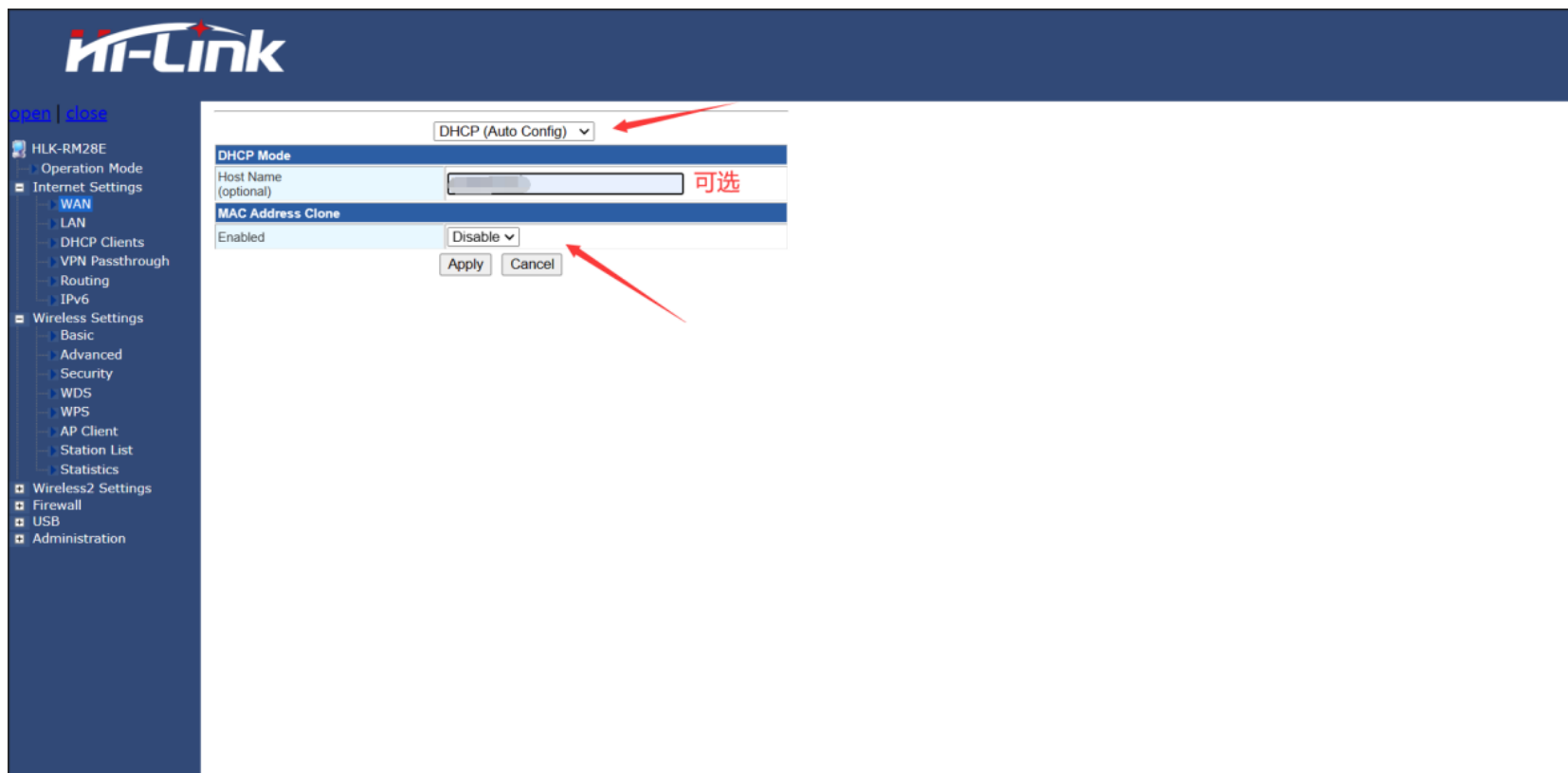
4、进入Hi-link设置页面，点击Operation Mode，选择AP Client，最后点击Apply。如下图所示。



3.1 硬件环境准备

(6) 机器狗中继连接外网

5、展开Internet Settings，点击WAN，参照下图进行设置，最后点击Apply。



3.1 硬件环境准备

(6) 机器狗中继连接外网

6、展开Wireless Settings，点击AP Client，查看site Survey表中是否有自己的手机热点名称，如若没有点击SCAN进行扫描，在AP client Parameters表中参照下图进行设置，最后点击Apply。

HLK-RM28E

Operation Mode

Internet Settings

Wireless Settings

- Basic
- Advanced
- Security
- WDS
- WPS
- AP Client
- Station List
- Statistics

Wireless2 Settings

Firewall

USB

Administration

AP Client Feature

You could configure AP Client parameters here.

AP Client Parameters

SSID

HelloWorld

MAC Address (Optional)

Security Mode

WPA2PSK

Encryption Type

AES

Pass Phrase

Apply

Cancel

SCAN

① 输入手机的热点名称

② 可选

③ 输入热点加密类型

④ 输入手机的热点密码

Site Survey

Ch	SSID	BSSID	Security	Signal(%)	W-Moe	ExtCh	NT
1	weichao	f4 84 8d 97 88 fc	WPA1PSKWPA2PSK/AES	68	11b/g/n	ABOVE	In
1	weichao	f4 84 8d 97 8f c2	WPA1PSKWPA2PSK/AES	70	11b/g/n	ABOVE	In
1	KRN1303	10 5d dc 2f a4 20	WPA1PSKWPA2PSK/AES	50	11b/g/n	NONE	In
1	iTV-X1oT	aa 00 74 e6 e3 46	WPAPSK/TKIPAES	52	11b/g/n	NONE	In
1	jsscsc	ec 60 73 e4 fc 3e	WPA1PSKWPA2PSK/AES	39	11b/g/n	ABOVE	In
1		ee 60 73 74 fc 3e	WPA1PSKWPA2PSK/AES	39	11b/g/n	ABOVE	In
1	0xE997B9E5A587E69CBAE599A8E4BABA	a2 2e 55 9a b0 38	WPA1PSKWPA2PSK/AES	100	11b/g/n	NONE	In
1	ChinaNet-X1oT	9a 00 74 e6 e3 46	WPAPSK/TKIPAES	52	11b/g/n	NONE	In
1	weichao	f4 84 8d 97 8b ab	WPA1PSKWPA2PSK/AES	100	11b/g/n	ABOVE	In
3	Baiyuan	b8 f0 b9 53 b0 f8	WPA1PSKWPA2PSK/AES	100	11b/g/n	ABOVE	In
4	409-DT-2G	80 3f 5d a9 ac 7e	WPA2PSK/AES	65	11b/g/n	ABOVE	In
4	ChinaNet-wmwX	b0 b1 94 ef d2 c3	WPAPSK/TKIPAES	68	11b/g/n	NONE	In
5	ChinaNet-zu2d	14 30 04 00 5d 84	WPA1PSKWPA2PSK/TKIPAES	63	11b/g/n	NONE	In
6	ChinaNet-WrQ4	2c 1a 01 d1 ea 88	WPA1PSKWPA2PSK/TKIPAES	78	11b/g/n	NONE	In
6	weichao	f4 84 8d 97 8e cc	WPA1PSKWPA2PSK/AES	86	11b/g/n	BELOW	In
6		18 3c b7 37 74 99	WPA2PSK/AES	94	11b/g/n	NONE	In

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Page 51

3.1 硬件环境准备

(6) 机器狗中继连接外网

7、重启机器狗，注意中继时热点需一直处于开启状态，电脑连接机器狗的wifi，使用远程软件登录，此时机器狗处于连接外网状态。测试是否有网络，点击浏览器，搜索任意内容，如“百度学术”，能正常显示下图说明正常。



目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

3.1 硬件环境准备

3.2 下载mindyolo项目代码

3.3 创建实验环境

3.4 安装环境依赖

3.5 开发板环境（机器狗网址配置）

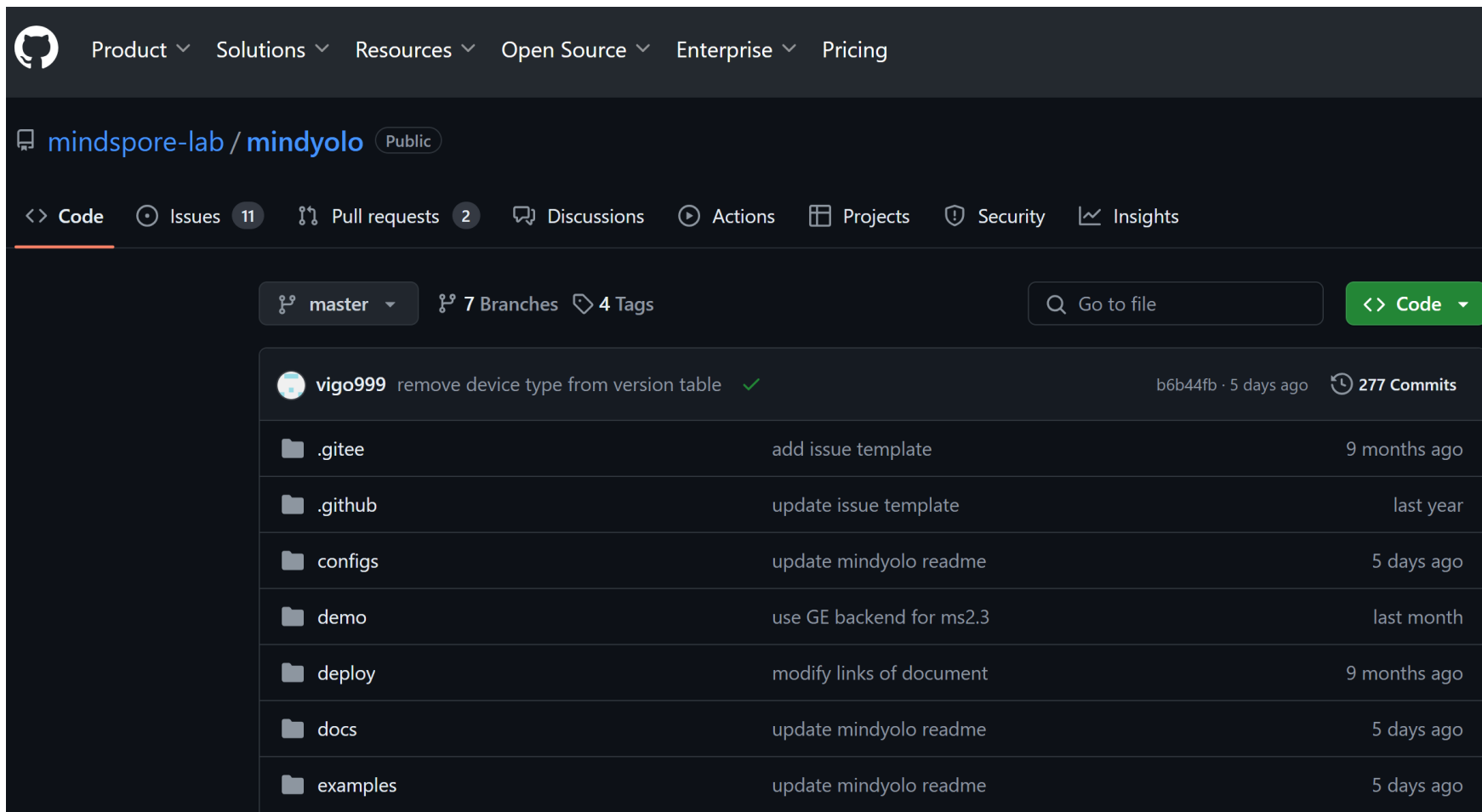
四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

3.2 下载mindyolo项目代码



项目地址: <https://github.com/mindspore-lab/mindyolo.git>

下载项目文件, 然后自行保存到一个文件夹中 (本实验中保存的路径是D:\pythonProject\)

目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

3.1 硬件环境准备

3.2 下载mindyolo项目代码

3.3 创建实验环境

3.4 安装环境依赖

3.5 开发板环境（机器狗网址配置）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

3.3 创建实验环境

创建一个3.9版本的环境取名为mindspore1，指令如下：

```
conda create-n mindspore1 python=3.9.2
```

然后进入到mindspore1的环境中：

```
conda activate mindspore1
```

如果实际效果如下图所示，则证明成功配置了mindyolo的实验环境。

```
(base) PS C:\Users\15210> conda activate mindspore1
(mindspore1) PS C:\Users\15210> d:
(mindspore1) PS D:\> cd .\pythonProject\mindyolo\
(mindspore1) PS D:\pythonProject\mindyolo> python --version
Python 3.9.2
```

目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

3.1 硬件环境准备

3.2 下载mindyolo项目代码

3.3 创建实验环境

3.4 安装环境依赖

3.5 开发板环境（机器狗网址配置）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

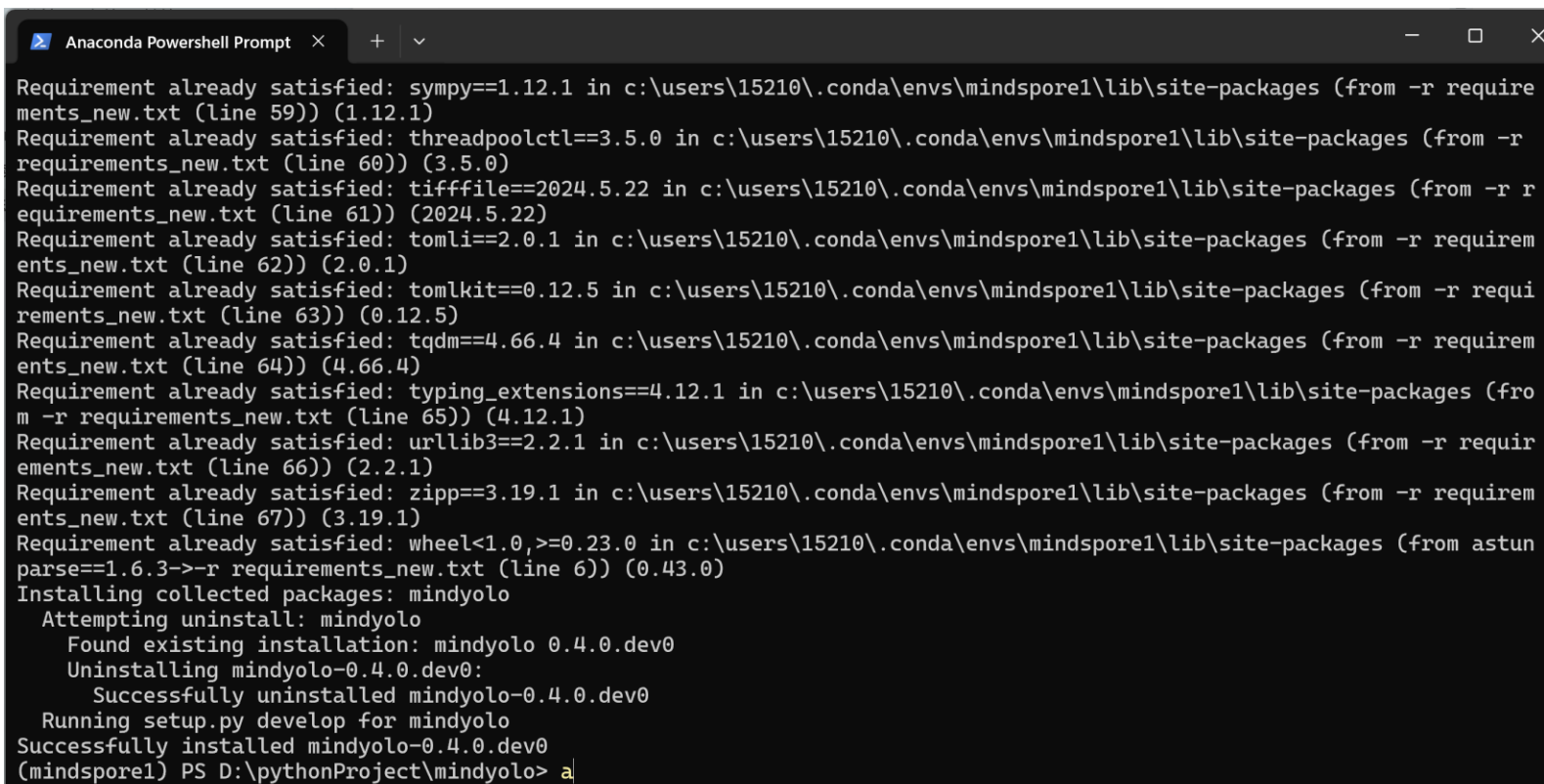
3.4 安装环境依赖

在这个环境中，我们直接输入指令，安装requirements_new.txt这个文件中的环境依赖。

```
pip install -r requirements_new.txt
```

因为实验已经实现安装好了，下面的安装就会显示已经安装了，如果下载速度太慢了，可以切换安装源。

```
pip install -r requirements_new.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```



```
Anaconda Powershell Prompt
Requirement already satisfied: sympy==1.12.1 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 59)) (1.12.1)
Requirement already satisfied: threadpoolctl==3.5.0 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 60)) (3.5.0)
Requirement already satisfied: tifffile==2024.5.22 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 61)) (2024.5.22)
Requirement already satisfied: tomli==2.0.1 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 62)) (2.0.1)
Requirement already satisfied: tomlkit==0.12.5 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 63)) (0.12.5)
Requirement already satisfied: tqdm==4.66.4 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 64)) (4.66.4)
Requirement already satisfied: typing_extensions==4.12.1 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 65)) (4.12.1)
Requirement already satisfied: urllib3==2.2.1 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 66)) (2.2.1)
Requirement already satisfied: zipp==3.19.1 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 67)) (3.19.1)
Requirement already satisfied: wheel<1.0,>=0.23.0 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from astunparse==1.6.3->-r requirements_new.txt (line 6)) (0.43.0)
Installing collected packages: mindyolo
  Attempting uninstall: mindyolo
    Found existing installation: mindyolo 0.4.0.dev0
    Uninstalling mindyolo-0.4.0.dev0:
      Successfully uninstalled mindyolo-0.4.0.dev0
  Running setup.py develop for mindyolo
Successfully installed mindyolo-0.4.0.dev0
(mindspore1) PS D:\pythonProject\mindyolo> a
```

目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

3.1 硬件环境准备

3.2 下载mindyolo项目代码

3.3 创建实验环境

3.4 安装环境依赖

3.5 开发板环境（机器狗网址配置）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

4.1 PC

4.2 Ascend

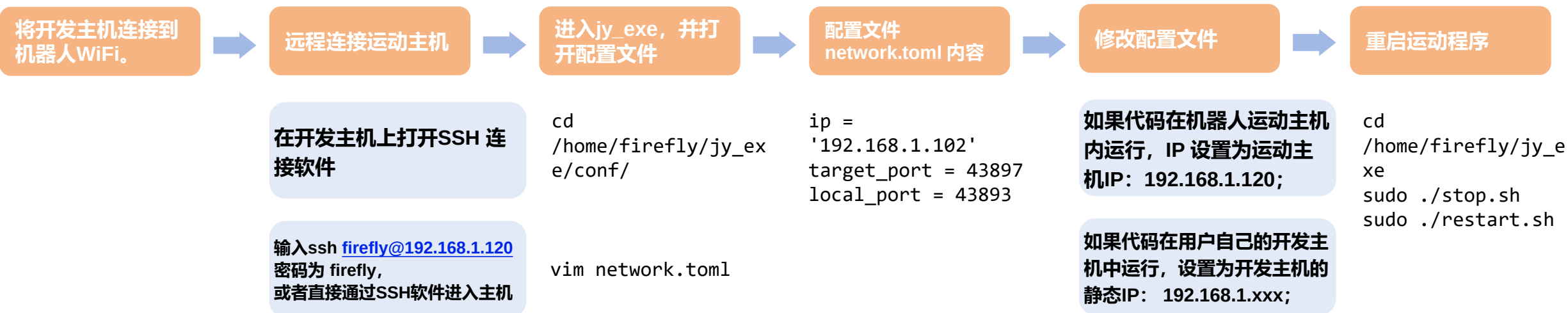
五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

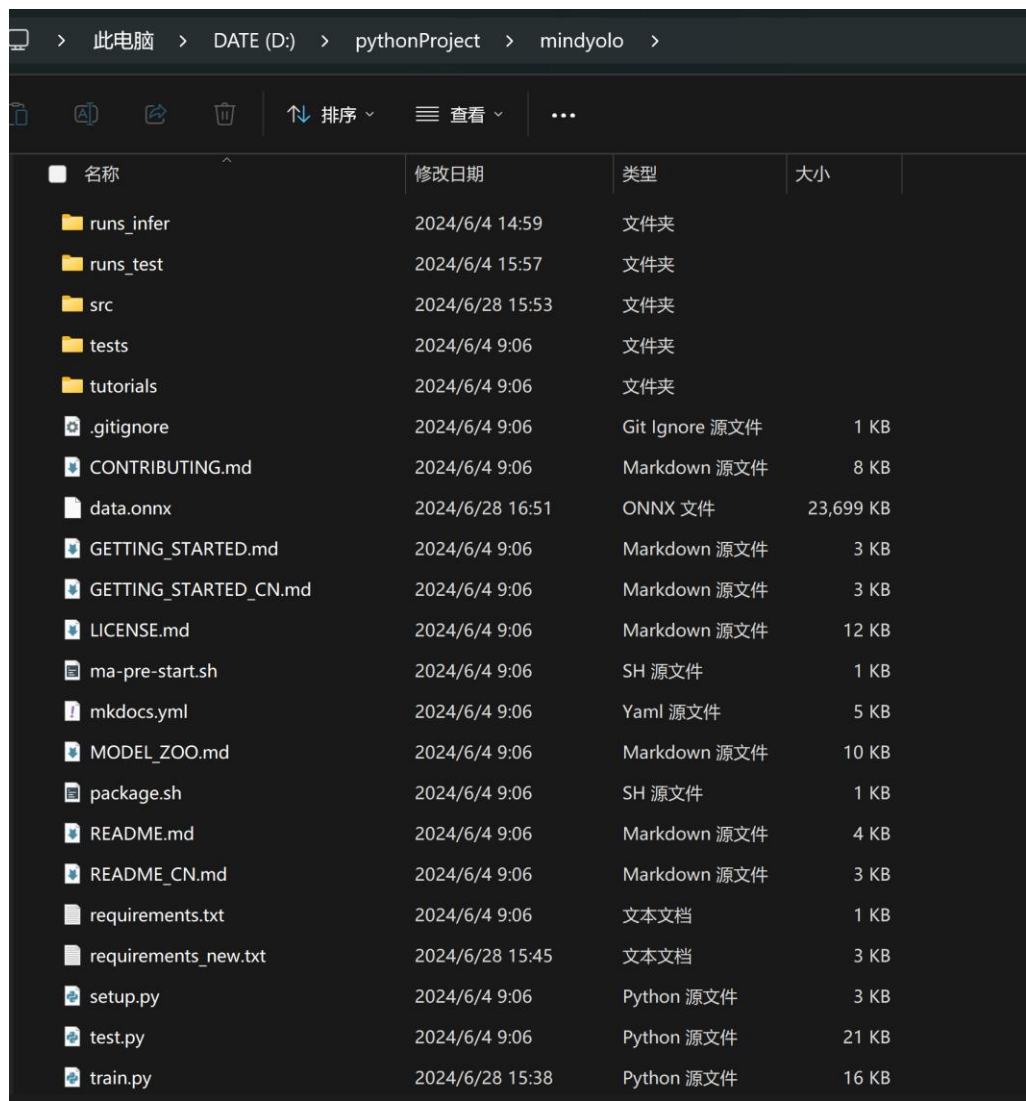
3.2 机器狗远程连接

用户可通过SSH 远程连接到运动主机。



4.1 PC

PC端的文件主要是用来本地训练的项目文件，下面是部分文件的预览。



名称	修改日期	类型	大小
runs_infer	2024/6/4 14:59	文件夹	
runs_test	2024/6/4 15:57	文件夹	
src	2024/6/28 15:53	文件夹	
tests	2024/6/4 9:06	文件夹	
tutorials	2024/6/4 9:06	文件夹	
.gitignore	2024/6/4 9:06	Git Ignore 源文件	1 KB
CONTRIBUTING.md	2024/6/4 9:06	Markdown 源文件	8 KB
data.onnx	2024/6/28 16:51	ONNX 文件	23,699 KB
GETTING_STARTED.md	2024/6/4 9:06	Markdown 源文件	3 KB
GETTING_STARTED_CN.md	2024/6/4 9:06	Markdown 源文件	3 KB
LICENSE.md	2024/6/4 9:06	Markdown 源文件	12 KB
ma-pre-start.sh	2024/6/4 9:06	SH 源文件	1 KB
mkdocs.yml	2024/6/4 9:06	Yaml 源文件	5 KB
MODEL_ZOO.md	2024/6/4 9:06	Markdown 源文件	10 KB
package.sh	2024/6/4 9:06	SH 源文件	1 KB
README.md	2024/6/4 9:06	Markdown 源文件	4 KB
README_CN.md	2024/6/4 9:06	Markdown 源文件	3 KB
requirements.txt	2024/6/4 9:06	文本文档	1 KB
requirements_new.txt	2024/6/28 15:45	文本文档	3 KB
setup.py	2024/6/4 9:06	Python 源文件	3 KB
test.py	2024/6/4 9:06	Python 源文件	21 KB
train.py	2024/6/28 15:38	Python 源文件	16 KB

目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

4.1 PC

4.2 Ascend

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

七、运行示例

4.2 Ascend

本章节主要展示的是在开发板上所有的文件，以及每个文件所对应的作用。本章节主要展示开发板上所有的文件及其对应的作用，以便更好地理解整个项目的结构和功能。

```
(base) root@davinci-mini:/opt/lab/ObstacleAvoidance# tree -I "__pycache__|*.pyc" -L 10
.
├── avoidance_models
│   ├── 1.om
│   ├── fusion_result.json
│   ├── model.onnx
│   └── predefined_classes.txt
├── camera
│   ├── 1.py
│   ├── HKcamera.py
│   └── det_utils.py
├── command
│   ├── __init__.py
│   └── udp_command.py
├── getAndSavePicture.py
├── obstacle_main.py
├── obstacle_model_cap.py
├── robotstatuswatcher
│   ├── __init__.py
│   └── listener.py
├── sendcommand
│   ├── SendToCommand.py
│   ├── __init__.py
│   └── heartbeat.py
├── socketnetwork
│   ├── __init__.py
│   └── network_utils.py
├── speeds
│   ├── __init__.py
│   └── sportspeed.py
├── threading_utils
│   └── ThreadTemplates.py
```

4.2 Ascend

开发板的文件夹总名为 ‘ObstacleAvoidance’，如图所示。obstacle_main.py是主程序，getAndSavePicture.py是用来采集数据集的脚本，obstacle_main.py是用来识别障碍物的脚本，该文件夹包含多个子文件夹和文件，每个文件都有其特定的用途。下面是各个文件夹及其主要文件的详细介绍：

Command 文件夹

- **udp_command.py**: 该文件用于存放指令码。它定义了各种控制命令，这些命令通过UDP协议发送到机器狗，以实现对其精确控制。

SendCommand 文件夹

- **heartbeat.py**: 该文件用于发送心跳包。心跳包是一种定期发送的信号，用于检测系统或设备是否在线和正常工作。通过发送心跳包，可以确保机器狗与控制系统之间的通信连接是稳定和可靠的。
- **SendToCommand.py**: 该文件负责发送指令。它包含了一系列函数，用于将指令发送到机器狗，确保指令能够正确地传达和执行。

4.2 Ascend

SocketNetwork 文件夹

- **network_utils.py**: 该文件是与机器狗建立通信的脚本。它包含了各种网络通信的实用函数，用于初始化和维护与机器狗的通信连接，使得控制指令能够通过网络传输到机器狗。

Speeds 文件夹

- **sportspeed.py**: 该文件定义了机器狗运动的速度方程。通过调整速度参数，可以控制机器狗的运动速度，使其能够按照预定的轨迹和速度执行任务。

ThreadingUtils 文件夹

- **ThreadTemplates.py**: 该文件定义了线程模板。线程是并发执行的基本单位，通过使用线程，可以提高系统的响应速度和处理效率。该文件中的模板可以用于创建和管理多个线程，以实现并行处理。

avoidance_models 文件夹

- 该文件夹主要存放的是用来识别的om模型文件。

在实际开发和调试过程中，了解每个文件的具体作用，有助于快速定位问题并进行相应的修改和优化。

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备（1学时）
- 四、工程文件目录结构
- 五、模型本地化训练及部署（2学时）
 - 5.1 配置开发板的HK驱动
 - 5.2 编写数据采集脚本
 - 5.3 配置PC端数据标注环境
 - 5.4 标注数据集
 - 5.5 启动项目文件
 - 5.6 数据集格式规范
 - 5.7 训练
 - 5.8 配置模型转换算子
 - 5.9 开发板转换模型
- 六、逻辑实现（5学时）
- 七、运行示例

5.1 配置开发板的HK驱动

Step 1

下载并安装MVS驱动文件（Linux）
海康威视官网下载链接：
<https://www.hikrobotics.com/cn/machinevision/service/download?module=0>

Step 2

打开MobaXterm
远程控制，进入开发板安装驱动指令：

```
dpkg -i MVS-3.0.1_aarch64_20240422.deb
```

Step 3

在代码中使用需要加入包的路径：

```
sys.path.append("/opt/MVS/Samples/aarch64/Python/MvImport")  
from MvCameraControl_class import*
```

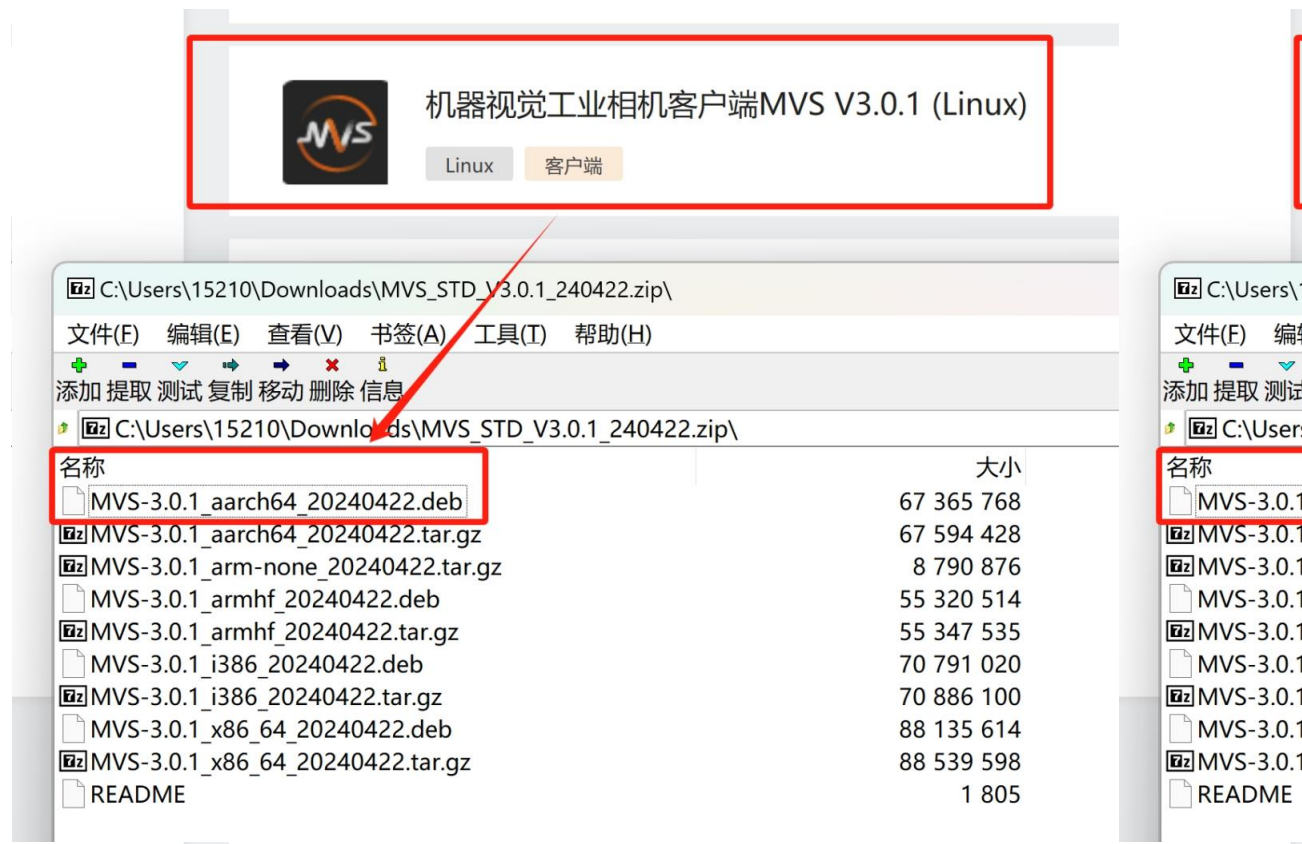
Step 4

使用摄像头的代码

```
python HKcamera.py
```


5.1 配置开发板的HK驱动

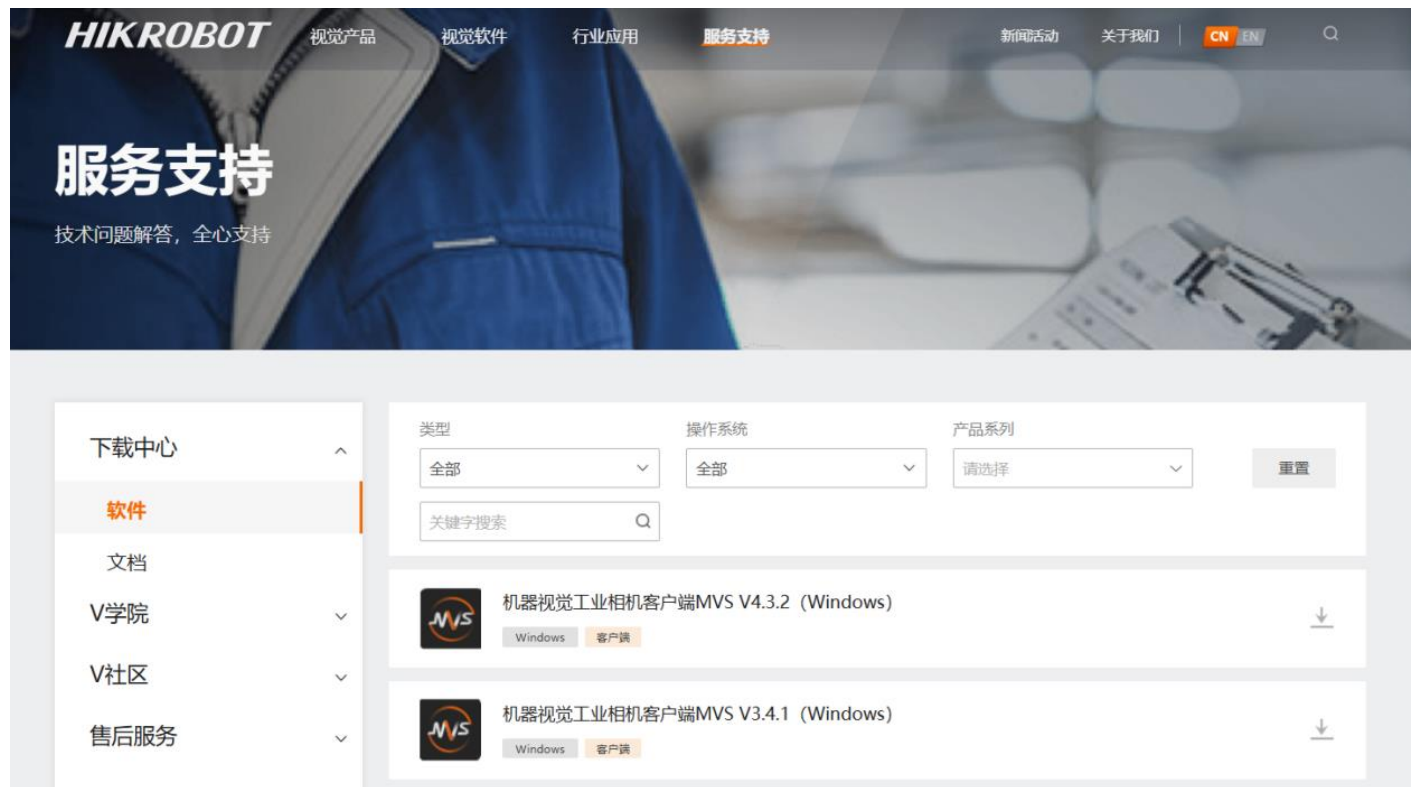
下载并安装MVS驱动文件（Linux）



海康威视官网下载链接:

<https://www.hikrobotics.com/cn/machinevision/service/download?module=0>

5.1 配置开发板的HK驱动



打开MobaXterm远程控制，进入开发板安装驱动指令：`dpkg -i MVS-3.0.1_aarch64_20240422.deb`

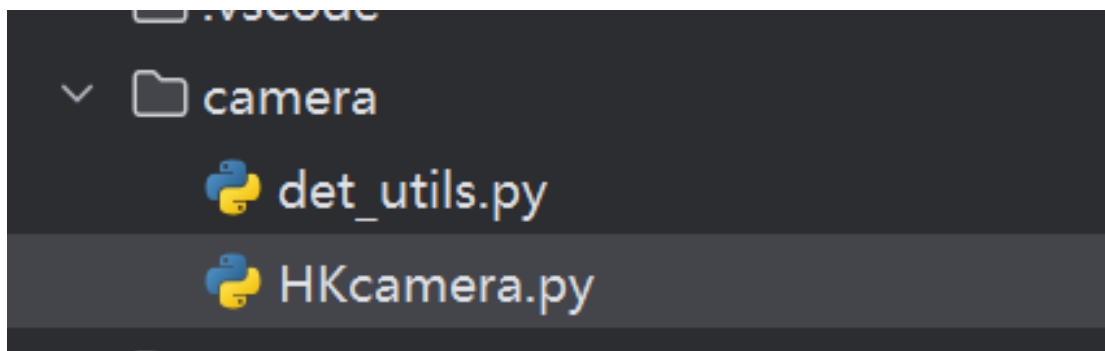
在代码中使用需要加入包的路径：

```
sys.path.append("/opt/MVS/Samples/aarch64/Python/MvImport")from MvCameraControl_class import*
```

5.1 配置开发板的HK驱动

```
HKcamera.py x det_utils.py
1 import sys
2 import numpy as np
3 import cv2
4 import time
5
6 sys.path.append("/opt/MVS/Samples/aarch64/Python/MvImport")
7 from MvCameraControl_class import *
```

使用摄像头的代码：python HKcamera.py



目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

5.1 配置开发板的HK驱动

5.2 编写数据采集脚本

5.3 配置PC端数据标注环境

5.4 标注数据集

5.5 启动项目文件

5.6 数据集格式规范

5.7 训练

5.8 配置模型转换算子

5.9 开发板转换模型

六、逻辑实现（5学时）

七、运行示例

5.2编写数据采集脚本

getAndSavePicture.py

```
22. def save_image(img):
23.     image_name = f'train_images/demo/{time.time()}.jpg'
24.     cv2.imwrite(image_name, img)
25.     print(f'图像已保存为: {image_name}')
.....
33. while True:
34.     img = getImage() # 获取图像
35.
36.     cv2.imshow('Camera Feed', img) # 显示图像
37.     key = cv2.waitKey(1) & 0xFF # 检测按键事件
38.
39.     if key == ord('s'):
40.         save_image(img) # 保存图像
41.     elif key == ord('q'):
42.         break # 退出循环
43.
44. # 清理工作
45. cv2.destroyAllWindows()
```

保存图片的路径

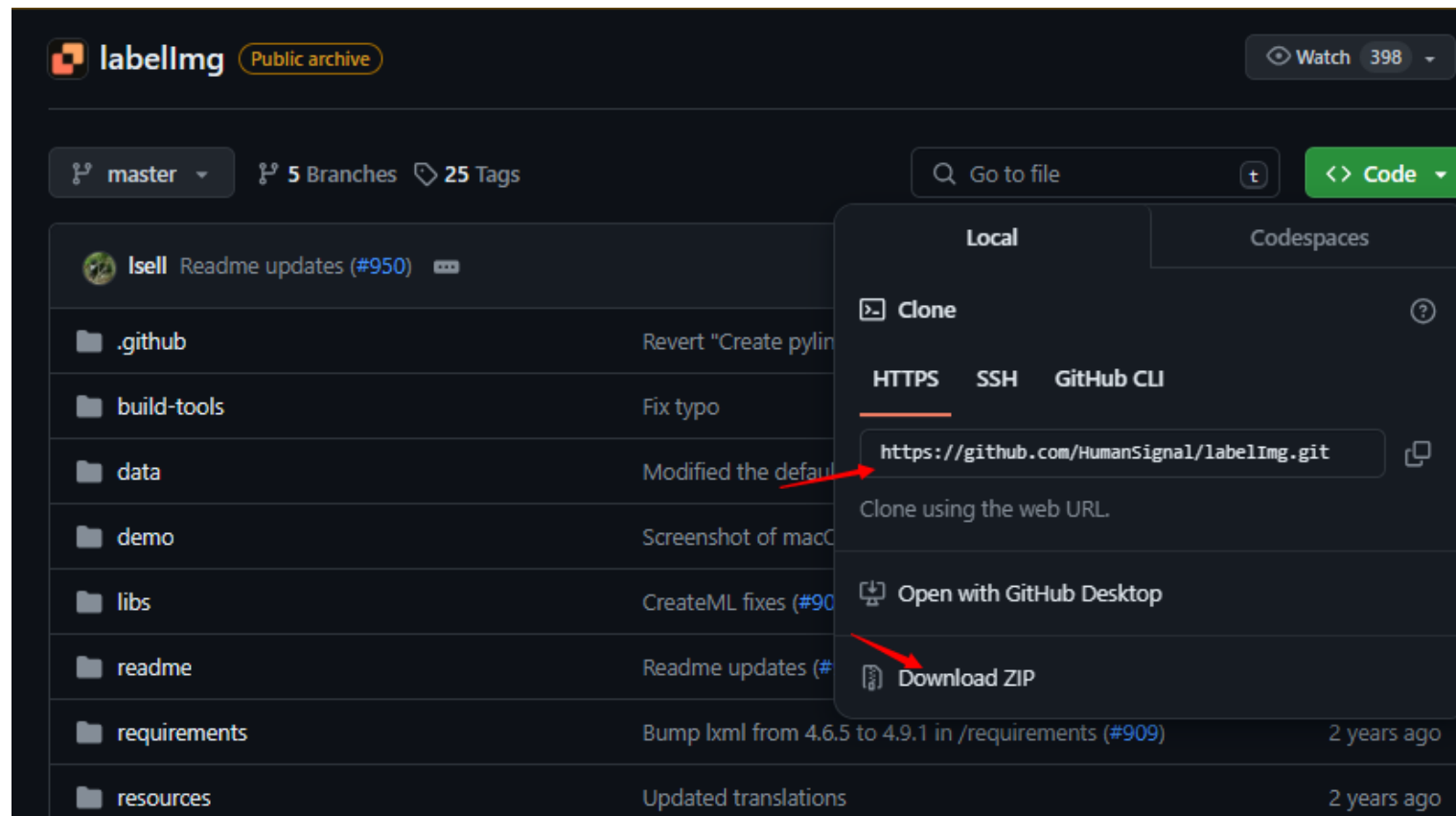
获取图片

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备（1学时）
- 四、工程文件目录结构
- 五、模型本地化训练及部署（2学时）
 - 5.1 配置开发板的HK驱动
 - 5.2 编写数据采集脚本
 - 5.3 配置PC端数据标注环境
 - 5.4 标注数据集
 - 5.5 启动项目文件
 - 5.6 数据集格式规范
 - 5.7 训练
 - 5.8 配置模型转换算子
 - 5.9 开发板转换模型
- 六、逻辑实现（5学时）
- 七、运行示例

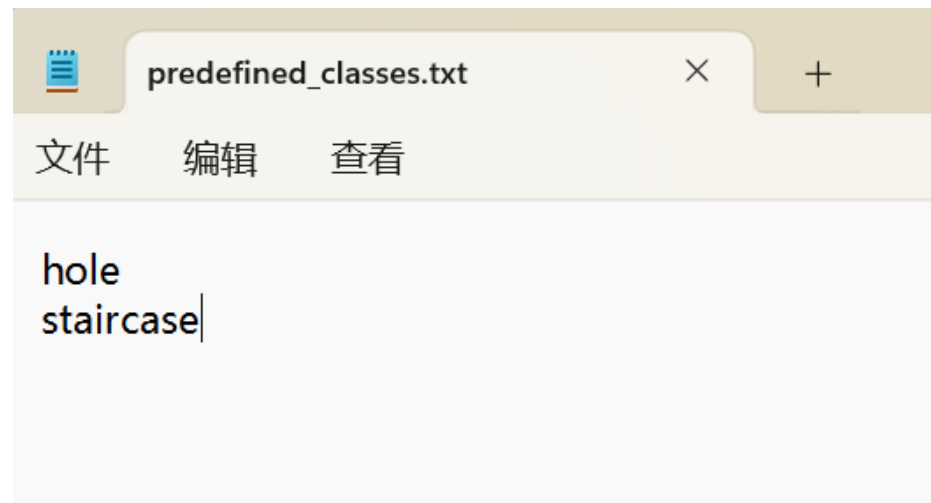
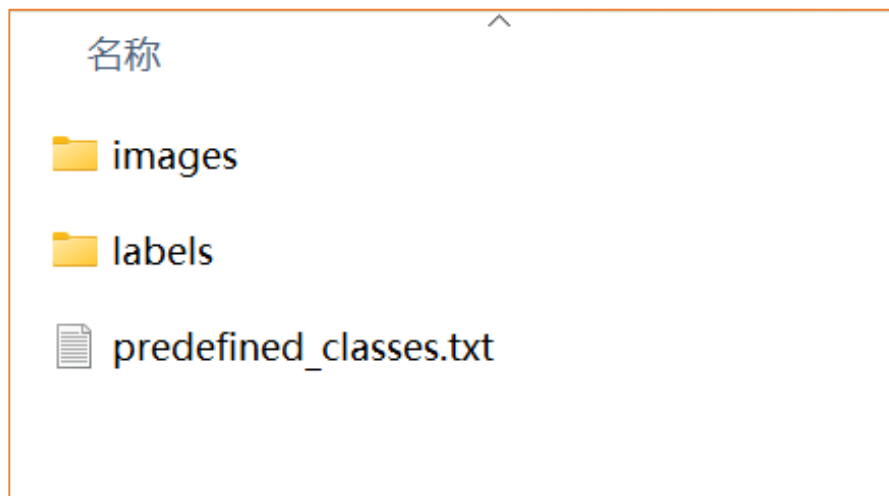
5.3 配置PC端数据标注环境

下载数据标注工具labelimg: <https://github.com/HumanSignal/labelImg>



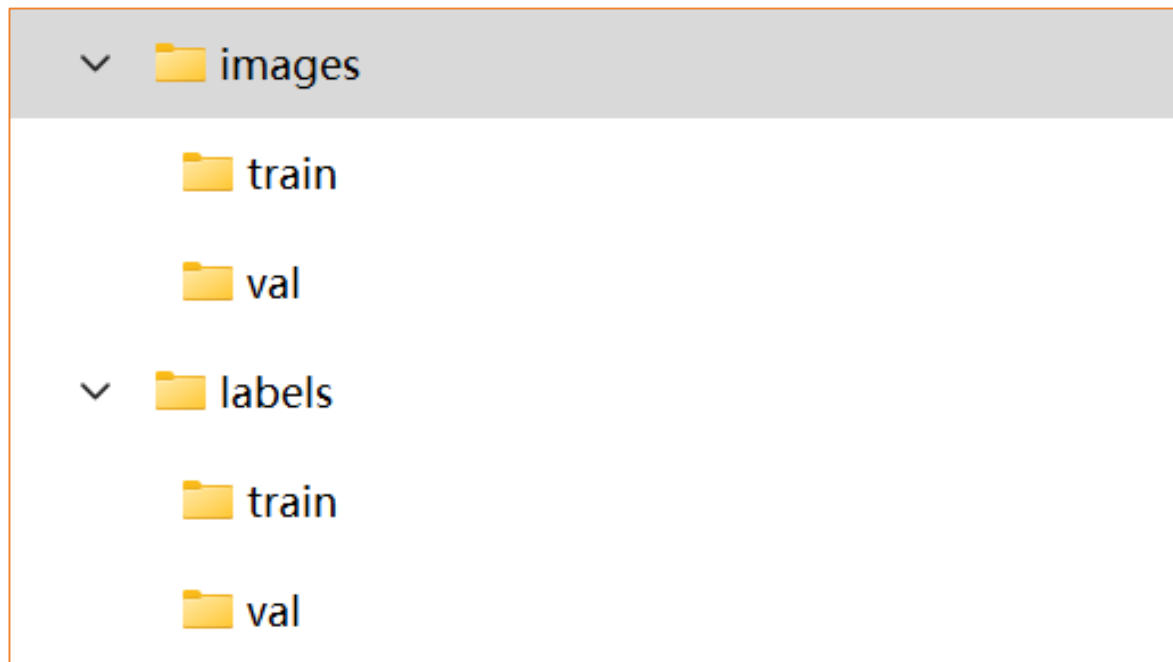
5.3配置PC端数据标注环境

建立两个文件夹，分别是images用来存放训练集train的图片和验证集val的图片，labels用来存放标注完train中对应的图片数据标签和val中的对应的图片数据标签，以及predefined_classes.txt存放标签类别。



5.3配置PC端数据标注环境

/images/train对应labelImg中的存放目录，而/labels/train对应labelImg的改变存放目录，用于存放标注完图片的数据标签。



目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

5.1 配置开发板的HK驱动

5.2 编写数据采集脚本

5.3 配置PC端数据标注环境

5.4 标注数据集

5.5 启动项目文件

5.6 数据集格式规范

5.7 训练

5.8 配置模型转换算子

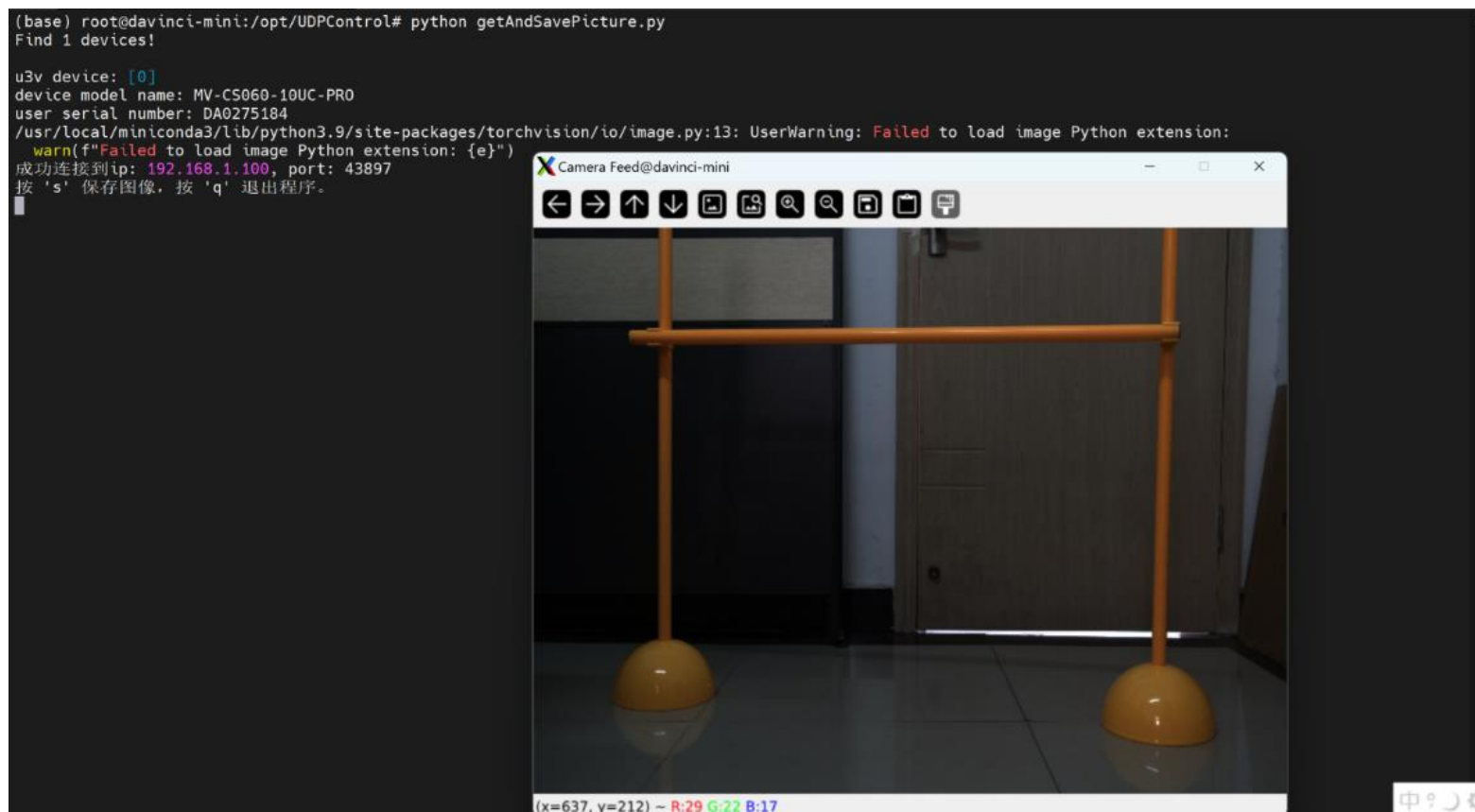
5.9 开发板转换模型

六、逻辑实现（5学时）

七、运行示例

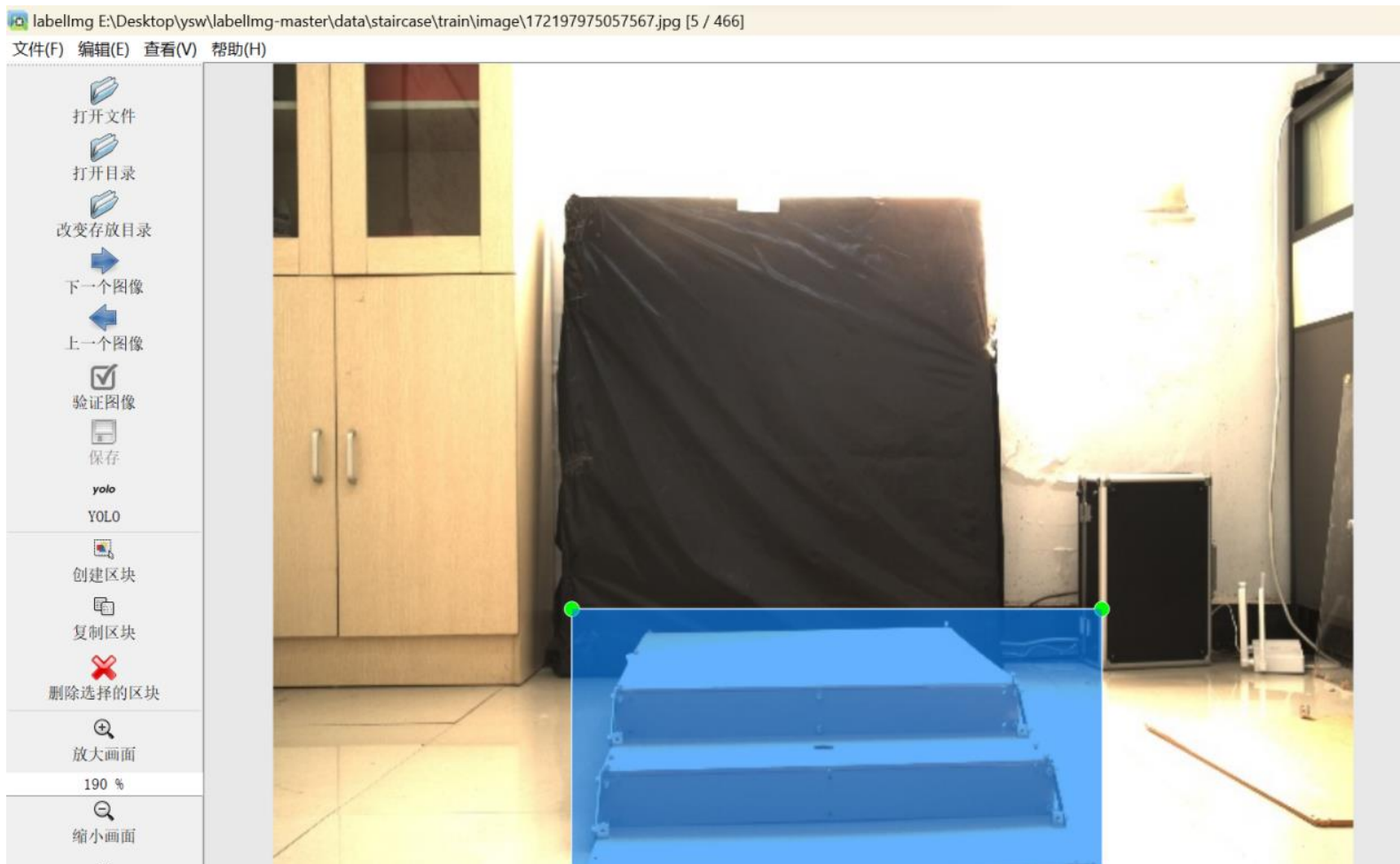
5.4.1 标注数据集

1. 连接开发板，运用脚本采集机器狗视角的图像



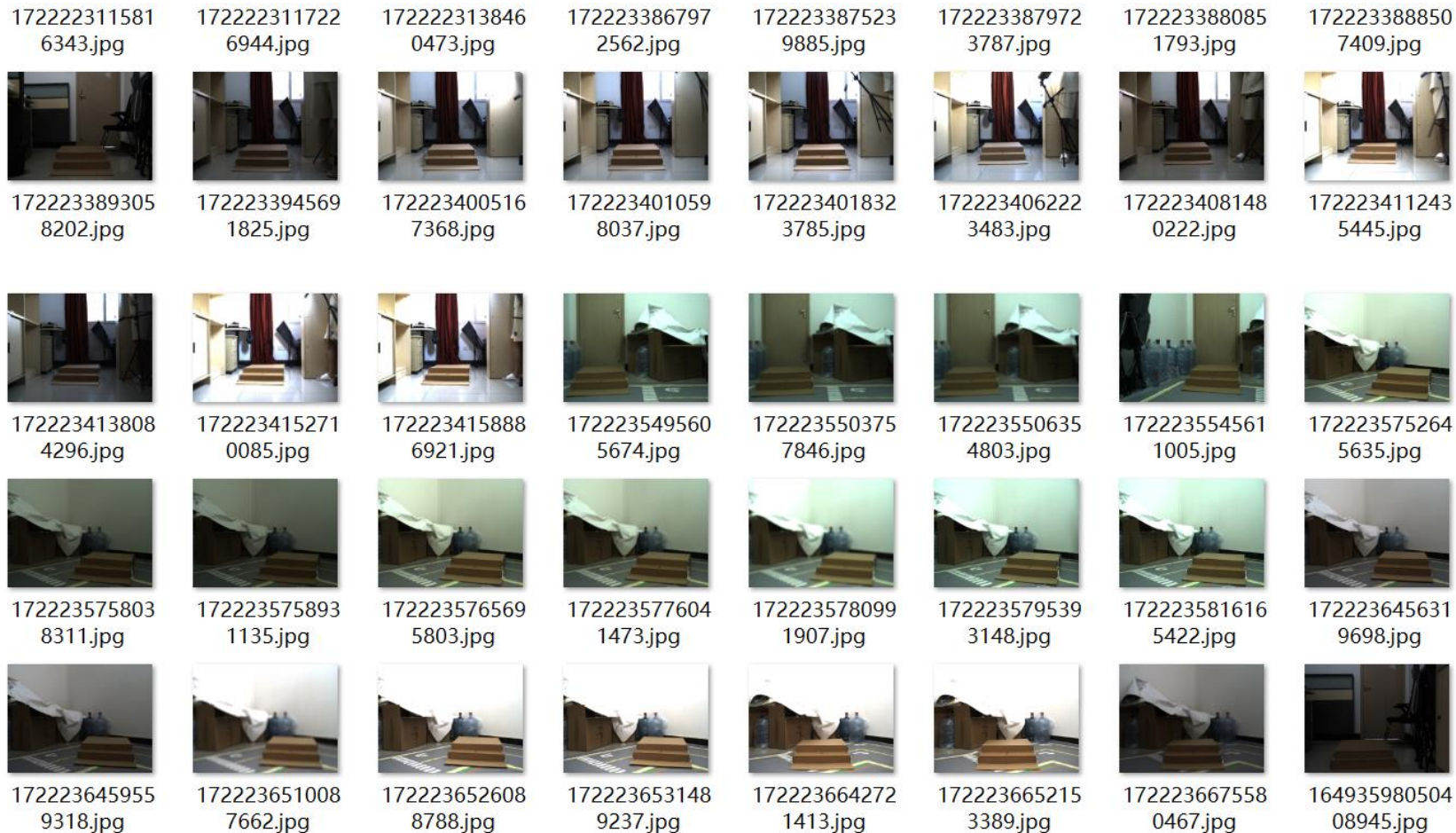
5.4.1 标注数据集

2.进入pc存放labellmg的目录中，运行python labellmg.py



5.4.1 标注数据集

在采集的两类巡检避障产品中，以其中一类staircase为例，可看到如下图所示。

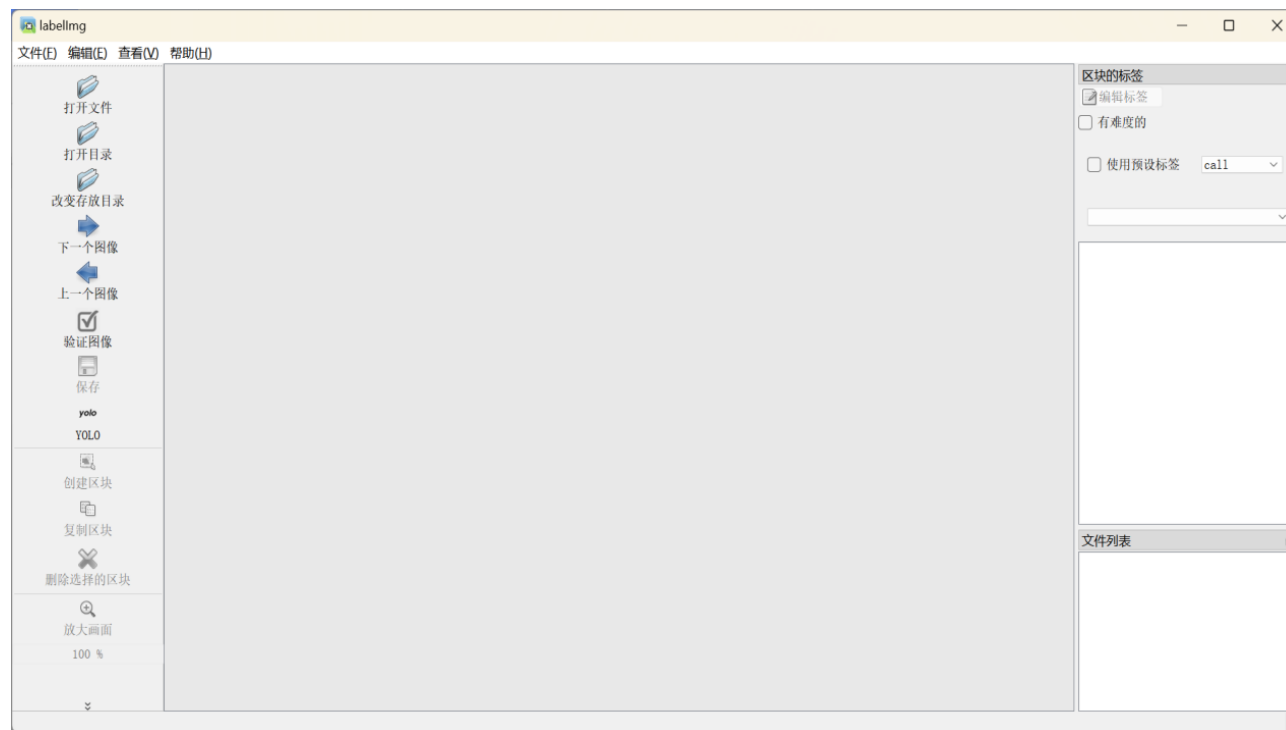


5.4.2使用labelImg进行数据集标注

1、打开anaconda命令行界面

2、同时在predefined_classes.txt中，设置所分类的标签名称；在data文件夹中分别新建images、label文件夹，同时在images文件夹中，新建train文件夹，里面存放训练集手势照片，与之对应应在label文件夹中，新建train文件夹，里面用来存放标注好的数据集信息；同时在images文件夹中，再新建val文件夹，用来存放验证集手势照片，与之对应应在label文件夹中，新建val文件夹，里面用来存放标注好的数据集信息

3、在“打开目录”中打开刚刚的data/images/train文件夹，在“改变存放目录”中更换保存路径为data/label/train文件夹



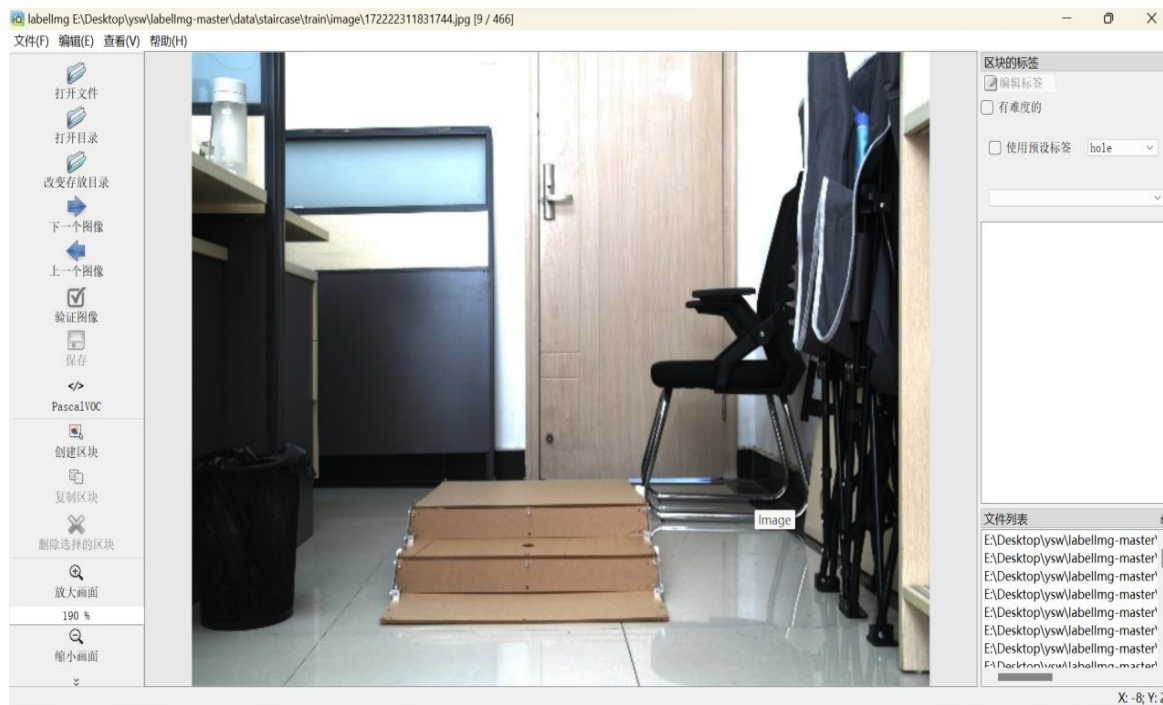
5.4.3数据标定

进入labelImg-master所在文件夹地址

```
(base) PS E:\> cd E:\Desktop\亿生为\labelImg-master
(base) PS E:\Desktop\亿生为\labelImg-master> python labelImg.py
E:\Desktop\亿生为\labelImg-master\labelImg.py:73: DeprecationWarning: sipPyTypeDict() is deprecated, the extension modul
e should use sipPyTypeDictRef() instead
  class MainWindow(QMainWindow, WindowMixin):
[('palm', [(207, 279), (310, 279), (310, 406), (207, 406)], None, None, False)]
```


5.4.3数据标定

选择好“打开目录”即图片所在位置、“改变存放目录”即标签存放位置



后输入`python labellmg.py`进入数据标注页面，

点击”创造区块”，然后出现十字架，进行拖动框住标注区域

然后输入标签，点击palm，再点击左侧的“保存”标签

在 data/label/train 里面可以看到我们的标签，在此介绍 labellmg 的三种标签格式：

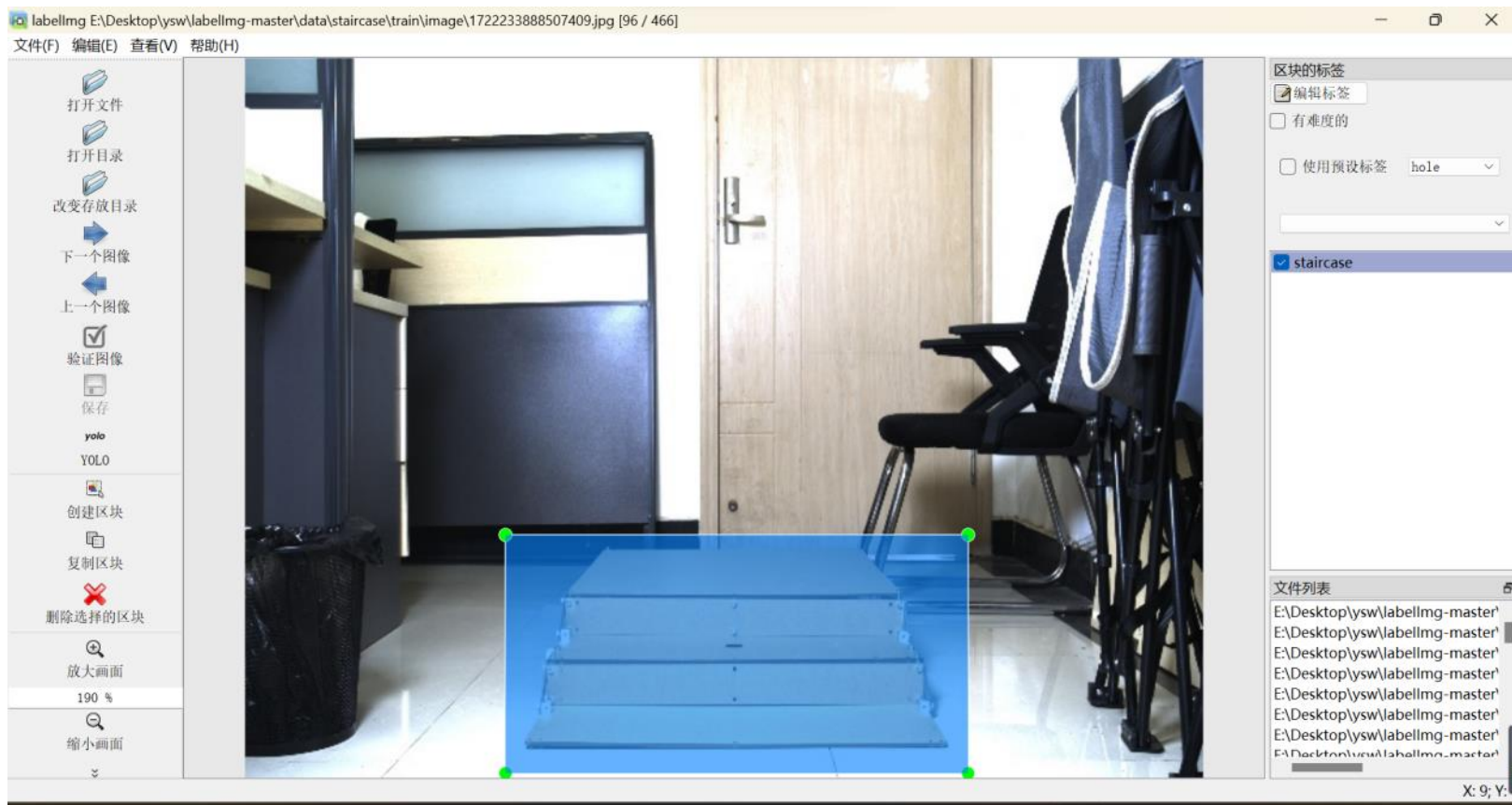
(1) PascalVOC 标签格式，保存为xml文件

(2) YOLO 标签格式，保存为txt文件

(3) CreateML 标签格式，保存为json文件

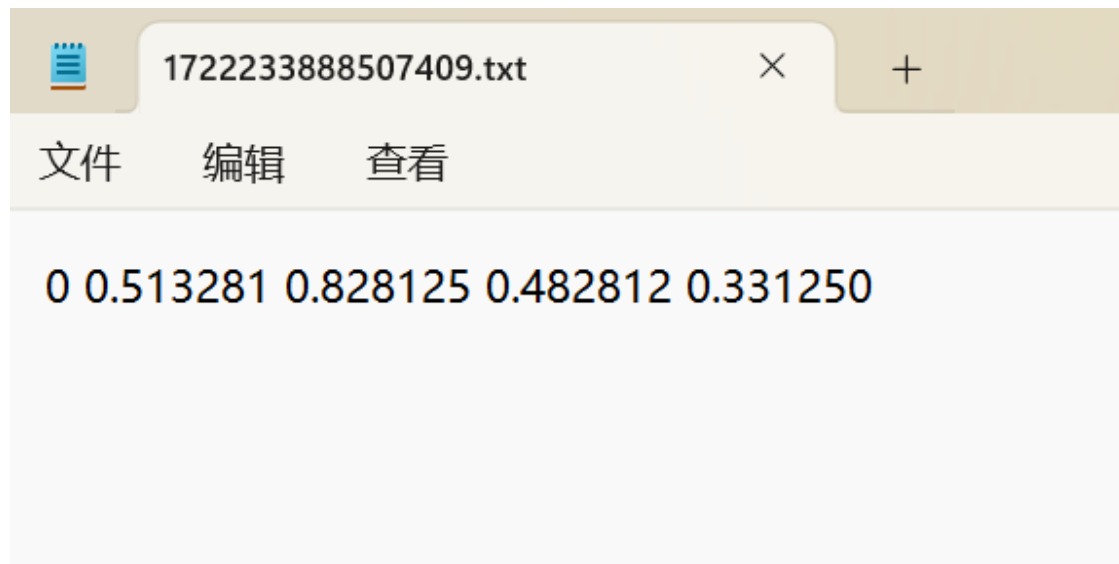
5.4.3数据标定

点击“下一个图像”继续标注下一张



5.4.4数据预览

数据标注完成后，若标注图片是\labeldata\images\train中，则可到\labeldata\labels\train中查看标签文件，images中存放训练集train图，labels中存放训练集train所对应的数据标签txt文件，可看到其文件名对应图片名称，txt中的四列数据分别对应数据标注区块各个含义，分别对应:类别-classes框中心坐标，(x_center,y_center),框长宽(width,height),这里所有的值都进行了归一化具体如下图所示。



目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

5.1 配置开发板的HK驱动

5.2 编写数据采集脚本

5.3 配置PC端数据标注环境

5.4 标注数据集

5.5 启动项目文件

5.6 数据集格式规范

5.7 训练

5.8 配置模型转换算子

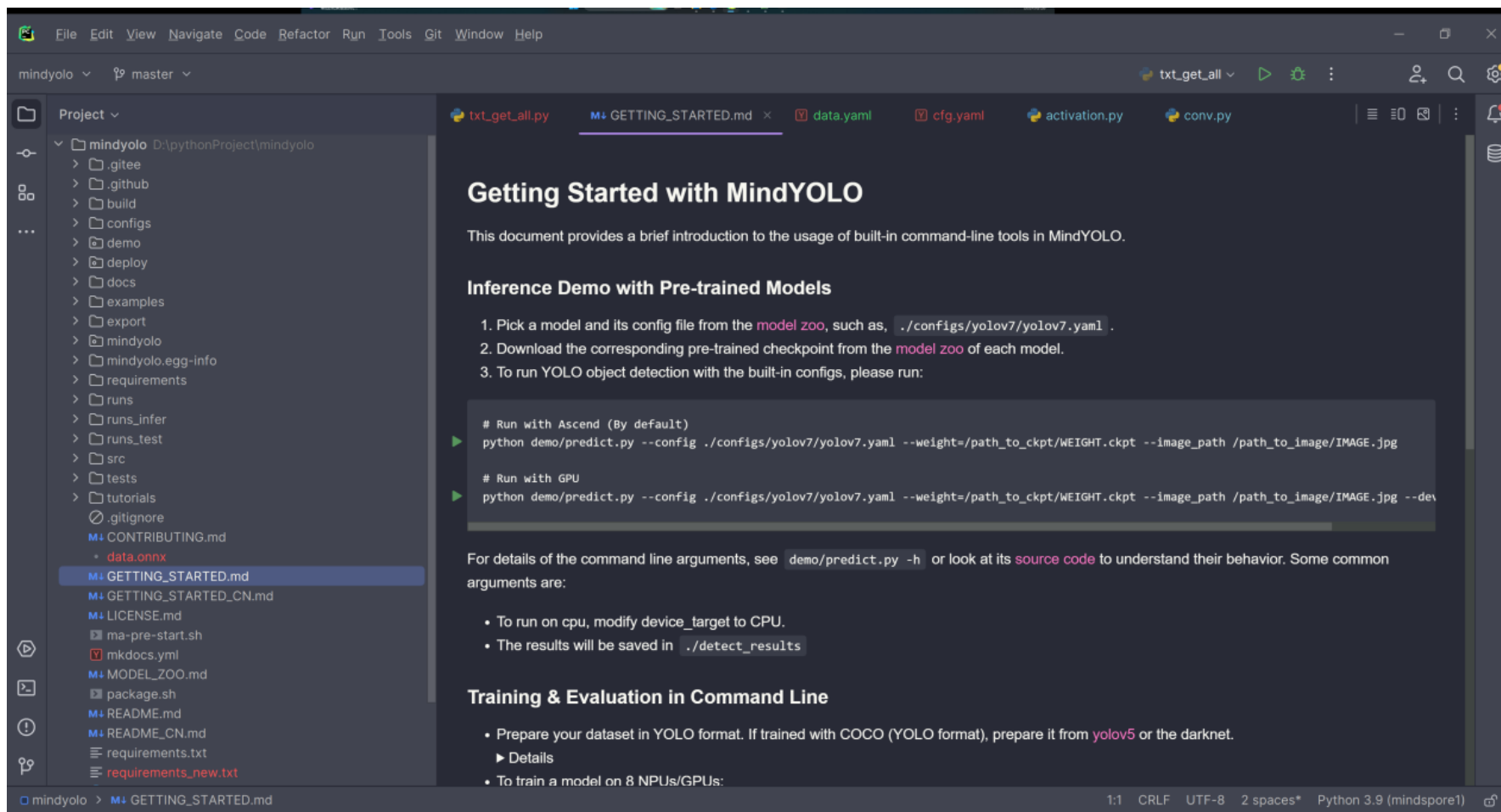
5.9 开发板转换模型

六、逻辑实现（5学时）

七、运行示例

5.5 启动项目文件

右键项目文件夹，选择使用pycharm打开。



如果是这样的那么就成功打开了项目文件，同时要注意右下角的python环境是否正确。

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备（1学时）
- 四、工程文件目录结构
- 五、模型本地化训练及部署（2学时）
 - 5.1 配置开发板的HK驱动
 - 5.2 编写数据采集脚本
 - 5.3 配置PC端数据标注环境
 - 5.4 标注数据集
 - 5.5 启动项目文件
 - 5.6 数据集格式规范
 - 5.7 训练
 - 5.8 配置模型转换算子
 - 5.9 开发板转换模型
- 六、逻辑实现（5学时）
- 七、运行示例

5.6 数据集格式规范

使用我们事先已经采集好的数据集来训练，下面讲解一下数据集目录的结构：

```
mindyolo D:\pythonProject\mindyolo
├── .gitee
├── .github
├── build
├── configs
├── demo
├── deploy
├── docs
├── examples
│   ├── code
│   └── custom_gpu_op
└── dataset1
    ├── images
    │   ├── train
    │   └── val
    ├── labels
    │   ├── train
    │   └── val
    │       ├── train.cache
    │       ├── train.cache.npy
    │       ├── val.cache
    │       └── val.cache.npy
    ├── data.yaml
    ├── predefined_classes.txt
    ├── train.txt
    ├── val.txt
    └── yolov7-tiny_300e_mAP375-d8972c94.ckpt
```

5.6 数据集格式规范

数据集的存放位置：examples/dataset1/

其中images/train存放的是训练的图片，images/val存放的是验证的图片；lables里面则是存放的标注好的标签txt文件。predefines_classes.txt里面是物品的种类名称。

data.yaml文件里面的内容如下：

```
1. __BASE__: [  
2.   '../../configs/yolov7/yolov7-tiny.yaml',  
3. ]  
4.  
5. per_batch_size: 1 # 16 * 8 = 128  
6. img_size: 640  
7.  
8. weight: ./examples/dataset1/yolov7-tiny_300e_mAP375-d8972c94.ckpt  
9. strict_load: False  
10. conf_thres: 0.6  
11. iou_thres: 0.5  
12.  
13. data:  
14.   dataset_name: shwd  
15.   train_set: ./examples/dataset1/train.txt  
16.   val_set: ./examples/dataset1/val.txt  
17.   test_set: ./examples/dataset1/val.txt  
18.  
19.   nc: 2  
20.   names: ['staircase', 'hole']  
21.  
22. optimizer:  
23.   lr_init: 0.001
```

5.6 数据集格式规范

配置文件用于配置 YOLOv7 模型训练的参数，在 MindSpore 深度学习框架中的。下面是对各部分配置的解释：

基础配置

1. `__BASE__`: [
2. `'../../configs/yolov7/yolov7-tiny.yaml'`,
3.]

这一行表示此配置文件继承自 `yolov7-tiny.yaml` 文件。

`__BASE__` 中指定的基础配置文件提供了一些默认设置，而当前配置文件会覆盖或添加一些特定的参数。

训练设置

1. `per_batch_size`: 1
2. `img_size`: 640

`per_batch_size`: 1: 每个批次的大小设置为 1，通常在小规模测试或调试时使用较小的批次大小。

`img_size`: 640: 输入图像的尺寸，图像会被调整为 640×640 像素。

预训练模型

1. `weight`: `./examples/dataset1/yolov7-tiny_300e_mAP375-d8972c94.ckpt`
2. `strict_load`: `False`

`weight`: 指定了预训练模型的权重文件路径。

`strict_load`: `False`: 表示加载预训练权重时不严格匹配所有参数。这对于使用不同结构的预训练模型或部分加载权重非常有用。

5.6 数据集格式规范

检测阈值

1. `conf_thres`: 0.6

2. `iou_thres`: 0.5

`conf_thres`: 0.6: 置信度阈值，只有置信度高于 0.6 的检测结果才会被保留。

`iou_thres`: 0.5: IoU (Intersection over Union) 阈值，用于非极大值抑制 (NMS)，决定哪些框会被保留。

数据集配置

1. `data`:

2. `dataset_name`: shwd

3. `train_set`: ./examples/dataset1/train.txt

4. `val_set`: ./examples/dataset1/val.txt

5. `test_set`: ./examples/dataset1/val.txt

6.

7. `nc`: 2

8. `names`: ['staircase', 'hole']

`dataset_name`: shwd: 数据集名称。

`train_set`、`val_set`、`test_set`: 分别指定训练集、验证集和测试集的文件路径，这些文件通常包含图片的路径列表。

`nc`: 2: 类别数量，这里表示有2个类别。

`names`: ['staircase', 'hole']: 类别名称列表，按顺序列出每个类别的名称。

5.6 数据集格式规范

优化器配置

1. yaml
2. optimizer:
3. lr_init: 0.001

lr_init: 0.001: 初始学习率。学习率决定了模型参数更新的步长，较高的学习率会使模型训练更快，但可能会不稳定；较低的学习率训练速度慢但更稳定。

这段配置文件指定了 YOLOv7-tiny 模型训练的具体参数，包括数据集路径、图像大小、批次大小、预训练权重、检测阈值、类别数量及其名称和优化器参数。通过修改这些参数，可以灵活地调整模型训练过程中的各个方面。

5.6 数据集格式规范

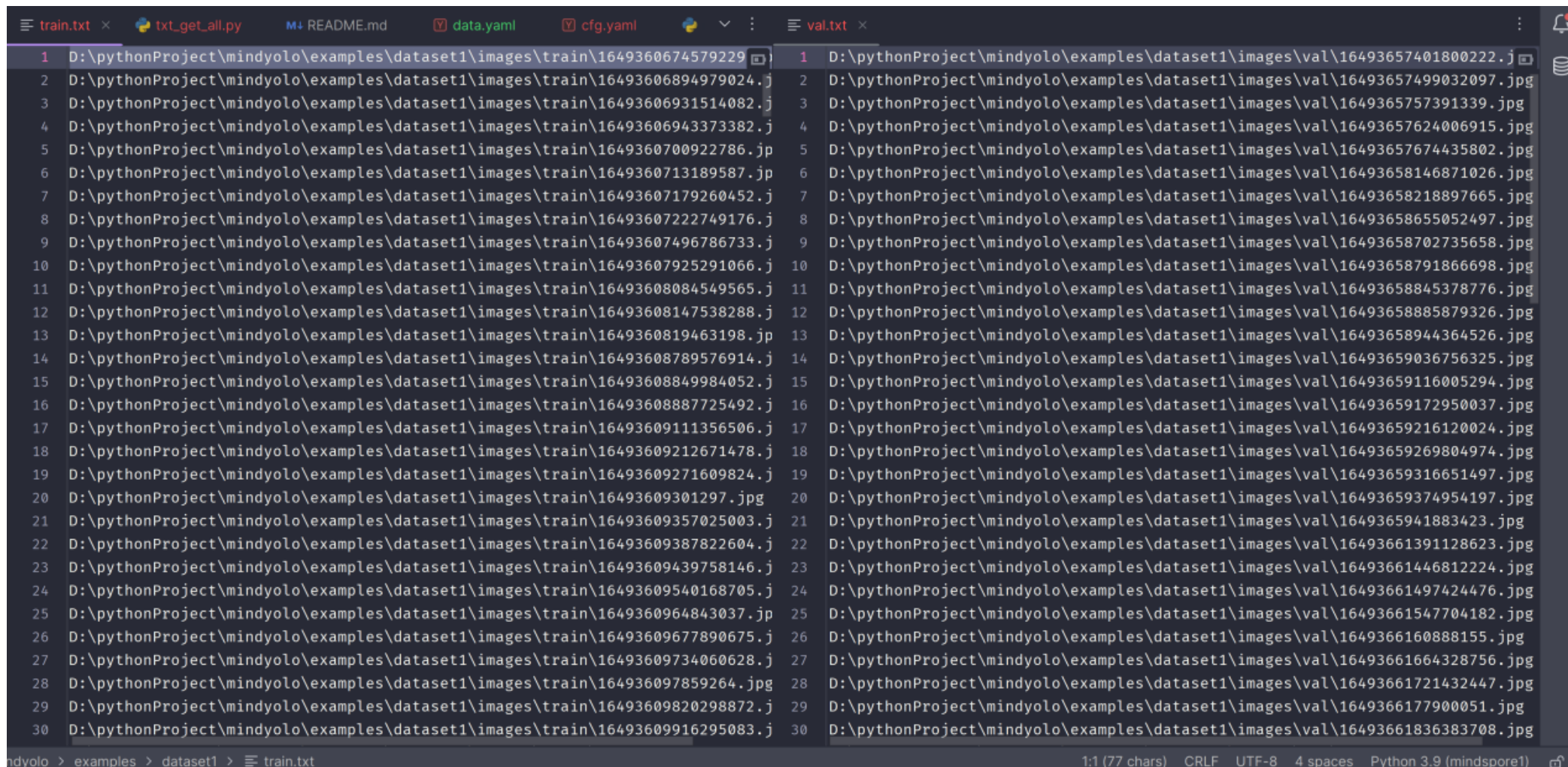
train.txt文件和val.txt文件里面存放的时候图片路径。txt_get_all.py脚本存放在了examples文件夹下。代码如下：

```
1. import os
2. sets = ['train', 'val']
3. for j in sets:
4.     # 图片和标签文件的目录
5.     images_dir = fr'D:\pythonProject\mindyolo\examples\dataset1\images\{j}'
6.     labels_dir = fr'D:\pythonProject\mindyolo\examples\dataset1\labels\{j}'
7.     # 要写入的txt文件路径
8.     output_txt_path = fr'D:\pythonProject\mindyolo\examples\dataset1\{j}.txt'
9.     # 获取所有图片文件的名称
10.    image_files = [f for f in os.listdir(images_dir) if f.endswith('.jpg')]
11.    number1, number2 = 0, 0
12.    picture_list = []
13.    file_contents = []
14.    # 打开输出文件
15.    for image_file in image_files:
16.        # 构建图片的完整路径
17.        image_path = os.path.join(images_dir, image_file)
18.        picture_list.append(image_path)
19.        number1 += 1
20.    print(picture_list)
21.    with open(output_txt_path, 'w', encoding='utf-8') as output_file:
22.        for i in range(number1):
23.            print(i)
24.            output_file.write(f"{picture_list[i]}\n")
```

本实验中使用的是yolov7-tiny_300e_mAP375-d8972c94.ckpt权重模型。

5.6 数据集格式规范

在使用的时候只需要注意images_dir,labels_dir,output_txt_path这三个参数的路径即可，运行之后可以得到如下文件：



```
train.txt
1 D:\pythonProject\mindyolo\examples\dataset1\images\train\1649360674579229.jpg
2 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493606894979024.jpg
3 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493606931514082.jpg
4 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493606943373382.jpg
5 D:\pythonProject\mindyolo\examples\dataset1\images\train\1649360700922786.jpg
6 D:\pythonProject\mindyolo\examples\dataset1\images\train\1649360713189587.jpg
7 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493607179260452.jpg
8 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493607222749176.jpg
9 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493607496786733.jpg
10 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493607925291066.jpg
11 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493608084549565.jpg
12 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493608147538288.jpg
13 D:\pythonProject\mindyolo\examples\dataset1\images\train\1649360819463198.jpg
14 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493608789576914.jpg
15 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493608849984052.jpg
16 D:\pythonProject\mindyolo\examples\dataset1\images\train\1649360887725492.jpg
17 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609111356506.jpg
18 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609212671478.jpg
19 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609271609824.jpg
20 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609301297.jpg
21 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609357025003.jpg
22 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609387822604.jpg
23 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609439758146.jpg
24 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609540168705.jpg
25 D:\pythonProject\mindyolo\examples\dataset1\images\train\1649360964843037.jpg
26 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609677890675.jpg
27 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609734060628.jpg
28 D:\pythonProject\mindyolo\examples\dataset1\images\train\164936097859264.jpg
29 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609820298872.jpg
30 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609916295083.jpg

val.txt
1 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493657401800222.jpg
2 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493657499032097.jpg
3 D:\pythonProject\mindyolo\examples\dataset1\images\val\1649365757391339.jpg
4 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493657624006915.jpg
5 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493657674435802.jpg
6 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493658146871026.jpg
7 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493658218897665.jpg
8 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493658655052497.jpg
9 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493658702735658.jpg
10 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493658791866698.jpg
11 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493658845378776.jpg
12 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493658885879326.jpg
13 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493658944364526.jpg
14 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493659036756325.jpg
15 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493659116005294.jpg
16 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493659172950037.jpg
17 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493659216120024.jpg
18 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493659269804974.jpg
19 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493659316651497.jpg
20 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493659374954197.jpg
21 D:\pythonProject\mindyolo\examples\dataset1\images\val\1649365941883423.jpg
22 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493661391128623.jpg
23 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493661446812224.jpg
24 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493661497424476.jpg
25 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493661547704182.jpg
26 D:\pythonProject\mindyolo\examples\dataset1\images\val\1649366160888155.jpg
27 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493661664328756.jpg
28 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493661721432447.jpg
29 D:\pythonProject\mindyolo\examples\dataset1\images\val\1649366177900051.jpg
30 D:\pythonProject\mindyolo\examples\dataset1\images\val\16493661836383708.jpg
```

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备（1学时）
- 四、工程文件目录结构
- 五、模型本地化训练及部署（2学时）
 - 5.1 配置开发板的HK驱动
 - 5.2 编写数据采集脚本
 - 5.3 配置PC端数据标注环境
 - 5.4 标注数据集
 - 5.5 启动项目文件
 - 5.6 数据集格式规范
 - 5.7 训练
 - 5.8 配置模型转换算子
 - 5.9 开发板转换模型
- 六、逻辑实现（5学时）
- 七、运行示例

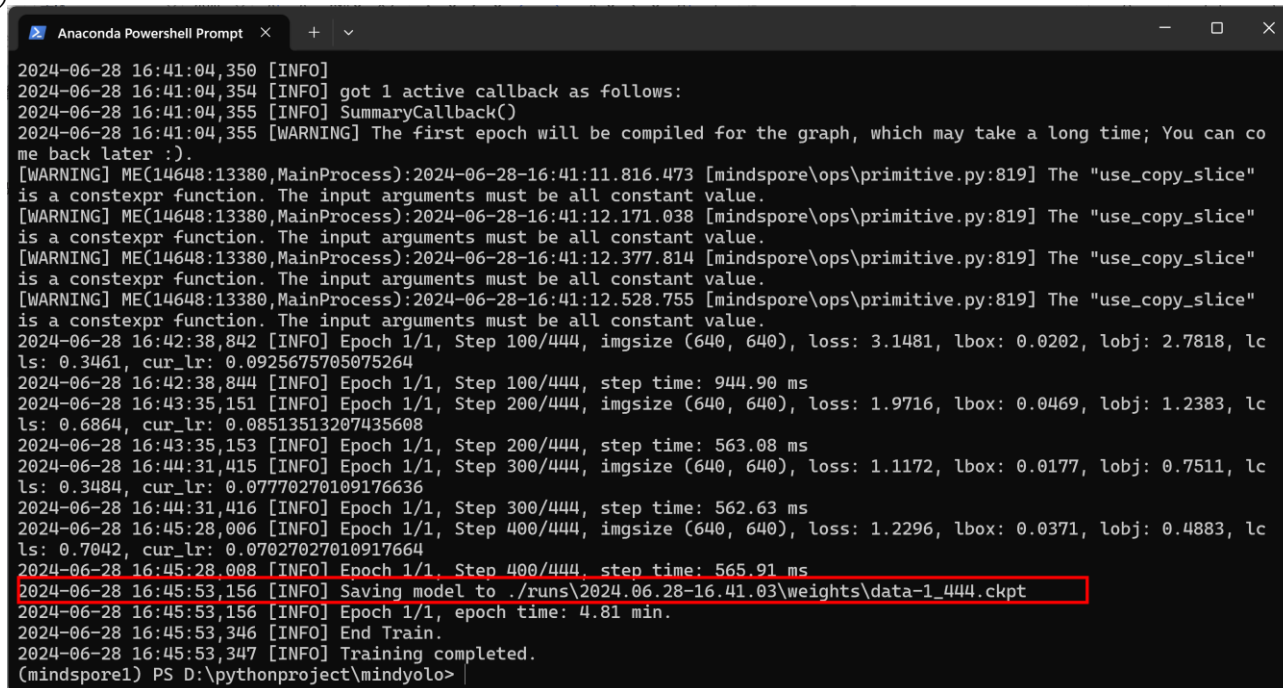
5.7 训练

训练的指令如下：

```
python train.py --config ./examples/dataset1/data.yaml --epochs 300 --device_target CPU
```

这里的epochs可以自己定义多少，然后由于框架限制的原因只能使用cpu训练，如果需要用gpu加速训练，超过了本实验教学的范畴，可以自行线下学习。

输入指令回车之后如果出现了下面的内容，则证明训练成功。（为了做演示，这里值训练1轮，使用了400多张图片，实际训练需要更多的epochs和图片。）



```
2024-06-28 16:41:04,350 [INFO]
2024-06-28 16:41:04,354 [INFO] got 1 active callback as follows:
2024-06-28 16:41:04,355 [INFO] SummaryCallback()
2024-06-28 16:41:04,355 [WARNING] The first epoch will be compiled for the graph, which may take a long time; You can come back later :).
[WARNING] ME(14648:13380,MainProcess):2024-06-28-16:41:11.816.473 [mindspore\ops\primitive.py:819] The "use_copy_slice" is a constexpr function. The input arguments must be all constant value.
[WARNING] ME(14648:13380,MainProcess):2024-06-28-16:41:12.171.038 [mindspore\ops\primitive.py:819] The "use_copy_slice" is a constexpr function. The input arguments must be all constant value.
[WARNING] ME(14648:13380,MainProcess):2024-06-28-16:41:12.377.814 [mindspore\ops\primitive.py:819] The "use_copy_slice" is a constexpr function. The input arguments must be all constant value.
[WARNING] ME(14648:13380,MainProcess):2024-06-28-16:41:12.528.755 [mindspore\ops\primitive.py:819] The "use_copy_slice" is a constexpr function. The input arguments must be all constant value.
2024-06-28 16:42:38,842 [INFO] Epoch 1/1, Step 100/444, imgsize (640, 640), loss: 3.1481, lbox: 0.0202, lobj: 2.7818, lcls: 0.3461, cur_lr: 0.0925675705075264
2024-06-28 16:42:38,844 [INFO] Epoch 1/1, Step 100/444, step time: 944.90 ms
2024-06-28 16:43:35,151 [INFO] Epoch 1/1, Step 200/444, imgsize (640, 640), loss: 1.9716, lbox: 0.0469, lobj: 1.2383, lcls: 0.6864, cur_lr: 0.08513513207435608
2024-06-28 16:43:35,153 [INFO] Epoch 1/1, Step 200/444, step time: 563.08 ms
2024-06-28 16:44:31,415 [INFO] Epoch 1/1, Step 300/444, imgsize (640, 640), loss: 1.1172, lbox: 0.0177, lobj: 0.7511, lcls: 0.3484, cur_lr: 0.07770270109176636
2024-06-28 16:44:31,416 [INFO] Epoch 1/1, Step 300/444, step time: 562.63 ms
2024-06-28 16:45:28,006 [INFO] Epoch 1/1, Step 400/444, imgsize (640, 640), loss: 1.2296, lbox: 0.0371, lobj: 0.4883, lcls: 0.7042, cur_lr: 0.07027027010917664
2024-06-28 16:45:28,008 [INFO] Epoch 1/1, Step 400/444, step time: 565.91 ms
2024-06-28 16:45:53,156 [INFO] Saving model to ./runs\2024.06.28-16.41.03\weights\data-1_444.ckpt
2024-06-28 16:45:53,156 [INFO] Epoch 1/1, epoch time: 4.81 min.
2024-06-28 16:45:53,346 [INFO] End Train.
2024-06-28 16:45:53,347 [INFO] Training completed.
(mindspore1) PS D:\pythonproject\mindyolo>
```

训练成功之后的模型文件保存到了./runs\2024.06.28-16.41.03\weights\data-1_444.ckpt中。

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备（1学时）
- 四、工程文件目录结构
- 五、模型本地化训练及部署（2学时）
 - 5.1 配置开发板的HK驱动
 - 5.2 编写数据采集脚本
 - 5.3 配置PC端数据标注环境
 - 5.4 标注数据集
 - 5.5 启动项目文件
 - 5.6 数据集格式规范
 - 5.7 训练
 - 5.8 配置模型转换算子
 - 5.9 开发板转换模型
- 六、逻辑实现（5学时）
- 七、运行示例

5.8 配置模型转换算子

将ckpt转换为onnx模型需要自行添加算子，首先mindyolo/models/layers文件夹，然后找到activation.py文件在里面添加如下所示代码：

```
20 class EdgeSiLU(nn.Cell):
21     """
22     SiLU activation function: x * sigmoid(x). To support ONNX export.
23     """
24
25     def __init__(self):
26         super().__init__()
27
28     def construct(self, x):
29         return x * ops.sigmoid(x)
```

```
1. class EdgeSiLU(nn.Cell):
2.     """
3.     SiLU activation function: x * sigmoid(x).
4.     To support ONNX export.
5.     """
6.     def __init__(self):
7.         super().__init__()
8.
9.     def construct(self, x):
10.         return x * ops.sigmoid(x)
```

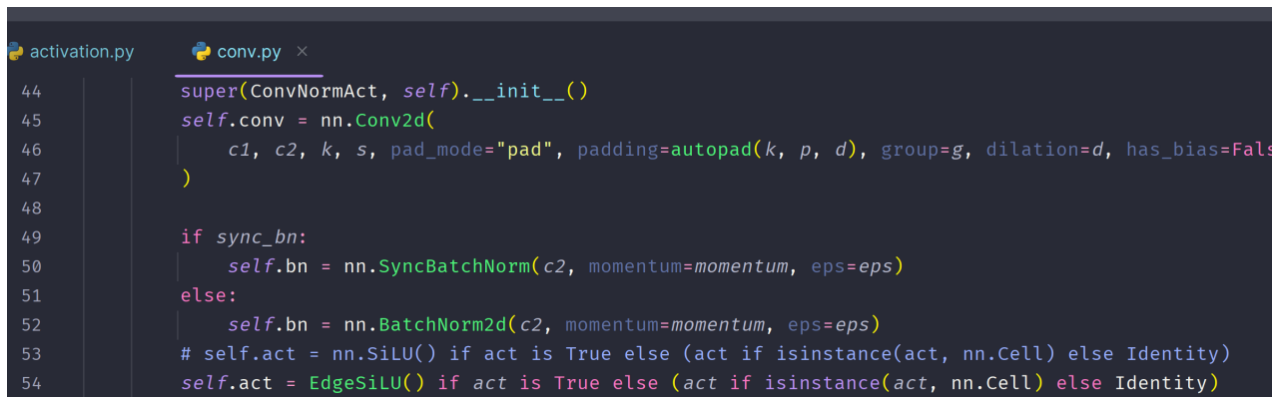

5.8 配置模型转换算子

然后再找到conv.py文件。首先导入我们自己定义的算子

1. `from .activation import EdgeSiLU` # 导入自定义的EdgeSiLU

```
6 from .activation import EdgeSiLU # 导入自定义的EdgeSiLU
```

再接着往下滑动，找到这一行

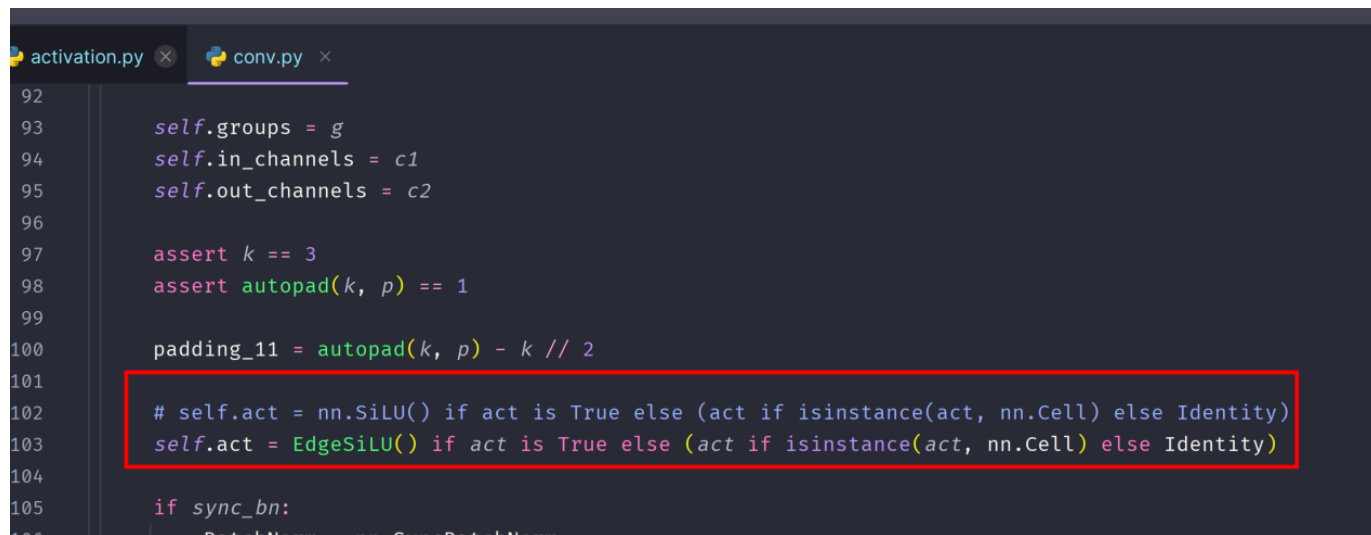


```
44 super(ConvNormAct, self).__init__()
45 self.conv = nn.Conv2d(
46     c1, c2, k, s, pad_mode="pad", padding=autpad(k, p, d), group=g, dilation=d, has_bias=False
47 )
48
49 if sync_bn:
50     self.bn = nn.SyncBatchNorm(c2, momentum=momentum, eps=eps)
51 else:
52     self.bn = nn.BatchNorm2d(c2, momentum=momentum, eps=eps)
53 # self.act = nn.SiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)
54 self.act = EdgeSiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)
```

1. `# self.act = nn.SiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)`
2. `self.act = EdgeSiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)`

将原来的self.act注释掉，然后换成我们自定义的算子。

5.8 配置模型转换算子



```
92
93     self.groups = g
94     self.in_channels = c1
95     self.out_channels = c2
96
97     assert k == 3
98     assert autopad(k, p) == 1
99
100    padding_11 = autopad(k, p) - k // 2
101
102    # self.act = nn.SiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)
103    self.act = EdgeSiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)
104
105    if sync_bn:
106        PatchNorm = nn.SyncBatchNorm
```

1. `# self.act = nn.SiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)`
2. `self.act = EdgeSiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)`

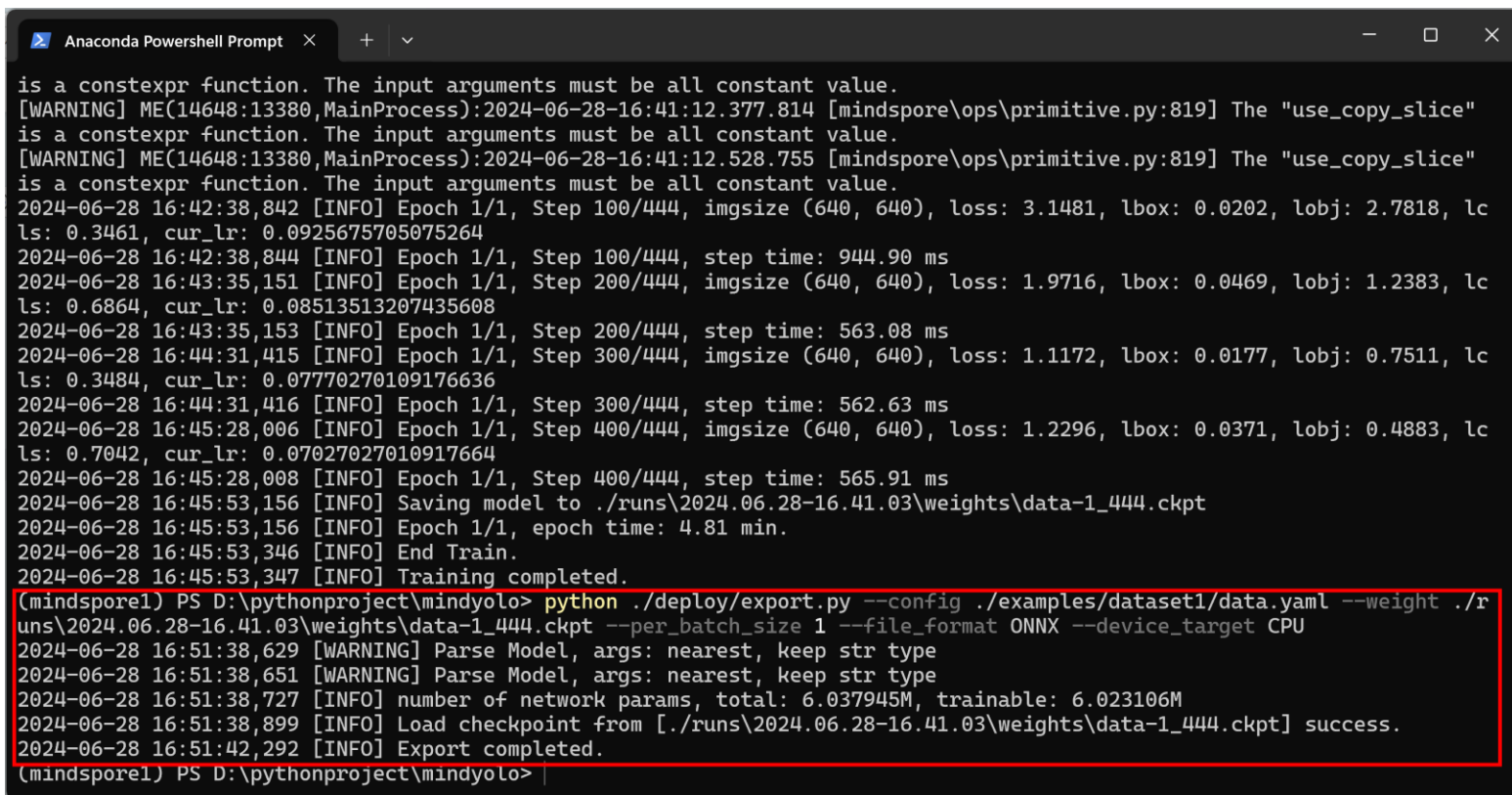
然后大概在100多行的位置，也是要修改掉，一共需要修改两处。

5.8 配置模型转换算子

修改之后我们继续输入模型转换的指令：

```
python ./deploy/export.py --config ./examples/dataset1/data.yaml --weight ./runs\2024.06.28-16.41.03\weights\data-1_444.ckpt --per_batch_size 1 --file_format ONNX --device_target CPU
```

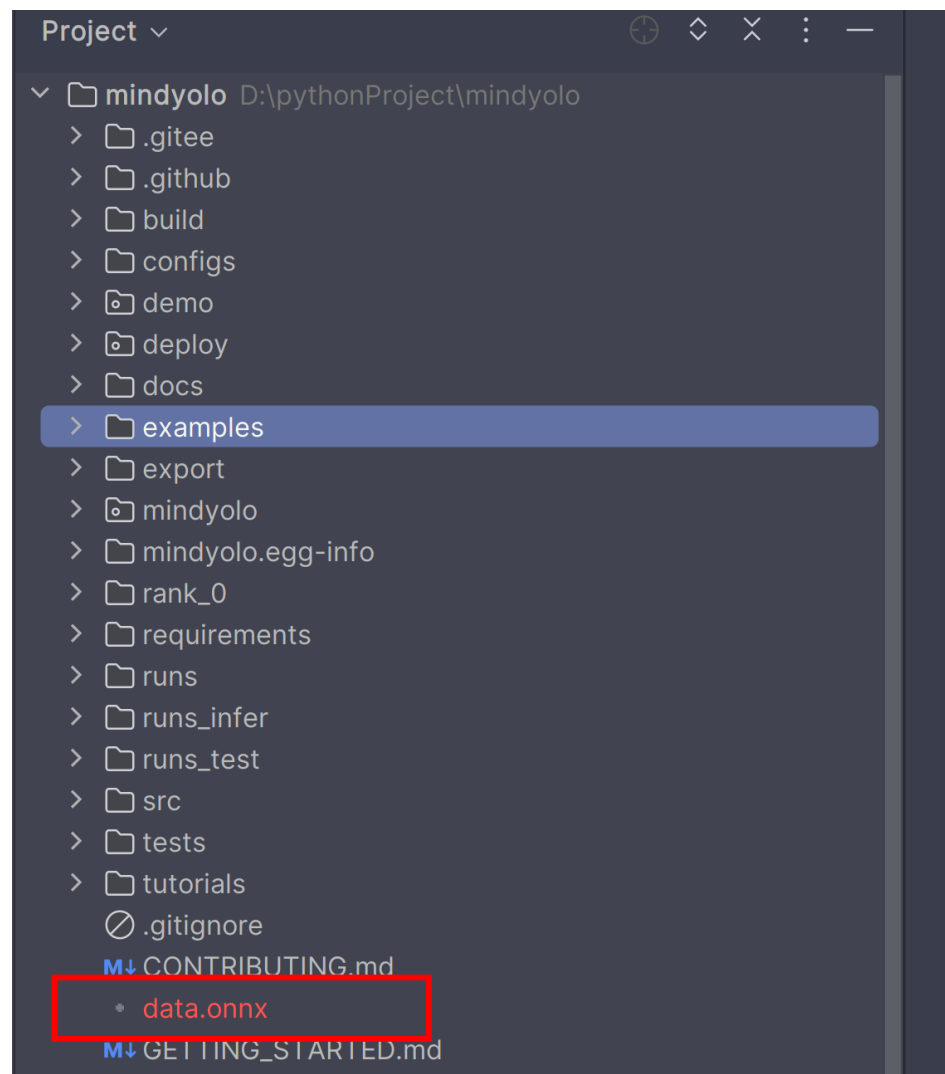
转换成功之后的显示如下图



```
is a constexpr function. The input arguments must be all constant value.
[WARNING] ME(14648:13380,MainProcess):2024-06-28-16:41:12.377.814 [mindspore\ops\primitive.py:819] The "use_copy_slice"
is a constexpr function. The input arguments must be all constant value.
[WARNING] ME(14648:13380,MainProcess):2024-06-28-16:41:12.528.755 [mindspore\ops\primitive.py:819] The "use_copy_slice"
is a constexpr function. The input arguments must be all constant value.
2024-06-28 16:42:38,842 [INFO] Epoch 1/1, Step 100/444, imgsize (640, 640), loss: 3.1481, lbox: 0.0202, lobj: 2.7818, lc
ls: 0.3461, cur_lr: 0.0925675705075264
2024-06-28 16:42:38,844 [INFO] Epoch 1/1, Step 100/444, step time: 944.90 ms
2024-06-28 16:43:35,151 [INFO] Epoch 1/1, Step 200/444, imgsize (640, 640), loss: 1.9716, lbox: 0.0469, lobj: 1.2383, lc
ls: 0.6864, cur_lr: 0.08513513207435608
2024-06-28 16:43:35,153 [INFO] Epoch 1/1, Step 200/444, step time: 563.08 ms
2024-06-28 16:44:31,415 [INFO] Epoch 1/1, Step 300/444, imgsize (640, 640), loss: 1.1172, lbox: 0.0177, lobj: 0.7511, lc
ls: 0.3484, cur_lr: 0.07770270109176636
2024-06-28 16:44:31,416 [INFO] Epoch 1/1, Step 300/444, step time: 562.63 ms
2024-06-28 16:45:28,006 [INFO] Epoch 1/1, Step 400/444, imgsize (640, 640), loss: 1.2296, lbox: 0.0371, lobj: 0.4883, lc
ls: 0.7042, cur_lr: 0.07027027010917664
2024-06-28 16:45:28,008 [INFO] Epoch 1/1, Step 400/444, step time: 565.91 ms
2024-06-28 16:45:53,156 [INFO] Saving model to ./runs\2024.06.28-16.41.03\weights\data-1_444.ckpt
2024-06-28 16:45:53,156 [INFO] Epoch 1/1, epoch time: 4.81 min.
2024-06-28 16:45:53,346 [INFO] End Train.
2024-06-28 16:45:53,347 [INFO] Training completed.
(mindspore1) PS D:\pythonproject\mindyolo> python ./deploy/export.py --config ./examples/dataset1/data.yaml --weight ./r
uns\2024.06.28-16.41.03\weights\data-1_444.ckpt --per_batch_size 1 --file_format ONNX --device_target CPU
2024-06-28 16:51:38,629 [WARNING] Parse Model, args: nearest, keep str type
2024-06-28 16:51:38,651 [WARNING] Parse Model, args: nearest, keep str type
2024-06-28 16:51:38,727 [INFO] number of network params, total: 6.037945M, trainable: 6.023106M
2024-06-28 16:51:38,899 [INFO] Load checkpoint from [./runs\2024.06.28-16.41.03\weights\data-1_444.ckpt] success.
2024-06-28 16:51:42,292 [INFO] Export completed.
(mindspore1) PS D:\pythonproject\mindyolo>
```

5.8 配置模型转换算子

可以看到上面显示export completed说明模型转换成功，转换之后的模型文件保留在了与train.py的同级目录下，data.onnx模型文件，就是我们想要的模型文件



目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

5.1 配置开发板的HK驱动

5.2 编写数据采集脚本

5.3 配置PC端数据标注环境

5.4 标注数据集

5.5 启动项目文件

5.6 数据集格式规范

5.7 训练

5.8 配置模型转换算子

5.9 开发板转换模型

六、逻辑实现（5学时）

七、运行示例

5.9 开发板转换模型

我们再将data.onnx文件上传到开发板子中，运行下面的指令：

```
atc --model=data.onnx --framework=5 --soc_version=Ascend310B4 --output=best --input_format=ND
```

然后等待转换模型

```
(base) root@davinci-mini:/opt/ddd# atc --model=data.onnx --framework=5 --soc_version=Ascend310B4 --output=best --input_format=ND
ATC start working now, please wait for a moment.
```

```
(base) root@davinci-mini:/opt/ddd# atc --model=data.onnx --framework=5 --soc_version=Ascend310B4 --output=best --input_format=ND
ATC start working now, please wait for a moment.
```

```
...
ATC run success, welcome to the next use.
```

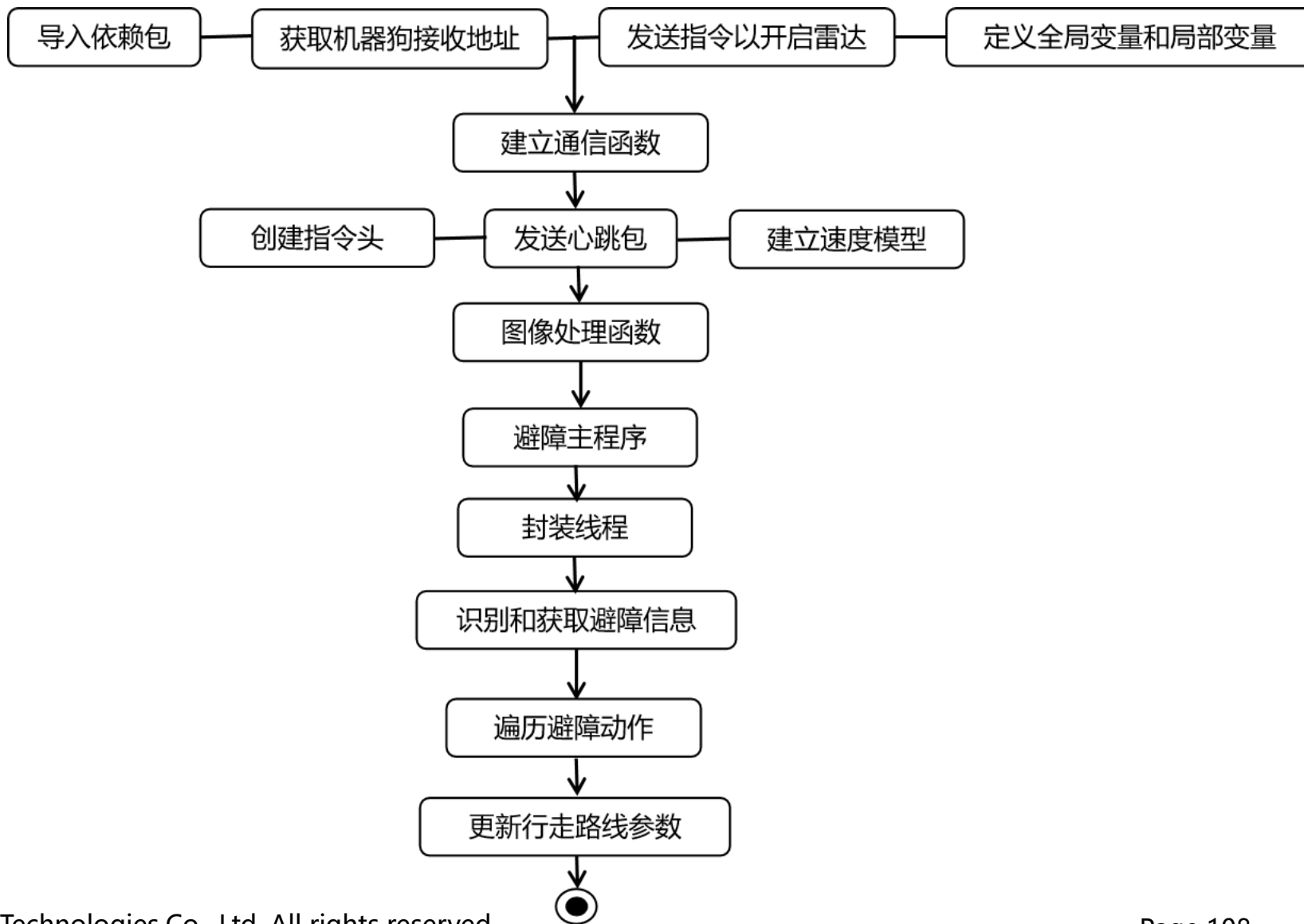
```
(base) root@davinci-mini:/opt/ddd# ls
best.om  data.onnx  fusion_result.json
```

当出现success的时候说明转换成功了，可以查看到已经有叫best.om的模型文件输出了。

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备 (1学时)
- 四、工程文件目录结构
- 五、模型本地化训练及部署 (2学时)
- 六、逻辑实现 (5学时)
 - 6.1 建立通信函数
 - 6.2 发送心跳包
 - 6.3 创建指令头
 - 6.4 建立速度模型
 - 6.5 图像处理函数
 - 6.6 封装线程
 - 6.7 避障主程序
 - 6.7.1 导入依赖包
 - 6.7.2 获取机器狗接收地址
 - 6.7.3 发送指令以开启雷达
 - 6.7.4 遍历避障动作
 - 6.7.5 更新行走路线参数
 - 6.7.6 定义全局变量和局部变量
 - 6.8 识别和获取避障信息
- 七、运行示例

逻辑实现



目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备 (1学时)
- 四、工程文件目录结构
- 五、模型本地化训练及部署 (2学时)
- 六、逻辑实现 (5学时)
 - 6.1 建立通信函数
 - 6.2 发送心跳包
 - 6.3 创建指令头
 - 6.4 建立速度模型
 - 6.5 图像处理函数
 - 6.6 封装线程
 - 6.7 避障主程序
 - 6.7.1 导入依赖包
 - 6.7.2 获取机器狗接收地址
 - 6.7.3 发送指令以开启雷达
 - 6.7.4 遍历避障动作
 - 6.7.5 更新行走路线参数
 - 6.7.6 定义全局变量和局部变量
 - 6.8 识别和获取避障信息
- 七、运行示例

6.1 建立通信函数

简单控制第一步：建立通信链接，socketnetwork/network_utils.py文件定义了一个函数 `setup_socket_and_address`，用于创建一个UDP套接字，并设置机器狗的IP地址和端口号作为目标地址。该通信链接是所有后续控制和数据传输的基础。不仅仅确保了指令传输的可靠性和及时性，还实现对机器狗的远程控制和监控以及为数据传输提供通道，代码如下：

```
1. import socket
2. from typing import *
3.
4. def setup_socket_and_address(dest_ip = '192.168.1.120',
   port=43893) -> Tuple[socket.socket, Tuple[str, int]]:
5.     # 创建UDP套接字
6.     sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
7.
8.     # 设置目标地址
9.     target_address = (dest_ip, port)
10.    # print(target_address)
11.
12.    return sfd, target_address
```

目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

6.1 建立通信函数

6.2 发送心跳包

6.3 创建指令头

6.4 建立速度模型

6.5 图像处理函数

6.6 封装线程

6.7 避障主程序

6.7.1 导入依赖包

6.7.2 获取机器狗接收地址

6.7.3 发送指令以开启雷达

6.7.4 遍历避障动作

6.7.5 更新行走路线参数

6.7.6 定义全局变量和局部变量

6.8 识别和获取避障信息

七、运行示例

6.2 发送心跳包

心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。这对于实时控制和数据传输至关重要。确保系统在运行过程中不会因为通信问题而出现故障，提高系统的整体可靠性。

发送心跳包代码位于sendcommand/heartbeat.py，代码如下：

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. import socket
5. import struct
6. import time
7.
8. def send_udp_heartbeat(sfd, target_address, code=0x21040001, parameters_size=0, type=0, heartbeat_interval=0.25) -
   > None:
9.     # 打包心跳指令数据
10.     heartbeat_command = struct.pack('<III', code, parameters_size, type)
11.
12.     try:
13.         while True:
14.             # 记录发送前的时间
15.             start_time = time.time()
16.
17.             # 发送心跳包
18.             sfd.sendto(heartbeat_command, target_address)
19.
20.             # 发送后的延时
21.             time.sleep(max(0, heartbeat_interval - (time.time() - start_time)))
22.
23.     except KeyboardInterrupt:
24.         print("Heartbeat sending stopped by user.")
25.     finally:
26.         # 关闭socket
27.         sfd.close()
```

一个socket对象，用于发送数据

参数大小，默认为 0

心跳包的类型，默认为 0

目标地址的元组，包含IP地址和端口号

心跳指令的代码，默认为 0x21040001

心跳包发送间隔，默认为 0.25 秒

6.2 发送心跳包

启动心跳线程, 代码如下:

```
1. # 建立通讯
2. sfd, target_address = network_utils.setup_socket_and_address()
3.
4. # 定义发送心跳包的线程执行体
5. # 注意*args必须以tuple形式传入, 包括sfd和target_address
6. heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_udp_heartbeat, 0.25, None
, sfd, target_address)
7. heartbeat_thread.start()
```

`heartbeat_thread.start()` 启动了心跳线程, 它将按照设定的时间间隔 (这里是0.25秒) 发送心跳包 (MyRepeatThread在后文中会解释)。

目录

一、实验准备

二、实验过程描述

三、实验环境准备 (1学时)

四、工程文件目录结构

五、模型本地化训练及部署 (2学时)

六、逻辑实现 (5学时)

6.1 建立通信函数

6.2 发送心跳包

6.3 创建指令头

6.4 建立速度模型

6.5 图像处理函数

6.6 封装线程

6.7 避障主程序

6.7.1 导入依赖包

6.7.2 获取机器狗接收地址

6.7.3 发送指令以开启雷达

6.7.4 遍历避障动作

6.7.5 更新行走路线参数

6.7.6 定义全局变量和局部变量

6.8 识别和获取避障信息

七、运行示例

6.3 创建指令头

```
1. action_CommandHead = CommandHead()
```

创建了一个CommandHead对象，这个对象包含了指令的头部信息，如动作代码、序列号等。

类的继承结构

这些类都继承自CommandHead，它们共享一些基本属性和方法。CommandHead定义了基本信息，而他的子类定义了用于解析命令头部的方法和属性。

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. import ctypes
5. import struct
6. import inspect
7.
8. class CommandHead:
9.     def __init__(self, code=0, parameters_size=0, type_=0):
10.         self.code = code                    # 指令码
11.         self.parameters_size = parameters_size # 指令值
12.         self.type_ = type_                  # 指令类型
13.
14. class Command:
15.     kDataSize = 256
```

6.3 创建指令头

JointStateReceived 类

作用：这个类用于接收关节状态的原始数据，并且解析出头部信息。

解析过程：使用`struct.unpack`从数据的前4个字节解析出命令字（`code`）。

设计意图：通过继承`CommandHead`，`JointStateReceived`可以利用基类中定义的方法来处理命令头信息。

```
1. class JointStateReceived(CommandHead):
2.     def __init__(self, data):
3.         """
4.
5.         1. 关节信息的头部信息为4字节命令字,4字节长度,4字节类型,后续为正文数据---(后续的所有都是类似)
6.
7.         2. 1字节(b),2字节无符号短整型(H),4字节的无符号整型(I),接着是12个8字节的双精度浮点数(12d)
8.
9.         3. code, parameters_size, type_ 的是继承于CommandHead头的,这样写是为了方便呈现出,数据都是基于CommandHead发送和接收的
10.        这样可以做到避免所有的复杂数据全部发在相同名字的Command中,而是重新定义一个基于CommandHead的结构体
11.
12.        4. "self.code, " 逗号是为了去除掉由 struct.unpack 操作产生的元组问题,可以去去掉逗号直接复制
13.
14.        """
15.        self.data = data          # 关节的状态数据, 全部接受, 再由code来决定分配给谁
16.        CommandHead.code, = struct.unpack('<I', self.data[:4])
```


6.3 创建指令头

JointAngle 类

作用：专门用于从关节状态数据中解析出关节的角度。

解析过程：构造函数接收一个JointStateReceived实例，然后从实例的数据中（跳过前12个字节的头部信息）解析出12个双精度浮点数，这些数表示关节的角度。

JointSpeed 类

作用：与JointAngle类似，但用于解析关节的速度。

解析过程：同样接收一个JointStateReceived实例，并从数据中解析出12个双精度浮点数表示关节速度。

```
1. class JointAngle(CommandHead):
2.     def __init__(self, joint_state_received):
3.         # 特定于关节角度的数据解析
4.         *self.joint_angles, = struct.unpack('<12d', joint_state_received.data[12:])
5.
6. class JointSpeed(CommandHead):
7.     def __init__(self, joint_state_received):
8.         # 特定于关节速度的数据解析
9.         *self.joint_speeds, = struct.unpack('<12d', joint_state_received.data[12:])
```

6.3 创建指令头

RobotState 类

作用：解析机器人的整体状态，包括运动状态、步态信息、动作状态、雷达距离等。

解析过程：

首先解析命令头信息，包括命令码、参数大小和类型。

然后解析机器人的基本运动状态、步态信息、动作状态等，每个状态占用4个字节。

```
1. class RobotState(CommandHead):
2.     def __init__(self, data):
3.
4.         CommandHead.code, = struct.unpack('<I', data[:4])
5.         CommandHead.parameters_size, = struct.unpack('<I', data[4:8])
6.         CommandHead.type_, = struct.unpack('<I', data[8:12])
7.         self.robot_basic_state, = struct.unpack('<I', data[12:16])           # 机器人基本运动状态
8.         self.robot_gait_state, = struct.unpack('<I', data[16:20])           # 机器人步态信息
9.         self.robot_motion_state, = struct.unpack('<i', data[176:180])       # 机器人动作状态
10.        self.distance_ahead, = struct.unpack('<d', data[-16:-8])           # 雷达前方的距离
11.        self.rear_distance, = struct.unpack('<d', data[-8:])
```

目录

一、实验准备

二、实验过程描述

三、实验环境准备 (1学时)

四、工程文件目录结构

五、模型本地化训练及部署 (2学时)

六、逻辑实现 (5学时)

6.1 建立通信函数

6.2 发送心跳包

6.3 创建指令头

6.4 建立速度模型

6.5 图像处理函数

6.6 封装线程

6.7 避障主程序

6.7.1 导入依赖包

6.7.2 获取机器狗接收地址

6.7.3 发送指令以开启雷达

6.7.4 遍历避障动作

6.7.5 更新行走路线参数

6.7.6 定义全局变量和局部变量

6.8 识别和获取避障信息

七、运行示例

6.4 建立速度模型

这个函数用于控制设备直线行驶。

```
1. def go_straight(long, speedgear=3, times = None, obs_void_distance = None):
2.     # 档位速度只是参考值, 具体速度按照实际来判断, 默认速度是3档
3.     speedgear_ditc = {
4.         . . . . .
10.     }
11.
12.     val = speedgear_ditc[speedgear]
13.     # 如果传入的是时间, 计算出已经走过的距离, Long传入9999是为在特殊情况才能会有下面的情况
14.     if times != None and obs_void_distance != None and long == 9999:
15.         speed_per_second_meter = (-7.237e-09) * val ** 2 + 0.0002933 * val - 1.66
16.         # 这里要计算出避障的距离的时间消耗
17.         times_obs_void_distance = obs_void_distance / speed_per_second_meter
18.
19.         long = abs((times - times_obs_void_distance) * speed_per_second_meter)
20.
21.         return long
22.
23.     else:
24.         if long < 0:
25.             val = -val
26.             speed_per_second_meter = (-1.5059e-08) * val ** 2 - (4.1944e-04) * val - 2.1804
27.             times = abs(long / speed_per_second_meter)
28.         else:
29.             speed_per_second_meter = (-7.237e-09) * val ** 2 + 0.0002933 * val - 1.66
30.             times = long / speed_per_second_meter
31.
32.     return [times, val]
```

函数首先定义了一个字典speedgear_ditc, 它将档位映射到一个速度值

函数返回行驶所需的时间和速度值

6.4 建立速度模型

```
1. def translate_left_and_right(long, speedgear=3):
2.     # 档位速度只是参考值, 具体速度按照实际来判断, 默认速度是3档
3.     speedgear_ditc = {
4.         1: 14000,
5.         2: 18000,
6.         3: 21000,
7.         4: 24000,
8.         5: 27000,
9.         6: 34000,
10.    }
11.    val = speedgear_ditc[speedgear]
12.    if long < 0:
13.        val = -val
14.        speed_per_second_meter = -(1.6e-05) * val - 0.2256
15.        times = abs(long / speed_per_second_meter)
16.    else:
17.        speed_per_second_meter = (1.517e-05) * val - 0.1748
18.        times = long / speed_per_second_meter
19.    return [times, val]
```

这个函数用于控制设备左右平移

根据long的正负, 选择相应的速度计算公式来计算时间

6.4 建立速度模型

revolve_left_and_right(angle) :

这个函数用于控制设备左右旋转。函数内部定义了一个辅助函数find_closest_output，用于找到最接近输入角度的输出值。还有一个get_formatted_stack_info方法，用于确定是哪个文件调用了旋转函数

angle: 表示要旋转的角度

```
1. def revolve_left_and_right(angle):
2.     def find_closest_output(input_value):
3.         # 判断角度是否超过了360度, 超过了则需要则算掉
4.         if input_value > 360:
5.             input_value = input_value / (input_value % 360)
6.
7.         # 输出到输入的映射字典
8.         output_to_input_map = {
9.             .....
35.         }
39.         # 获取所有输出键并将其转换为列表
40.         keys = list(output_to_input_map.keys())
41.         # 寻找与输入值最接近的输出键
42.         closest_key = min(keys, key=lambda x: abs(x - input_value))
43.         # 返回与最接近键关联的输入值
44.         return output_to_input_map[closest_key]
45.
46.     caller_function = StackTraceParser()
47.     # print(caller_function.get_formatted_stack_info())
48.
49.     # 根据栈的追踪, 如果旋转的调用执行不同的旋转方式
50.     if caller_function.get_stack_info()[0]['file'].split('/')[1] == 'hand_distance_main.py':
65.         .....
66.     else:
67.         times = find_closest_output(abs(angle)) # 获取时间与角度的关系
68.         if angle < 0:
69.             val = -10000
70.         else:
71.             val = 10000
72.
73.     return [times, val]
```

将角度映射到一个输出值

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备 (1学时)
- 四、工程文件目录结构
- 五、模型本地化训练及部署 (2学时)
- 六、逻辑实现 (5学时)
 - 6.1 建立通信函数
 - 6.2 发送心跳包
 - 6.3 创建指令头
 - 6.4 建立速度模型
 - 6.5 图像处理函数
 - 6.6 封装线程
 - 6.7 避障主程序
 - 6.7.1 导入依赖包
 - 6.7.2 获取机器狗接收地址
 - 6.7.3 发送指令以开启雷达
 - 6.7.4 遍历避障动作
 - 6.7.5 更新行走路线参数
 - 6.7.6 定义全局变量和局部变量
 - 6.8 识别和获取避障信息
- 七、运行示例

6.5 图像处理函数

letterbox: 这个函数用于将图像调整到指定的新尺寸，同时保持长宽比，通过添加边框来填充图像。它还计算了缩放比例和填充尺寸。

```
1. def letterbox(img, new_shape=(640, 640), color=(114, 114, 114), auto=False, scaleFill=False, scaleup=True):
2.     # 获取当前图像的尺寸 [height, width]
3.     shape = img.shape[:2]
4.     # 如果 new_shape 是一个整数，则将其转换为一个元组 (new_shape, new_shape)
5.     if isinstance(new_shape, int):
6.         new_shape = (new_shape, new_shape)
7.     # 计算新旧尺寸的比例，取宽度和高度比例的最小值作为缩放比例
8.     r = min(new_shape[0] / shape[0], new_shape[1] / shape[1])
9.     # 如果 scaleup 为 False，则限制 r 的值不超过 1，即不放大图像
10.    if not scaleup:
11.        r = min(r, 1.0)
12.    # 计算缩放后的图像尺寸
13.    new_unpad = int(round(shape[1] * r)), int(round(shape[0] * r))
14.    # 计算宽度和高度的填充量
15.    dw, dh = new_shape[1] - new_unpad[0], new_shape[0] - new_unpad[1] # wh padding
16.    # 如果 auto 为 True，则使用最小矩形，填充量应为 64 的倍数
17.    if auto:
18.        dw, dh = np.mod(dw, 64), np.mod(dh, 64) # wh padding
19.    # 如果 scaleFill 为 True，则不进行缩放，直接使用新尺寸填充
20.    elif scaleFill:
21.        dw, dh = 0.0, 0.0
22.        new_unpad = (new_shape[1], new_shape[0]) # 使用新尺寸
23.        ratio = new_shape[1] / shape[1], new_shape[0] / shape[0] # 重新计算宽高比例
24.    # 将填充量分为上下和左右两部分
25.    dw /= 2
26.    dh /= 2
27.    # 如果需要调整图像尺寸，则进行缩放
28.    if shape[0] != new_unpad[0]:
29.        img = cv2.resize(img, new_unpad, interpolation=cv2.INTER_LINEAR)
30.    # 计算上、下、左、右边界的具体填充量，并进行四舍五入
31.    top, bottom = int(round(dh - 0.1)), int(round(dh + 0.1))
32.    left, right = int(round(dw - 0.1)), int(round(dw + 0.1))
33.    # 使用 cv2.copyMakeBorder 函数添加边框，颜色为 color 指定的颜色
34.    img = cv2.copyMakeBorder(img, top, bottom, left, right, cv2.BORDER_CONSTANT, value=color)
35.    # 返回调整尺寸并添加边框后的图像，缩放比例，以及左右和上下的填充量
36.    return img, ratio, (dw, dh)
```


6.5 图像处理函数

`non_max_suppression`: 这个函数实现了非极大值抑制（NMS），用于从预测结果中去除重叠的检测框。它接受多个参数，包括置信度阈值、IoU阈值等，并返回经过NMS处理的检测结果。

```
1. def non_max_suppression(  
2.     prediction, # 预测结果, 是模型输出的张量  
3.     conf_thres=0.25, # 置信度阈值, 默认为0.25  
4.     iou_thres=0.45, # 交并比 (IoU) 阈值, 默认为0.45  
5.     classes=None, # 指定要检测的类别列表, 如果为None, 则检测所有类别  
6.     agnostic=False, # 是否进行类别不可知的NMS  
7.     multi_label=False, # 是否允许单个实例有多个标签  
8.     labels=(), # 可选的标签列表, 用于某些模型的自动标签注入  
9.     max_det=300, # 每张图像上最多检测的目标数量  
10.    nm=0, # 掩码数量, 当前未使用  
11. ):
```

6.5 图像处理函数

`xywh2xyxy`: 将坐标格式从(x, y, w, h)转换为(x1, y1, x2, y2), 即从中心点和宽高转换为左上角和右下角的坐标。

```
1. # 定义 xywh2xyxy 函数, 用于将 nx4 边界框从 [x, y, w, h] 格式转换为 [x1, y1, x2, y2] 格式
2. # 其中 xy1 表示左上角坐标, xy2 表示右下角坐标
3. def xywh2xyxy(x):
4.     # 如果输入是 torch.Tensor 类型, 则克隆数据; 如果是 numpy 数组, 则复制数据
5.     y = x.clone() if isinstance(x, torch.Tensor) else np.copy(x)
6.     # 计算左上角 x 坐标
7.     y[:, 0] = x[:, 0] - x[:, 2] / 2
8.     # 计算左上角 y 坐标
9.     y[:, 1] = x[:, 1] - x[:, 3] / 2
10.    # 计算右下角 x 坐标
11.    y[:, 2] = x[:, 0] + x[:, 2] / 2
12.    # 计算右下角 y 坐标
13.    y[:, 3] = x[:, 1] + x[:, 3] / 2
    return y
```

6.5 图像处理函数

`get_labels_from_txt`: 从文本文件中读取类别标签，并将它们存储在字典中。

```
1. # 定义 get_labels_from_txt 函数，用于从文本文件中读取标签并创建字典
2. def get_labels_from_txt(path):
3.     labels_dict = dict() # 初始化一个空字典来存储标签
4.     with open(path) as f: # 打开文件
5.         for cat_id, label in enumerate(f.readlines()): # 遍历文件中的每一行
6.             labels_dict[cat_id] = label.strip() # 将读取的标签去除两端空白后存入字典
7.     return labels_dict # 返回标签字典
```

6.5 图像处理函数

scale_coords: 将坐标从一个图像尺寸调整到另一个图像尺寸，考虑了缩放和平移。

```
1. # 定义 scale_coords 函数，用于调整坐标点以适应不同尺寸的图像
2. def scale_coords(img1_shape, coords, img0_shape, ratio_pad=None):
3.     # 根据 img1_shape 和 img0_shape 调整坐标
4.     if ratio_pad is None: # 如果没有提供 ratio_pad，则从 img0_shape 计算
5.         gain = min(img1_shape[0] / img0_shape[0], img1_shape[1] / img0_shape[1]) # 计算缩放比
6.         pad = (img1_shape[1] - img0_shape[1] * gain) / 2, (img1_shape[0] -
7.         img0_shape[0] * gain) / 2 # 计算填充量
8.     else: # 如果提供了 ratio_pad，则使用提供的值
9.         gain = ratio_pad[0][0]
10.        pad = ratio_pad[1]
11.        # 应用 x 方向的填充
12.        coords[:, [0, 2]] -= pad[0]
13.        # 应用 y 方向的填充
14.        coords[:, [1, 3]] -= pad[1]
15.        # 应用缩放比例
16.        coords[:, :4] /= gain
17.        # 裁剪坐标点，确保它们在有效范围内
18.        clip_coords(coords, img0_shape)
    return coords # 返回调整后的坐标点
```

6.5 图像处理函数

`clip_coords`: 将坐标限制在给定图像尺寸的范围內，确保检测框不会超出图像边界。

```
1. def clip_coords(boxes, shape):
2.     # 如果 boxes 是 torch.Tensor 类型，则逐个快速裁剪
3.     if isinstance(boxes, torch.Tensor):
4.         boxes[:, 0].clamp_(0, shape[1]) # 裁剪 x1 坐标，确保在 0 到图像宽度之间
5.         boxes[:, 1].clamp_(0, shape[0]) # 裁剪 y1 坐标，确保在 0 到图像高度之间
6.         boxes[:, 2].clamp_(0, shape[1]) # 裁剪 x2 坐标，确保在 0 到图像宽度之间
7.         boxes[:, 3].clamp_(0, shape[0]) # 裁剪 y2 坐标，确保在 0 到图像高度之间
8.     # 如果 boxes 是 numpy 数组，则整体快速裁剪
9.     else:
10.         boxes[:, [0, 2]] = boxes[:, [0, 2]].clip(0, shape[1]) # 裁剪 x1 和 x2
            boxes[:, [1, 3]] = boxes[:, [1, 3]].clip(0, shape[0]) # 裁剪 y1 和 y2
```

6.5 图像处理函数

nms: 一个包装函数，用于调用non_max_suppression函数并返回NMS处理后的检测框。

```
1. # 定义 nms 函数，作为非极大值抑制（NMS）的封装，调用 non_max_suppression 函数
2. def nms(box_out, conf_thres=0.4, iou_thres=0.5):
3.     try:
4.         # 尝试使用 multi_label=True 的 NMS
5.         boxout = non_max_suppression(box_out, conf_thres=conf_thres, iou_thres=iou_thres, multi_label=True)
6.     except:
7.         # 如果出错，则使用默认的 NMS
8.         boxout = non_max_suppression(box_out, conf_thres=conf_thres, iou_thres=iou_thres)
9.     return boxout # 返回 NMS 处理后的结果
```

6.5 图像处理函数

`draw_bbox`: 在图像上绘制检测框，并在框旁边显示类别名称和置信度。

```
1. def draw_bbox(bbox, img0, color, wt, names):
2.     det_result_str = '' # 初始化检测结果字符串
3.     for idx, class_id in enumerate(bbox[:, 5]): # 遍历边界框和类别 ID
4.         if float(bbox[idx][4]) < float(0.05): # 如果置信度小于 0.05, 则跳过
5.             continue
6.         print(str(idx) + ' ' + names[int(class_id)]) # 打印边界框索引和类别名称
7.         # 在图像上绘制矩形边界框
8.         img0 = cv2.rectangle(img0, (int(bbox[idx][0]), int(bbox[idx][1])), (int(bbox[idx][2]), i
           nt(bbox[idx][3])), color, wt)
9.         # 在边界框左上角绘制类别名称
10.        img0 = cv2.putText(img0, str(idx) + ' ' + names[int(class_id)], (int(bbox[idx][0]), int(
           bbox[idx][1] + 16)), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1)
11.        # 在边界框下方绘制置信度
12.        img0 = cv2.putText(img0, '{:.2f}'.format(bbox[idx][4]), (int(bbox[idx][0]), int(bbox[idx][
           1] + 32)), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1)
13.        # 将检测结果添加到字符串
14.        det_result_str += '{} {} {} {} {} {} \n'.format(names[bbox[idx][5]], str(bbox[idx][4]), bbox
           [idx][0], bbox[idx][1], bbox[idx][2], bbox[idx][3])
15.    return img0 # 返回绘制了边界框和标签的图像
```

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备 (1学时)
- 四、工程文件目录结构
- 五、模型本地化训练及部署 (2学时)
- 六、逻辑实现 (5学时)
 - 6.1 建立通信函数
 - 6.2 发送心跳包
 - 6.3 创建指令头
 - 6.4 建立速度模型
 - 6.5 图像处理函数
 - 6.6 封装线程
 - 6.7 避障主程序
 - 6.7.1 导入依赖包
 - 6.7.2 获取机器狗接收地址
 - 6.7.3 发送指令以开启雷达
 - 6.7.4 遍历避障动作
 - 6.7.5 更新行走路线参数
 - 6.7.6 定义全局变量和局部变量
 - 6.8 识别和获取避障信息
- 七、运行示例

6.6 封装线程

ThreadTemplates.py定义了一个名为 `MyRepeatThread` 的自定义线程类，用于执行周期性动作，并在达到时间阈值或特定条件时停止。它包含了日志记录、属性打印、时间检查和停止方法。代码还定义了三个动作函数（前进、平移、旋转），每个函数创建一个`MyRepeatThread` 线程实例以执行对应的机器狗动作。这些动作函数被组织在`actions_dict` 字典中，便于根据动作 ID 快速索引。此外，还有一个 `action_the_pitch_angle` 函数用于调整机器狗的俯仰角。所有线程实例在创建时都不会立即启动，而是需要外部显式调用`start()` 方法。

```
1. class MyRepeatThread(threading.Thread):
2.     # MyRepeatThread 类的构造函数, 接受线程名称、动作函数、时间间隔和其他参数
3.     def __init__(self, name, action, interval, time_limit = None, *args) -> None:
4.         super(MyRepeatThread, self).__init__() # 调用父类构造函数
5.         self.name = name # 线程名称
6.         self.action = action # 线程要执行的动作函数
7.         self.interval = interval # 执行动作的时间间隔
8.         self.time_limit = time_limit # 线程执行的最大时间阈值
9.         self.args = args # 传递给动作函数的参数
10.        self.stopped = threading.Event() # 创建一个线程停止事件
11.        self.start_time = time.monotonic() # 记录线程开始执行的时间
12.        self.global_var = 0 # 用于存储全局变量
13.        self.current_time = time.monotonic() # 记录当前时间
14.        self.current_time_start = 0 # 记录动作开始的时间
```

6.6 封装线程

```
1. # 线程的 run 方法, 定义了线程要执行的代码
2.     def run(self) -> None:
3.         logging.info(f"Starting {self.name}") # 记录线程开始的信息
4.         try:
5.             while not self.stopped.is_set(): # 如果线程停止事件未被设置, 则继续循环
6.                 self.current_time = time.monotonic() # 更新当前时间
7.
8.                 # 如果设置了时间阈值并且当前时间超过时间阈值, 则停止线程
9.                 if self.time_limit is not None and self.check_time_and_stop(self.current_time):
10.                     break
11.
12.                 try:
13.                     self.action(*self.args) # 执行动作函数, 并将参数展开
14.                 except KeyboardInterrupt:
15.                     self.stopped = True # 如果遇到 KeyboardInterrupt 异常, 则设置停止事件
16.                 finally:
17.                     pass # finally 块通常用于执行无论 try 块是否成功的操作, 这里没有具体操作
18.
19.                 # 计算自动动作开始以来经过的时间, 并计算需要休眠多长时间以保持固定频率
20.                 action_start_time = time.monotonic()
21.                 elapsed_time = action_start_time - self.current_time
22.                 time_to_wait = max(0, self.interval - elapsed_time)
23.                 time.sleep(time_to_wait) # 休眠以保持固定频率
24.
25.         finally:
26.             self.stopped.set() # 确保线程停止事件被设置
27.             logging.info(f"Exiting thread: {self.name}") # 记录线程退出的信息
```

这部分主要是写了运行线程之后对于时间计算

6.6 封装线程

```
1. # 定义检查是否超过时间阈值并停止线程的方法
2. def check_time_and_stop(self, current_time) -> bool:
3.     # 如果当前时间与线程开始时间的差值超过时间阈值, 则记录日志信息, 设置停止事件, 并返回True
4.     if current_time - self.start_time > self.time_limit:
5.         logging.info(f'{self.name}由于超过时间阈值{self.time_limit}秒, 系统自动停止! ')
6.         self.stopped.set()
7.         return True
8.     # 如果全局变量 global_var 等于 1, 则设置停止事件, 并返回True
9.     elif self.global_var == 1:
10.         self.stopped.set()
11.         return True
12.     # 如果以上条件都不满足, 则返回False
13.     return False
14.
15. # 定义停止线程的方法
16. def stop(self) -> None:
17.     # 设置停止事件, 停止线程
18.     self.stopped.set()
19.
20. # 定义打印对象所有属性及其值的方法
21. def print_attributes(self) -> dict:
22.     # 更新当前时间开始时间
23.     self.current_time_start = time.monotonic() - self.start_time
24.     # 返回一个字典, 包含对象的所有属性和值
25.     return {attr: value for attr, value in vars(self).items()}
```

check_time_and_stop()函数式用来停止线程的, print_attributes()主要是为了输出线程所有相关的信息

目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

6.1 建立通信函数

6.2 发送心跳包

6.3 创建指令头

6.4 建立速度模型

6.5 图像处理函数

6.6 封装线程

6.7 避障主程序

6.7.1 导入依赖包

6.7.2 获取机器狗接收地址

6.7.3 发送指令以开启雷达

6.7.4 遍历避障动作

6.7.5 更新行走路线参数

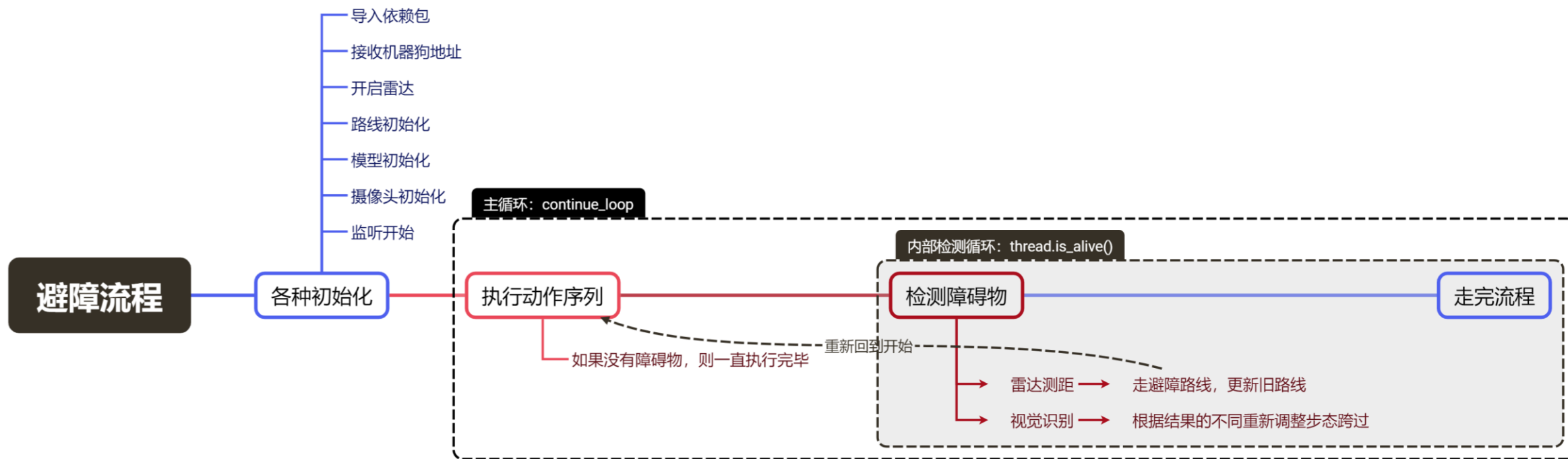
6.7.6 定义全局变量和局部变量

6.8 识别和获取避障信息

七、运行示例

6.7 避障主程序

obstacle_main.py是一个用于控制机器狗的自动化脚本，它集成了网络通信、多线程控制、避障、楼梯和坑洞识别，以及状态管理。脚本通过发送指令控制机器狗，实现自主导航，并能根据环境反馈调整行动路线。主要功能包括初始化通信、启动监听和处理线程、执行避障动作、处理特殊地形（楼梯和狗洞），以及根据机器狗状态发送相应指令。



目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备 (1学时)
- 四、工程文件目录结构
- 五、模型本地化训练及部署 (2学时)
- 六、逻辑实现 (5学时)
 - 6.1 建立通信函数
 - 6.2 发送心跳包
 - 6.3 创建指令头
 - 6.4 建立速度模型
 - 6.5 图像处理函数
 - 6.6 封装线程
 - 6.7 避障主程序
 - 6.7.1 导入依赖包
 - 6.7.2 获取机器狗接收地址
 - 6.7.3 发送指令以开启雷达
 - 6.7.4 遍历避障动作
 - 6.7.5 更新行走路线参数
 - 6.7.6 定义全局变量和局部变量
 - 6.8 识别和获取避障信息
- 七、运行示例

6.7.1 导入依赖包

它导入了多个模块以支持线程管理、通信、命令发送和状态监听。脚本设置了日志记录、全局变量和锁，定义了避障距离限制和行动路线。脚本还包括了避障动作参数和序列，以及用于控制线程暂停和继续的事件对象。整体上，这段代码构成了机器狗自动导航和避障功能的基础架构。

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4.
5. import threading
6. import time
7. import logging
8. import math
9. from contextlib import ExitStack
10.
11. # 导入定义的机器狗相关的包
12. from threading_utils.ThreadTemplates import * # 线程的模板与基本轴指令的发送
13. from sendcommand import SendToCommand, heartbeat # 发送简单指令与心跳包
14. from socketnetwork import network_utils # 通信函数
15. from command.udp_command import * # 存放各种结构体与状态数据、指令
16. from speeds.sportspeed import * # 运动精准控制的基础版模型
17. from robotstatuswatcher.listener import * # 监听机器狗状态
18. from HandDistance.ridge_np import * # 测量距离
19. from obstacle_model_cap import *
```

6.7.1 导入依赖包

```
1. # 初始化日志, 级别为最低级
2. logging.basicConfig(level=logging.DEBUG)
4. # 全局变量和锁
5. status_list = []
6. status_list_lock = threading.Lock()
7. result = []
8. result_lock = threading.Lock()
10. # 避障的距离限制
11. obs_void_distance = 0.4
13. # 固定路线
14. actions_params = {
15.     1: [(2,), (1.1,)],
16.     3: [(90,)]
17. }
18. actions_sequence = [(1, 0), (3, 0), (1, 1)]
19.
20. # 避障路线
21. avoid_actions_params = {1: [(0.7,), (0.7,)], 3: [(-46.0,), (91.0,), (-30.0,)]}
25. avoid_actions_sequence = [(3, 0), (1, 0), (3, 1), (1, 1), (3, 2)]
27. # 更新后的路线
28. updated_actions_params = actions_params.copy()
29.
30. # 避障次数
31. obstacle = 0
32. staircase = 0
33. hole = 0
35. # 记录队列的角标
36. sequence_index = 0
38. # 循环控制变量
39. continue_loop = True
41. # 创建一个全局的 Event 对象用于控制线程暂停和继续
42. pause_event = threading.Event()
43. pause_event.set() # 初始状态为设置, 表示不暂停
```

定义了所有变量的初始化, 包括一些路线的规定和控制循环的标志位。

目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

6.1 建立通信函数

6.2 发送心跳包

6.3 创建指令头

6.4 建立速度模型

6.5 图像处理函数

6.6 封装线程

6.7 避障主程序

6.7.1 导入依赖包

6.7.2 获取机器狗接收地址

6.7.3 发送指令以开启雷达

6.7.4 遍历避障动作

6.7.5 更新行走路线参数

6.7.6 定义全局变量和局部变量

6.8 识别和获取避障信息

七、运行示例

6.7.2 获取机器狗接收地址

这段代码定义了一个名为`initialize_communication` 的函数，其主要作用是初始化网络通信。函数通过调用`network_utils.setup_socket_and_address` 方法设置套接字和目标地址，然后创建并启动一个心跳包发送线程，以保持与机器狗的持续通信。最后，函数返回初始化的套接字和目标地址。

```
1. def initialize_communication():
2.     # 设置网络通信, 初始化套接字和目标地址
3.     sfd, target_address = network_utils.setup_socket_and_address()
4.     # 创建并启动心跳包发送线程
5.     heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_u
6.         dp_heartbeat, 0.25, None, sfd, target_address)
7.     heartbeat_thread.start()
8.     # 返回套接字和目标地址
9.     return sfd, target_address
```

目录

一、实验准备

二、实验过程描述

三、实验环境准备（1学时）

四、工程文件目录结构

五、模型本地化训练及部署（2学时）

六、逻辑实现（5学时）

6.1 建立通信函数

6.2 发送心跳包

6.3 创建指令头

6.4 建立速度模型

6.5 图像处理函数

6.6 封装线程

6.7 避障主程序

6.7.1 导入依赖包

6.7.2 获取机器狗接收地址

6.7.3 发送指令以开启雷达

6.7.4 遍历避障动作

6.7.5 更新行走路线参数

6.7.6 定义全局变量和局部变量

6.8 识别和获取避障信息

七、运行示例

6.7.3 发送指令以开启雷达

这段代码定义了一个名为initialize_threads 的函数，负责初始化并启动多个线程。函数首先发送指令以开启机器狗的雷达功能，然后创建并启动两个线程：一个是雷达状态监听线程，用于持续监听雷达状态；另一个是图像识别处理线程，用于执行图像推理任务。这些线程的创建和启动为机器狗的状态监控和环境感知提供了并行处理能力。

```
1. def initialize_threads():
2.     # 向机器狗发送指令以开启雷达
3.     SendToCommand.perform_action(sfd, target_address, 0x21012109, 0x40, 0)
4.     # 创建雷达状态监听线程并启动
5.     radar_thread = threading.Thread(target=status_listener_radar, args=(status_list, status_
        list_lock))
6.     radar_thread.start()
7.     # 创建图像识别处理线程并启动
8.     picture_thread = threading.Thread(target=inference_loop, args=(result, result_lock))
9.     picture_thread.start()
```

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备 (1学时)
- 四、工程文件目录结构
- 五、模型本地化训练及部署 (2学时)
- 六、逻辑实现 (5学时)
 - 6.1 建立通信函数
 - 6.2 发送心跳包
 - 6.3 创建指令头
 - 6.4 建立速度模型
 - 6.5 图像处理函数
 - 6.6 封装线程
 - 6.7 避障主程序
 - 6.7.1 导入依赖包
 - 6.7.2 获取机器狗接收地址
 - 6.7.3 发送指令以开启雷达
 - 6.7.4 遍历避障动作
 - 6.7.5 更新行走路线参数
 - 6.7.6 定义全局变量和局部变量
 - 6.8 识别和获取避障信息
- 七、运行示例

6.7.4 遍历避障动作

这段代码定义了一个名为`execute_avoid_sequence`的函数，其作用是执行避障动作序列。函数通过遍历避障动作参数和序列，获取每个动作的函数和参数。如果参数是元组类型，函数将创建并启动一个新线程来执行相应的避障动作，并等待该线程完成。此外，函数在执行前后打印日志信息以标明避障动作的开始和结束。整个过程用于在遇到障碍物时控制机器狗进行避障。

```
1. def execute_avoid_sequence(params=avoid_actions_params, sequence=avoid_actions_sequence):
2.     logging.info("进入 execute_avoid_sequence")
3.     logging.info("-----params-----: %s", params)
4.     # 遍历避障动作序列
5.     for action_id, params_index in sequence:
6.         # 根据动作ID获取对应的动作函数
7.         action_func = actions_dict.get(action_id, None)
8.         if action_func:
9.             # 获取避障参数
10.            avoid_params = params[action_id][params_index]
11.            # 如果参数是元组类型，则创建并启动避障动作线程
12.            if isinstance(avoid_params, tuple):
13.                avoid_thread=action_func(*avoid_params)
14.                avoid_thread.start()
15.                avoid_thread.join()
16.                time.sleep(1)
17.     logging.info("离开 execute_avoid_sequence")
```

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备 (1学时)
- 四、工程文件目录结构
- 五、模型本地化训练及部署 (2学时)
- 六、逻辑实现 (5学时)
 - 6.1 建立通信函数
 - 6.2 发送心跳包
 - 6.3 创建指令头
 - 6.4 建立速度模型
 - 6.5 图像处理函数
 - 6.6 封装线程
 - 6.7 避障主程序
 - 6.7.1 导入依赖包
 - 6.7.2 获取机器狗接收地址
 - 6.7.3 发送指令以开启雷达
 - 6.7.4 遍历避障动作
 - 6.7.5 更新行走路线参数
 - 6.7.6 定义全局变量和局部变量
 - 6.8 识别和获取避障信息
- 七、运行示例

6.7.5 更新行走路线参数

这段代码定义了一个名为`update_route_params` 的函数，它用于更新机器狗的行走路线参数。函数接收当前线程、避障前的行走距离、临时变量作为参数，并从全局变量`updated_actions_params` 中获取和更新当前动作序列的参数。新的行走距离通过从原始参数中减去已行走的距离和临时变量来计算。最后，函数记录并打印更新后的路线参数。这个过程是为了在避障后重新计算并设置机器狗的下一步行动路线。

```
1. def update_route_params(thread, params, before_long, temp):
2.     # global updated_actions_params
3.     # 更新机器狗的行走路线参数
4.     # 这里updated_actions_params是全局变量，存储更新后的路线参数
5.     # 获取当前动作序列的索引和键
6.     current_params_index = actions_sequence[sequence_index]
7.     current_params_key, current_params_index = actions_sequence[sequence_index][0], actions_sequence[sequence_index][1]
8.     # 更新路线参数，计算新的行走距离
9.     updated_actions_params[current_params_key][current_params_index] = (params[0] - before_long - temp,)
10.    logging.info(f'更新后的路线updated_actions_params: {updated_actions_params}')
```


目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备 (1学时)
- 四、工程文件目录结构
- 五、模型本地化训练及部署 (2学时)
- 六、逻辑实现 (5学时)
 - 6.1 建立通信函数
 - 6.2 发送心跳包
 - 6.3 创建指令头
 - 6.4 建立速度模型
 - 6.5 图像处理函数
 - 6.6 封装线程
 - 6.7 避障主程序
 - 6.7.1 导入依赖包
 - 6.7.2 获取机器狗接收地址
 - 6.7.3 发送指令以开启雷达
 - 6.7.4 遍历避障动作
 - 6.7.5 更新行走路线参数
 - 6.7.6 定义全局变量和局部变量
 - 6.8 识别和获取避障信息
- 七、运行示例

6.7.6 定义全局变量和局部变量

```
1. def main_loop(sfd, target_address):
2.     # 定义全局变量和局部变量
3.     global continue_loop, sequence_index, actions_params, actions_sequence
4.     while continue_loop:
5.         continue_loop = False # 重置循环控制变量
6.         # 遍历动作序列
7.         for action_id, params_index in actions_sequence:
8.             # 根据动作ID获取对应的动作函数
9.             action_func = actions_dict.get(action_id, None)
10.            if action_func:
11.                # 获取动作参数
12.                params = actions_params[action_id][params_index]
13.                # 打印当前参数
14.                logging.info(f'params: {params}')
15.                # 如果参数是元组类型, 则创建线程执行动作
16.                if isinstance(params, tuple):
17.                    thread = action_func(*params)
18.                    thread.start() # 启动线程
19.                .....
20.            # 如果循环控制变量被设置为False, 则退出循环
21.            if not continue_loop:
22.                # 增加队列角标
23.                sequence_index += 1 # 成功走完一个, 队列角标加一
24.                thread.join()
25.                handle_status(sfd, target_address)
26.            # 如果循环控制变量被设置为True, 则跳出当前循环
27.            if continue_loop:
28.                break
29.            time.sleep(1)
```

main_loop 是主循环，遍历执行预定义的动作序列，处理避障、楼梯和坑洞，并根据状态更新行动路线。

6.7.6 定义全局变量和局部变量

```
1. def handle_obstacles(thread, params):
2.     # 定义全局变量
3.     global continue_loop, obstacle
4.     # 如果未遇到障碍物, 并且状态列表中有数据, 并且距离小于等于避障距离限制
5.     if obstacle == 0 and status_list and status_list[3] <= obs_void_distance:
6.         # 增加障碍物计数
7.         obstacle += 1
8.         # 执行直线前进动作, 参数为长距离, 当前时间, 避障距离限制
9.         before_long = go_straight(long=9999, times=thread.print_attributes()['current_time_start'], obs_void_distance=obs_void_distance)
10.        # 计算临时变量
11.        temp = 2 * avoid_actions_params[1][0][0] / math.sqrt(2)
12.        # 更新路线参数
13.        update_route_params(thread, params, before_long, temp)
14.        # 停止当前线程
15.        thread.stop()
16.        thread.join()
17.        # 打印停止信息
18.        logging.info('原始路线停止。')
19.        # 如果线程已停止
20.        if thread._is_stopped:
21.            # 执行避障序列
22.            execute_avoid_sequence()
23.        .....
24.        # 设置循环控制变量为True, 继续执行
25.        continue_loop = True
26.        # 清除暂停事件
27.        pause_event.clear()
```

handle_obstacles 处理避障逻辑, 更新路线参数, 并在遇到障碍时执行避障动作序列。

6.7.6 定义全局变量和局部变量

handle_staircases 和 handle_holes 分别处理楼梯和坑洞的检测，发送指令以调整机器狗的行为。执行避障动作序列。

```
1. def handle_staircases(sfd, target_address):
2.     # 定义全局变量
3.     global staircase
4.     # 如果未遇到楼梯，并且结果列表为['vegetables']，表示检测到楼梯
5.     if staircase == 0 and result == ['vegetables']:
6.         # 增加楼梯计数
7.         staircase += 1
8.         # 打印检测到楼梯的信息
9.         logging.info("Detected 'vegetables', pausing thread.")
10.        # 发送指令以暂停机械狗
11.        SendToCommand.perform_action(sfd, target_address, 0x21010407, 0, 0)

1. def handle_holes(sfd, target_address):
2.     # 定义全局变量
3.     global hole
4.     # 如果未遇到坑洞，并且结果列表为['drink']，表示检测到坑洞
5.     if hole == 0 and result == ['drink']:
6.         # 增加坑洞计数
7.         hole += 1
8.         # 打印检测到坑洞的信息
9.         logging.info("Detected 'drink', pausing thread.")
10.        # 发送指令以暂停机械狗
11.        SendToCommand.perform_action(sfd, target_address, 0x21010406, 0, 0)
```

6.7.6 定义全局变量和局部变量

handle_status 根据机器狗的状态发送相应的控制指令。

```
1. def handle_status(sfd, target_address):
2.     # 定义全局变量
3.     global hole
4.     # print(status_listener())
5.     if status_listener() == [6, 13, 1]: # 力控状态 (静止站立) 且步态为高踏步越障步态
6.         SendToCommand.perform_action(sfd, target_address, 0x21010300, 0, 0)
7.         time.sleep(1)
8.     elif status_listener() == [6, 0, 1] and hole == 1: # 正在以平地低速步态踏步 (匍匐状态)
9.         # 增加坑洞计数
10.        hole += 1
11.        SendToCommand.perform_action(sfd, target_address, 0x21010406, 0, 0)
12.        time.sleep(1)
```

6.7.6 定义全局变量和局部变量

脚本最后检查 `__name__` 来确定是否作为主程序运行，并调用上述函数来启动整个过程。

```
1. if __name__ == "__main__":
2.     # 建立通讯地址
3.     sfd, target_address = initialize_communication()
4.     # 初始化模型、雷达、识别
5.     initialize_threads()
6.     # 站立
7.     SendToCommand.perform_action(sfd, target_address, 0x21010202, 0, 0)
8.     # 2秒延时保证机器狗站稳再开始执行任务
9.     time.sleep(2)
10.    # 进入主程序
11.    main_loop(sfd, target_address)
```

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备 (1学时)
- 四、工程文件目录结构
- 五、模型本地化训练及部署 (2学时)
- 六、逻辑实现 (5学时)
 - 6.1 建立通信函数
 - 6.2 发送心跳包
 - 6.3 创建指令头
 - 6.4 建立速度模型
 - 6.5 图像处理函数
 - 6.6 封装线程
 - 6.7 避障主程序
 - 6.7.1 导入依赖包
 - 6.7.2 获取机器狗接收地址
 - 6.7.3 发送指令以开启雷达
 - 6.7.4 遍历避障动作
 - 6.7.5 更新行走路线参数
 - 6.7.6 定义全局变量和局部变量
 - 6.8 识别和获取避障信息
- 七、运行示例

6.8 识别和获取避障信息

```
1. # 定义Image_inference函数, 用于执行图像推理
2. def Image_inference(model, labels_dict):
3.     try:
4.         img = getImage() # 获取图像
5.         frame = img.copy() # 复制图像用于后续处理
6.         # 使用Letterbox函数对图像进行尺寸调整和填充
7.         img, scale_ratio, pad_size = letterbox(frame, new_shape=[640, 640])
8.         ill_sets = [] # 初始化用于存储检测结果的列表
9.         # 对图像进行格式转换和归一化处理, 准备输入到模型中
10.        img = img[:, :, :-1].transpose(2, 0, 1)
11.        img = np.expand_dims(img, 0).astype(np.float32)
12.        img = np.ascontiguousarray(img) / 255.0
13.        img = Tensor(img)
14.        # 使用模型进行推理
15.        output = model.infer([img])[0]
16.        output.to_host() # 将推理结果从设备传输到主机
17.        output = np.array(output) # 将推理结果转换为NumPy数组
18.        # 使用nms函数进行非极大值抑制处理
19.        boxout = nms(torch.tensor(output), conf_thres=0.7, iou_thres=0.5)
20.        # 将nms处理后的预测结果转换为NumPy数组
21.        pred_all = boxout[0].numpy()
22.        # 调整坐标点以适应原始图像尺寸
23.        scale_coords([640, 640], pred_all[:, :4], frame.shape, ratio_pad=(scale_ratio, pad_size))
24.        # 遍历预测结果, 提取类别和置信度信息
25.        for idx, class_id in enumerate(pred_all[:, 5]):
26.            # 记录检测到的类别、类别ID和置信度
27.            ill_sets.append([labels_dict[int(class_id)], int(class_id), round(pred_all[idx][4], 2)])
28.        # 返回检测结果列表
29.        return ill_sets
30.    except KeyboardInterrupt:
31.        # 如果发生键盘中断 (Ctrl+C), 返回空列表
32.        return []
```

这个函数的作用是用来推理模型，同时输出识别结果的

6.8 识别和获取避障信息

初始化模型

```
1. # 定义model_init函数, 用于初始化模型和标签字典
2. def model_init(model_path, device_id, label_path):
3.     base.mx_init() # 初始化MindX SDK
4.     model = base.model(modelPath=model_path, deviceId=device_id) # 加载模型
5.     labels_dict = get_labels_from_txt(label_path) # 从标签文件中获取类别标签
6.     return model, labels_dict # 返回模型和标签字典
```

6.8 识别和获取避障信息

```
1. def inference_loop(result, result_lock):
2.     # 初始化一个空列表, 用于存储推理结果
3.     list0 = []
4.     # 初始化计数器
5.     k = 0
6.
7.     # 设备ID设置为0
8.     DEVICE_ID = 0
9.     # 模型路径设置为 'smart_logistics_models/1.om'
10.    model_path = 'smart_logistics_models/1.om'
11.    # 标签文件路径设置为 'smart_logistics_models/predefined_classes.txt'
12.    label_path = 'smart_logistics_models/predefined_classes.txt'
13.    # 调用 model_init 函数初始化模型和标签字典
14.    model, labels_dict = model_init(model_path, DEVICE_ID, label_path)
15.
16.    try:
17.        # 进入一个无限循环, 持续进行推理
18.        while True:
19.            # 调用 Image_inference 函数进行图像推理, 并获取结果
20.            ill_sets = Image_inference(model, labels_dict)
21.            # 如果推理结果为空, 继续下一次循环
22.            if not ill_sets:
23.                continue
24.            # 将推理结果中的第一个元素添加到列表 list0 中
25.            list0.append(ill_sets[0][1])
26.            # 计数器 k 加 1
27.            k += 1
```

用来初始化模型, 并且对模型的输出结果进行统计, 多次识别到同一个种类, 才会得到正确结果输出给机器狗

6.8 识别和获取避障信息

用来初始化模型，并且对模型的输出结果进行统计，多次识别到同一个种类，才会得到正确结果输出给机器狗

```
29.         # 如果计数器 k 等于 2，表示已经收集到两个连续的推理结果
30.         if k == 2:
31.             # 创建一个集合，用于存储 list0 中的唯一元素
32.             unique_elements = set(list0)
33.             # 如果集合的长度为 1，表示两次推理结果相同
34.             if len(unique_elements) == 1:
35.                 # 将对象类别设置为 list0 中的第一个元素
36.                 object_class = ill_sets[0][0]
37.                 # 重置计数器 k 和 list0
38.                 k, list0 = 0, []
39.                 # 使用结果锁进行线程安全操作
40.                 with result_lock:
41.                     # 清空结果列表
42.                     result.clear()
43.                     # 将新的结果添加到结果列表中
44.                     result.append(object_class)
45.                     # 打印当前结果
46.                     print('result:', result)
47.             # 如果集合长度不为 1，表示两次推理结果不一致
48.             else:
49.                 # 设置 object_class 为 None
50.                 object_class = None
51.                 # 重置计数器 k 和 list0
52.                 k, list0 = 0, []
53.
54.         # 如果捕获到 KeyboardInterrupt 异常（例如使用 Ctrl+C），不执行任何操作
55.     except KeyboardInterrupt:
56.         pass
```

目录

- 一、实验准备
- 二、实验过程描述
- 三、实验环境准备（1学时）
- 四、工程文件目录结构
- 五、模型本地化训练及部署（2学时）
- 六、逻辑实现（5学时）
- 七、运行示例

七、运行示例

首先在文件夹的目录下执行先执行这个指令

```
. /usr/local/Ascend/mxVision-5.0.RC3/set_env.sh && . /usr/local/Ascend/ascend-toolkit/set_env.sh
```

然后再输入：`python obstacle_main.py` 让机器狗识别到相对应的类别即可。

```
(base) root@davinci-mini:/opt/UDPControl# python obstacle_main.py
成功连接到ip: 192.168.1.100, port: 43897
Begin to initialize Log.
The output directory of logs file exist.
WARNING: Logging before InitGoogleLogging() is written to STDERR
I20220408 03:48:30.542012 20473 FileUtils.cpp:331] The input file is empty
I20220408 03:48:30.542076 20473 FileUtils.cpp:475] Check Other group permission: Current permission is 4, but required no greater than 0.
Save logs information to specified directory.
Find 1 devices!

u3v device: [0]
device model name: MV-CS060-10UC-PRO
user serial number: DA0275184
/usr/local/miniconda3/lib/python3.9/site-packages/torchvision/io/image.py:13: UserWarning: Failed to load image Python extension:
  warn(f"Failed to load image Python extension: {e}")
INFO:root:Starting HeartbeatThread
^Aresult: ['hole']
result: ['hole']
```

感谢

版权所有©2024，华为技术有限公司，保留所有权利。

本资料是华为的保密信息，所有内容仅供华为授权的培训客户内部使用，禁止用于任何其他用途。未经许可，任何人不得对本资料进行复制、修改、改编、也不得将本资料或其任何部分或基于本资料的衍生作品提供给他人。

华为开发者创新中心

<https://connect.huaweicloud.com>